БАЗА ДЛЯ РЕВЕРС-ИНЖЕНЕРА

Windows, Linux, embedded, firmware, процессоры, стек, ABI, компиляторы и инструменты

Универсальный фундаментальный учебник: от битов, памяти, регистров, функций и компиляции до PE, ELF, raw firmware, Cortex-M, embedded Linux, JTAG/SWD, Ghidra, отладки и методики проверяемого reverse engineering.

Версия проекта: 1.0.0
Составлено по фактическому исходному коду проекта
26 июня 2026 года

Содержание

База для реверс-инженера с нуля: Windows, Linux и embedded . . . 12
Предисловие к исправленному изданию . . . 12
Как читать эту книгу . . . 13
Этическая и практическая рамка . . . 13
Часть I. Нулевая база: из чего состоит машинный мир . . . 14
Глава 1. Байт: маленькая клетка, из которой строится все . . . 14
Глава 2. Память: почему адрес не равен месту в файле . . . 15
Глава 3. Процессор: кто исполняет байты . . . 16
Глава 4. Assembly: не язык демонов, а журнал изменений состояния . . . 17
Часть II. Функции, стек и ABI . . . 19
Глава 5. Что такое функция, когда исходного кода уже нет . . . 19
Глава 6. Стек: временная память, где программа помнит дорогу назад . . . 19
Глава 7. Windows x64 calling convention: договор Windows о вызове функций . . . 20
Глава 8. System V AMD64, ARM AAPCS и RISC-V ABI: тот же смысл, другие правила . . . 21
Часть III. Как исходный код превращается в бинарник . . . 23
Глава 9. Компилятор, assembler, linker, loader: четыре разных существа . . . 23
Глава 10. Оптимизации: почему бинарник не похож на исходник . . . 23
Глава 11. Символы, debug information и почему им нельзя слепо верить . . . 24
Часть IV. Форматы: PE, ELF и raw firmware . . . 26
Глава 12. PE-файл Windows: не просто EXE, а договор с загрузчиком . . . 26
Глава 13. ELF Linux: сегменты для загрузчика, секции для инструментов . . . 26
Глава 14. Raw firmware, Intel HEX, S-record, UF2 и контейнеры прошивок . . . 27
Часть V. Загрузка и выполнение . . . 29
Глава 15. Что делает loader, когда программа стартует . . . 29
Глава 16. Runtime: невидимая обвязка вокруг твоего кода . . . 29
Часть VI. Embedded: железо, прошивки и низкий уровень без ОС . . . 31
Глава 17. Что такое embedded и почему Windows-привычек мало . . . 31
Глава 18. Cortex-M: vector table, reset handler и interrupts . . . 31
Глава 19. Периферия: GPIO, UART, SPI, I2C, DMA и clocks . . . 32
Глава 20. RTOS: задачи, планировщик и контекст . . . 33
Часть VII. Инструменты реверсера . . . 34
Глава 21. Hex editor и strings: маленькие инструменты до тяжелой артиллерии . . . 34
Глава 22. Ghidra, IDA, Rizin, Binary Ninja: что делает дизассемблер . . . 34
Глава 23. Debugger: наблюдение живого поведения . . . 35
Глава 24. Emulation: когда железа нет или запуск опасен . . . 35
Часть VIII. Методика настоящего исследования . . . 36
Глава 30. Как строить дальнейший путь . . . 36
Часть X. Глубокие разборы: медленно, от начала до конца . . . 37
Глава 31. Один байт проходит путь от файла до инструкции . . . 37
Глава 32. Как PE-секция превращается в память процесса . . . 38
Глава 33. Windows x64 unwind: зачем программе описание собственного стека . . . 39
Глава 34. ELF-загрузка: почему sections не главное для kernel . . . 40

Глава 35. От C-структуры к offset-ам в дизассемблере . . . 40
Глава 36. Pointer: число, которое обещает быть адресом . . . 41
Глава 37. Interrupt, fault и exception: три слова, которые нельзя мешать в кучу . . . 42
Глава 38. Cortex-M reset: полный путь от питания до main . . . 42
Глава 39. MMIO-разбор по-настоящему: включаем GPIO . . . 43
Глава 40. UART как след в прошивке . . . 43
Глава 41. DMA: почему код не записывает буфер, а данные появляются . . . 44
Глава 42. Embedded Linux firmware: когда прошивка внутри похожа на маленький Linux . . . 44
Глава 43. Packing, compression, encryption: почему видимый код не всегда главный . . . 45
Глава 44. Как читать официальную документацию, чтобы не утонуть . . . 45
Глава 45. Как объяснять английские термины самому себе . . . 46
Часть XI. Практикум с разбором мышления . . . 47
Глава 46. Практикум: маленькая функция x64 до уровня регистров . . . 47
Глава 47. Практикум: switch и jump table . . . 47
Глава 48. Практикум: строка как повод найти функцию . . . 48
Глава 49. Практикум: восстановление структуры по offset-ам . . . 48
Глава 50. Практикум: маленький bootloader . . . 48
Глава 51. Практикум: FreeRTOS-задачи в бинарнике . . . 49
Глава 52. Практикум: распаковка embedded rootfs . . . 49
Глава 53. Практикум: оформление вывода для ВКР . . . 50
Часть XII. Частые тупики новичка и как из них выходить . . . 51
Глава 54. «Я ничего не понимаю в Ghidra» . . . 51
Глава 55. «Декомпилятор показывает бред» . . . 51
Глава 56. «Я нашел адрес, но не знаю, что он значит» . . . 51
Глава 57. «Я боюсь официальной документации» . . . 52
Глава 58. «Как понять, что я уже достаточно доказал?» . . . 52
Теоретическая редакция: что изменено . . . 53
Авторская опора на лучшие книги и документацию . . . 54
Eldad Eilam - Reversing: Secrets of Reverse Engineering . . . 54
Michael Sikorski, Andrew Honig - Practical Malware Analysis . . . 54
Dennis Andriesse - Practical Binary Analysis . . . 54
Chris Eagle - The IDA Pro Book . . . 54
Chris Eagle, Kara Nance - The Ghidra Book . . . 55
Windows Internals . . . 55
Bryant и O'Hallaron - Computer Systems: A Programmer's Perspective . . . 55
The Hardware Hacking Handbook . . . 55
Rootkits и Bootkits . . . 55
Официальные спецификации Microsoft, Intel, AMD, Arm, RISC-V, ELF gABI, DWARF, OpenOCD, QEMU . . . 56
Как читать эту книгу после такой опоры . . . 56
Часть I. Большая теоретическая база . . . 57
Теоретическая глава 1. Что такое реверс-инжиниринг как наука о слоях . . . 57
Теоретическая глава 2. Информация: бит, байт, число, знак и порядок байтов . . . 57
Теоретическая глава 3. Модель CPU: состояние, инструкция и переход времени . . . 58
Теоретическая глава 4. Память: адресные пространства, страницы, стек и куча . . . 58
Теоретическая глава 5. Функции и ABI: договор, без которого регистры молчат . . . 59

Теоретическая глава 6. Компилятор: почему бинарник не обязан быть похожим на исходник . . . 60
Теоретическая глава 7. Форматы бинарных файлов: договор между producer и loader . . . 60
Теоретическая глава 8. Loader, runtime и операционная система . . . 61
Теоретическая глава 9. Данные и типы: структуры, указатели, массивы и объекты . . . 61
Теоретическая глава 10. Control flow, data flow и смысл программы . . . 62
Теоретическая глава 11. Windows-фундамент для твоей ВКР: PE, x64 ABI и unwind . . . 62
Теоретическая глава 12. Linux и ELF: другой мир, те же фундаментальные роли . . . 63
Теоретическая глава 13. Embedded-фундамент: firmware, memory map, MMIO, interrupts, DMA . . . 63
Теоретическая глава 14. Managed и mobile: байткод, metadata, runtime и платформа . . . 64
Теоретическая глава 15. Kernel, drivers и UEFI: код ниже обычного процесса . . . 64
Теоретическая глава 16. Обфускация, упаковка и автоматизированный анализ как теория препятствий . . . 65
Теоретическая глава 17. Аппаратный слой: сигналы, шины, питание и физическая проверяемость . . . 65
Часть II. Живой фундамент реверса . . . 67
Глава 1. Машина не понимает программу как человек . . . 67
Глава 2. Байт как кирпич, но не как смысл . . . 67
Глава 3. Адреса: почему одно число может указывать на разные миры . . . 68
Глава 4. CPU как театр маленьких причин . . . 69
Глава 5. Функция, стек и ABI как социальный договор машинного кода . . . 69
Глава 6. Компилятор не переводит, а преобразует . . . 70
Глава 7. Loader и runtime: кто оживляет файл . . . 70
Глава 8. PE, ELF и Mach-O как разные грамматики исполняемости . . . 71
Глава 9. Decompiler, disassembler и опасность красивого псевдокода . . . 71
Глава 10. Embedded как мир, где адрес может включить железо . . . 72
Глава 11. Managed runtime и mobile: когда смысл живет выше CPU . . . 72
Глава 12. Kernel, drivers и UEFI: там, где нет обычного main . . . 73
Глава 13. Обфускация и упаковка как намеренное разрушение карты . . . 73
Глава 14. Hardware security: когда программа становится сигналом . . . 73
Глава 15. Доказательность как теоретический принцип, а не практика . . . 74
Часть III. Связная теория понятий . . . 75
Том 1. Том I. Грамотность байтов и форматов . . . 75
Понятие 1. байт . . . 76
Понятие 2. число . . . 78
Понятие 3. hex . . . 80
Понятие 4. endian . . . 82
Понятие 5. кодировка . . . 84
Понятие 6. выравнивание . . . 86
Понятие 7. checksum . . . 88
Понятие 8. entropy . . . 90
Понятие 9. container . . . 92
Понятие 10. signature . . . 94
Понятие 11. offset . . . 96
Понятие 12. evidence . . . 98
Системная связка тома 1 . . . 99
Том 2. Том II. C, память и данные как материал реверса . . . 100
Понятие 13. pointer . . . 101

Понятие 14. array . . . 103
Понятие 15. struct . . . 105
Понятие 16. union . . . 107
Понятие 17. padding . . . 109
Понятие 18. signedness . . . 111
Понятие 19. cast . . . 113
Понятие 20. heap . . . 115
Понятие 21. stack object . . . 117
Понятие 22. function pointer . . . 119
Понятие 23. undefined behavior . . . 121
Понятие 24. buffer . . . 123
Системная связка тома 2 . . . 124
Том 3. Том III. CPU, assembly и исполнение . . . 124
Понятие 25. fetch . . . 126
Понятие 26. decode . . . 128
Понятие 27. register . . . 130
Понятие 28. flags . . . 132
Понятие 29. addressing . . . 134
Понятие 30. branch . . . 136
Понятие 31. call . . . 138
Понятие 32. return . . . 140
Понятие 33. interrupt . . . 142
Понятие 34. fault . . . 144
Понятие 35. cache . . . 146
Понятие 36. atomic . . . 148
Системная связка тома 3 . . . 149
Том 4. Том IV. Компиляция, линковка и runtime . . . 149
Понятие 37. preprocessor . . . 151
Понятие 38. compiler IR . . . 153
Понятие 39. object file . . . 155
Понятие 40. symbol . . . 157
Понятие 41. relocation . . . 159
Понятие 42. linker script . . . 161
Понятие 43. static library . . . 163
Понятие 44. dynamic library . . . 165
Понятие 45. optimization . . . 167
Понятие 46. debug info . . . 169
Понятие 47. runtime . . . 171
Понятие 48. startup . . . 173
Системная связка тома 4 . . . 174
Том 5. Том V. Windows reverse engineering . . . 175
Понятие 49. DOS header . . . 176
Понятие 50. PE headers . . . 178
Понятие 51. section table . . . 180
Понятие 52. imports . . . 182

Понятие 53. exports . . . 184
Понятие 54. relocations . . . 186
Понятие 55. TLS . . . 188
Понятие 56. resources . . . 190
Понятие 57. SEH . . . 192
Понятие 58. x64 ABI . . . 194
Понятие 59. unwind . . . 196
Понятие 60. PDB . . . 198
Системная связка тома 5 . . . 199
Том 6. Том VI. Linux reverse engineering . . . 200
Понятие 61. ELF header . . . 201
Понятие 62. program header . . . 203
Понятие 63. section header . . . 205
Понятие 64. GOT . . . 207
Понятие 65. PLT . . . 209
Понятие 66. ld.so . . . 211
Понятие 67. syscall . . . 213
Понятие 68. signal . . . 215
Понятие 69. procfs . . . 217
Понятие 70. ptrace . . . 219
Понятие 71. DWARF . . . 221
Понятие 72. core dump . . . 223
Системная связка тома 6 . . . 224
Том 7. Том VII. Embedded bare metal . . . 225
Понятие 73. memory map . . . 226
Понятие 74. vector table . . . 228
Понятие 75. reset handler . . . 230
Понятие 76. MMIO . . . 232
Понятие 77. GPIO . . . 234
Понятие 78. UART . . . 236
Понятие 79. SPI . . . 238
Понятие 80. I2C . . . 240
Понятие 81. timer . . . 242
Понятие 82. DMA . . . 244
Понятие 83. Flash . . . 246
Понятие 84. clock tree . . . 248
Системная связка тома 7 . . . 249
Том 8. Том VIII. Архитектуры: x86-64, ARM, RISC-V, Xtensa . . . 250
Понятие 85. x86-64 . . . 251
Понятие 86. AArch64 . . . 253
Понятие 87. Cortex-M . . . 255
Понятие 88. Thumb . . . 257
Понятие 89. RISC-V . . . 259
Понятие 90. Xtensa . . . 261
Понятие 91. MIPS . . . 263

Понятие 92. calling convention . . . 265
Понятие 93. privileged mode . . . 267
Понятие 94. endianness . . . 269
Понятие 95. SIMD . . . 271
Понятие 96. DSP . . . 273
Системная связка тома 8 . . . 274
Том 9. Том IX. Firmware, hardware tools и лабораторная работа с устройствами . . . 275
Понятие 97. JTAG . . . 276
Понятие 98. SWD . . . 278
Понятие 99. OpenOCD . . . 280
Понятие 100. logic analyzer . . . 282
Понятие 101. UART console . . . 284
Понятие 102. flash dump . . . 286
Понятие 103. chip-off . . . 288
Понятие 104. binwalk . . . 290
Понятие 105. filesystem . . . 292
Понятие 106. QEMU . . . 294
Понятие 107. Unicorn . . . 296
Понятие 108. hardware safety . . . 298
Системная связка тома 9 . . . 300
Том 10. Том X. Инструменты и рабочий процесс . . . 300
Понятие 109. Ghidra . . . 301
Понятие 110. IDA . . . 303
Понятие 111. Rizin . . . 305
Понятие 112. Binary Ninja . . . 307
Понятие 113. x64dbg . . . 309
Понятие 114. WinDbg . . . 311
Понятие 115. GDB . . . 313
Понятие 116. Python scripting . . . 315
Понятие 117. types . . . 317
Понятие 118. signatures . . . 319
Понятие 119. notes . . . 321
Понятие 120. report . . . 323
Системная связка тома 10 . . . 324
Том 11. Том XI. Защитный анализ, malware и уязвимости . . . 325
Понятие 121. triage . . . 326
Понятие 122. sandbox . . . 328
Понятие 123. packer . . . 330
Понятие 124. anti-debug . . . 332
Понятие 125. crypto constants . . . 334
Понятие 126. protocol . . . 336
Понятие 127. persistence . . . 338
Понятие 128. network behavior . . . 340
Понятие 129. rootkit concept . . . 342
Понятие 130. memory forensics . . . 344

Понятие 131. vulnerability . . . 346
Понятие 132. disclosure . . . 348
Системная связка тома 11 . . . 349
Том 12. Том XII. Сквозные проекты и защита результатов . . . 350
Понятие 133. PE parser . . . 351
Понятие 134. unwind analyzer . . . 353
Понятие 135. ELF loader view . . . 355
Понятие 136. shell command table . . . 357
Понятие 137. vector recovery . . . 359
Понятие 138. STM32 GPIO . . . 361
Понятие 139. UART protocol . . . 363
Понятие 140. RTOS tasks . . . 365
Понятие 141. bootloader . . . 367
Понятие 142. embedded Linux rootfs . . . 369
Понятие 143. evidence table . . . 371
Понятие 144. study plan . . . 373
Системная связка тома 12 . . . 374
Том 13. Том XIII. Mach-O, Apple и мобильная среда . . . 375
Понятие 145. Mach-O header . . . 376
Понятие 146. load command . . . 378
Понятие 147. segment/section . . . 380
Понятие 148. dyld . . . 382
Понятие 149. codesign . . . 384
Понятие 150. entitlements . . . 386
Понятие 151. Objective-C runtime . . . 388
Понятие 152. Swift metadata . . . 390
Понятие 153. iOS sandbox . . . 392
Понятие 154. IPA . . . 394
Понятие 155. arm64e/PAC . . . 396
Понятие 156. crash log . . . 398
Системная связка тома 13 . . . 399
Том 14. Том XIV. Android, ART, JNI и мобильный Linux . . . 400
Понятие 157. APK . . . 401
Понятие 158. DEX . . . 403
Понятие 159. ART . . . 405
Понятие 160. OAT/VDEX . . . 407
Понятие 161. JNI . . . 409
Понятие 162. Binder IPC . . . 411
Понятие 163. SELinux . . . 413
Понятие 164. A/B update . . . 415
Понятие 165. boot image . . . 417
Понятие 166. native library . . . 419
Понятие 167. Frida on Android . . . 421
Понятие 168. logcat . . . 423
Системная связка тома 14 . . . 424

Том 15. Том XV. Managed runtimes: .NET, JVM, bytecode . . . 425
Понятие 169. managed code . . . 426
Понятие 170. IL . . . 428
Понятие 171. metadata token . . . 430
Понятие 172. assembly manifest . . . 432
Понятие 173. .NET JIT . . . 434
Понятие 174. GC . . . 436
Понятие 175. Java class file . . . 438
Понятие 176. JVM operand stack . . . 440
Понятие 177. constant pool . . . 442
Понятие 178. verification . . . 444
Понятие 179. reflection . . . 446
Понятие 180. obfuscator . . . 448
Системная связка тома 15 . . . 449
Том 16. Том XVI. Kernel, drivers, UEFI и boot chain . . . 450
Понятие 181. UEFI . . . 451
Понятие 182. DXE driver . . . 453
Понятие 183. PEI . . . 455
Понятие 184. ACPI table . . . 457
Понятие 185. kernel mode . . . 459
Понятие 186. driver object . . . 461
Понятие 187. IRP . . . 463
Понятие 188. IOCTL . . . 465
Понятие 189. Linux kernel module . . . 467
Понятие 190. sysfs/debugfs . . . 469
Понятие 191. eBPF . . . 471
Понятие 192. Secure Boot . . . 473
Системная связка тома 16 . . . 474
Том 17. Том XVII. Защита, obfuscation и автоматизированный анализ . . . 475
Повторная связка 193. packer . . . 475
Повторная связка 194. anti-debug . . . 475
Понятие 195. control-flow flattening . . . 476
Понятие 196. opaque predicate . . . 478
Понятие 197. string encryption . . . 480
Понятие 198. VM-based obfuscation . . . 482
Понятие 199. YARA . . . 484
Понятие 200. capa . . . 486
Понятие 201. Frida . . . 488
Понятие 202. fuzzing . . . 490
Понятие 203. symbolic execution . . . 492
Понятие 204. taint analysis . . . 494
Системная связка тома 17 . . . 496
Том 18. Том XVIII. Hardware security и полевые системы . . . 496
Понятие 205. side channel . . . 497
Понятие 206. fault injection . . . 499

Понятие 207. glitching . . . 501
Повторная связка 208. logic analyzer . . . 502
Понятие 209. oscilloscope . . . 503
Понятие 210. SPI Flash . . . 505
Понятие 211. JTAG boundary scan . . . 507
Понятие 212. CAN bus . . . 509
Понятие 213. radio firmware . . . 511
Понятие 214. boot ROM . . . 513
Понятие 215. TrustZone . . . 515
Понятие 216. secure element . . . 517
Системная связка тома 18 . . . 519
Карта терминов: как английские слова связываются в одну систему . . . 520
Узел 1. offset . . . 520
Узел 2. address . . . 520
Узел 3. register . . . 520
Узел 4. stack . . . 521
Узел 5. function . . . 521
Узел 6. ABI . . . 521
Узел 7. loader . . . 521
Узел 8. runtime . . . 522
Узел 9. MMIO . . . 522
Узел 10. unwind . . . 522
Узел 11. bytecode . . . 523
Узел 12. hook . . . 523
Как пользоваться картой . . . 523
Как проверять теорию по документации . . . 524
Приложение A. Большой словарь терминов . . . 525
Карточки ключевых понятий . . . 527
ABI . . . 527
offset . . . 527
stack . . . 527
register . . . 528
RVA . . . 528
VA . . . 528
MMIO . . . 528
loader . . . 528
linker . . . 529
relocation . . . 529
unwind . . . 529
prologue . . . 529
epilogue . . . 529
interrupt . . . 530
handler . . . 530
firmware . . . 530
vector table . . . 530

volatile . . . 530
DMA . . . 531
GOT . . . 531
PLT . . . 531
symbol . . . 531
xref . . . 531
entry point . . . 532
TLS . . . 532
heap . . . 532
syscall . . . 532
watchpoint . . . 532
raw firmware . . . 532
RTOS . . . 533
Матрица проверки фактов и источников . . . 534
Как читать матрицу . . . 535
Приложение B. Официальные источники и книги для углубления . . . 536
Официальная документация . . . 536
Книги и большие курсы, на которые стоит опираться . . . 537

База для реверс-инженера с нуля: Windows, Linux и embedded

Предисловие к исправленному изданию

Эта книга написана для человека, который может быть умным, внимательным и технически любопытным, но при этом никогда не жил внутри IT. Поэтому мы не начинаем с дизассемблера, не пугаем таблицами заголовков и не бросаем слова `stack`, `ABI`, `offset`, `relocation`, `MMIO` так, будто они обязаны быть знакомыми. Мы начинаем ниже: с вопроса, что такое байт, почему память похожа на карту, как процессор вообще понимает команду, почему функция в исходном коде превращается в странную последовательность инструкций и почему один и тот же адрес может означать разные вещи в файле, в процессе Windows, в процессе Linux и в прошивке микроконтроллера.

Reverse engineering - это не магия и не угадайка. По-русски это обычно называют обратной разработкой или реверс-инжинирингом: мы берем уже готовый артефакт - программу, библиотеку, драйвер, прошивку, дамп памяти - и восстанавливаем, как он устроен. Слово reverse здесь означает «в обратную сторону»: обычный инженер пишет исходный код и получает бинарный файл, а реверсер берет бинарный файл и пытается восстановить структуру, намерения, данные, протоколы и поведение.

Главная привычка реверсера проста: не верить первому впечатлению. Байты не становятся кодом только потому, что дизассемблер так решил. Адрес не становится указателем только потому, что похож на адрес. Имя функции не становится истиной только потому, что декомпилятор красиво его нарисовал. Сильный анализ строится слоями: сначала наблюдение, потом правило чтения, потом гипотеза, потом проверка, потом аккуратное объяснение.

Книга универсальная. Твоя ВКР сейчас связана с Windows, PE32+, x64 и unwind metadata, но фундамент нужен шире: Linux, ELF, embedded, Cortex-M, ARM, RISC-V, ESP32, прошивки, загрузчики, JTAG/SWD, UART, RTOS. Поэтому мы будем постоянно сравнивать три мира.

- Windows: есть процесс, PE-файл, загрузчик ОС, DLL, виртуальная память,

исключения, WinDbg, x64 calling convention.

- Linux: есть процесс, ELF-файл, системные вызовы, динамический загрузчик,

`/proc`, GDB, System V ABI.

- Embedded: часто нет привычного процесса и ОС; есть микроконтроллер,

Flash, RAM, MMIO-регистры, vector table, interrupts, linker script, bootloader, отладочный интерфейс и документация на железо.

В тексте много английских терминов, потому что документация, инструменты и книги по теме в основном англоязычные. Но каждый важный термин здесь разбирается: буквальный смысл, русское объяснение, зачем он нужен, где его увидеть и с какими соседними понятиями он связан. Английское слово не должно быть стеной. Оно должно стать ярлыком на понятную идею.

Как читать эту книгу

Не пытайся читать как словарь. Реверс лучше учится как карта местности. Сначала нужно увидеть крупные районы: байты, память, процессор, функции, форматы файлов, загрузка, отладка, embedded. Потом возвращаться к деталям. Если глава кажется тяжелой, не значит, что ты не понимаешь тему. Значит, мозг встретил новый слой абстракции. Это нормально.

У каждой большой темы есть три уровня:

1. Человеческий смысл. Например: стек - это место, куда программа временно

складывает то, что понадобится при возврате из функции.

2. Машинная механика. Например: на x86-64 регистр `rsp` хранит адрес вершины

стека, а команда `call` кладет адрес возврата на стек.

3. Инженерное применение. Например: если стек испорчен, unwind не сможет

восстановить цепочку вызовов, а отладчик покажет обрыв или ложный stack trace.

Когда видишь английский термин, задавай четыре вопроса:

- Что это буквально означает?
- Какую проблему это решает?
- Где это находится: в файле, в памяти, в процессоре, в ОС, в железе?
- Как я могу проверить это инструментом?

Например, `offset` буквально означает «смещение». Это не обязательно адрес. Это расстояние от начала чего-то: от начала файла, от начала структуры, от начала секции, от начала стека. Если не уточнить, от какого начала считается offset, анализ развалится. Поэтому хороший реверсер всегда добавляет контекст: file offset, section offset, stack offset, structure offset.

Этическая и практическая рамка

Реверс-инжиниринг может быть полезен: исследование собственного проекта, анализ совместимости, восстановление форматов, поиск уязвимостей, аудит безопасности, анализ вредоносного ПО в изолированной лаборатории, изучение прошивок устройств, которые принадлежат тебе или исследуются с разрешения. Но он может быть и опасен юридически и этически. Не запускай подозрительные файлы на основной машине. Не обходи лицензии и защиту ради чужого доступа. Не публикуй рабочие инструкции для злоупотреблений. В этой книге мы изучаем механику и методику анализа, а не эксплуатацию людей и систем.

Практическое правило: все, что можно запускать, лучше запускать в отдельной виртуальной машине или на отдельном лабораторном устройстве. Все, что можно документировать, нужно документировать: хэш файла, источник, дата, версия инструмента, наблюдения, гипотезы, проверки, ограничения вывода.

Часть I. Нулевая база: из чего состоит машинный мир

Глава 1. Байт: маленькая клетка, из которой строится все

Если открыть любой исполняемый файл, библиотеку, дамп памяти или прошивку как текст, получится каша. Это не потому, что файл «сломанный». Просто человек пытается читать машинный материал человеческим способом. Для машины файл - это последовательность байтов.

Байт - это обычно 8 бит. Бит - самый маленький двоичный выбор: 0 или 1. Восемь таких выборов дают 256 вариантов, от 0 до 255. Поэтому байт можно записать десятичным числом, например 77, но в реверсе почти всегда используют hex - шестнадцатеричную запись. Hex удобен не для красоты, а потому что один байт записывается ровно двумя hex-цифрами:

```
00  - нулевой байт
01  - единица
7F  - 127
80  - 128
FF  - 255
```

Две hex-цифры - один байт. Это первое спокойное правило. Когда ты видишь дамп:

```
4D 5A 90 00 03 00 00 00
```

ты смотришь не на заклинание, а на восемь маленьких ячеек. Каждая ячейка может быть частью числа, текста, машинной инструкции, таблицы, адреса или просто заполнителем. Смысл не лежит в байте сам по себе. Смысл появляется, когда мы знаем правило чтения.

Пример: байты `4D 5A` в начале Windows PE-файла читаются как ASCII-символы `MZ`. ASCII - старый, но очень важный способ сопоставить числам символы. Число `0x4D` означает букву `M`, а `0x5A` означает `Z`. Но те же байты можно прочитать как 16-битное число. Тогда вопрос будет уже не «какие это буквы?», а «в каком порядке складывать байты?».

Так мы приходим к endian. Endianness - порядок байтов внутри многобайтового числа. Термин странный, исторический, но идея простая. Число `0x12345678` занимает четыре байта. В big-endian старший байт лежит первым:

```
12 34 56 78
```

В little-endian младший байт лежит первым:

```
78 56 34 12
```

Большинство привычных x86/x64 машин и многие микроконтроллеры в типичных режимах используют little-endian. Поэтому если в прошивке Cortex-M по адресу 0 лежат байты:

```
00 40 00 20
```

это часто читается как число `0x20004000`, а не `0x00400020`. Для новичка это выглядит как перевернутый мир, но через неделю практики становится автоматическим движением: «вижу четыре байта - уточняю endian - читаю число».

Еще один базовый термин - mask, по-русски битовая маска. Маска - это число, которое используют, чтобы проверить или выделить отдельные биты. Представь регистр состояния, где каждый бит означает флажок:

```
bit 0 - включено питание
bit 1 - готовность
bit 2 - ошибка
bit 3 - передача идет
```

Если значение регистра равно `0b00000110`, значит выставлены биты 1 и 2: готовность есть, но есть и ошибка. В hex это будет `0x06`. В embedded такая логика встречается постоянно: периферия управляется не красивыми объектами, а числами, где каждый бит имеет смысл из reference manual.

Низкоуровневое мышление начинается с честности: перед нами не «программа», а байты. Мы не имеем права сразу сказать, что байты делают. Мы сначала должны понять, как они читаются: как строка, как число, как адрес, как инструкция, как заголовок формата, как таблица указателей или как регистр железа.

Глава 2. Память: почему адрес не равен месту в файле

Память удобно представлять как длинную улицу с домами. У каждого дома есть номер. Этот номер называется address - адрес. Но здесь сразу появляется ловушка: адрес может быть адресом в памяти, а может быть смещением в файле. Новичок часто смешивает эти вещи и потом не понимает, почему дизассемблер, hex editor и отладчик показывают разные числа.

File offset - смещение в файле. Это расстояние от начала файла на диске. Если hex editor показывает, что байт находится на offset `0x200`, это значит: от начала файла нужно пройти 512 байтов.

Virtual address, или VA, - виртуальный адрес в адресном пространстве процесса. Виртуальный адрес видит программа во время выполнения. Windows и Linux не отдают процессу всю физическую память напрямую. Они создают для каждого процесса свой «воображаемый город» адресов. Процесс думает, что у него есть непрерывная карта, а операционная система и процессор через MMU переводят виртуальные адреса в физические страницы.

RVA - relative virtual address, относительный виртуальный адрес. В PE-файлах это адрес относительно image base. Image base - базовый адрес, куда модуль предпочтительно загружается. Если image base равен `0x140000000`, а RVA равен `0x1234`, то VA будет:

```
0x140000000 + 0x1234 = 0x140001234
```

Почему это важно? Потому что PE-файл на диске и PE-образ в памяти не обязаны быть одинаково разложены. Секции в файле выровнены по FileAlignment, а в памяти - по SectionAlignment. Поэтому «адрес в памяти» нельзя бездумно подставлять как «смещение в файле».

В ELF-мире похожая идея выражается через file offset, virtual address и program headers. Program header говорит загрузчику Linux: такой-то кусок файла нужно отобразить в память по такому-то виртуальному адресу с такими-то правами. Section headers больше нужны линкеру, отладчику и инструментам анализа, а загрузчик в первую очередь смотрит на сегменты.

В embedded все снова меняется. У микроконтроллера может не быть процесса и виртуальной памяти. Адрес `0x08000000` может прямо означать Flash-память, а `0x20000000` - SRAM. Адрес `0x40021000` может быть вообще не RAM, а MMIO-регистр. MMIO расшифровывается как memory-mapped I/O - ввод-вывод, отображенный в память. То есть процессор использует обычные команды чтения и записи по адресу, но на другом конце находится не ячейка памяти, а аппаратный блок: GPIO, UART, таймер, контроллер тактирования.

Сравним три ситуации:

Мир	Что означает адрес	Где искать правило
Windows process	Виртуальное место внутри процесса	PE headers, loader, Virtual Memory API
Linux process	Виртуальное место внутри процесса	ELF program headers, `/proc/<pid>/maps`

Мир	Что означает адрес	Где искать правило
Cortex-M firmware	Часто физический адрес Flash/RAM/MMIO	Reference manual и linker script

Linker script - сценарий линковщика. Это файл, который говорит линкеру: какие секции куда положить. В embedded он особенно важен. Там нельзя просто сказать «положи как получится». Нужно знать, что код идет во Flash, данные после старта копируются в RAM, `.bss` очищается нулями, стек кладется в конец SRAM, а таблица векторов должна оказаться там, где ее ожидает CPU.

Чтобы не путаться, всегда записывай координатную систему. Не «адрес 0x1234», а «file offset 0x1234», «RVA 0x1234», «VA 0x140001234», «MMIO address 0x40021000», «stack offset -0x20 from rbp». Это выглядит занудно, но именно такая занудность спасает от ложных выводов.

Глава 3. Процессор: кто исполняет байты

Процессор, или CPU, не читает C-код, Python-код и красивые имена функций. Он исполняет машинные инструкции. Инструкция - это маленькая команда: загрузить значение, сложить, сравнить, перейти, вызвать функцию, вернуть управление, записать в память. Набор доступных инструкций называется ISA - Instruction Set Architecture. По-русски: архитектура набора инструкций.

ISA отвечает на вопросы:

- какие есть регистры;
- как кодируются инструкции в байтах;
- как инструкция обращается к памяти;
- где хранится адрес следующей инструкции;
- как вызываются переходы, исключения и прерывания;
- какие режимы работы есть у процессора.

Register - регистр - это маленькое, очень быстрое хранилище внутри CPU. Представь рабочий стол, на который процессор кладет самые нужные листки. Память большая, но обращение к ней дороже. Регистры маленькие, зато очень быстрые. На x86-64 ты встретишь `rax`, `rbx`, `rcx`, `rdx`, `rsp`, `rbp`, `rip`, `r8` и другие. На ARM - `r0`, `r1`, `sp`, `lr`, `pc`. На RISC-V - `a0`, `a1`, `sp`, `ra`, `t0`, `s0`.

Instruction pointer - указатель инструкции. На x86-64 он называется `rip`. Он хранит адрес текущей или следующей инструкции, в зависимости от контекста описания. На ARM похожую роль играет `pc`, program counter. Если обычные регистры похожи на рабочие переменные процессора, то instruction pointer - это палец на строке книги: «сейчас выполняем здесь».

Stack pointer - указатель стека. На x86-64 это `rsp`, на ARM и RISC-V часто `sp`. Он показывает верхушку стека. Стек нужен для временного хранения: адреса возврата, сохраненных регистров, локальных переменных, иногда аргументов. Мы разберем стек подробно позже, но уже сейчас важно: если `sp/rsp` сломан, программа часто теряет способность нормально возвращаться из функций.

Flags - флаги состояния. После сравнения или арифметики CPU выставляет маленькие признаки: результат ноль или нет, был перенос или нет, получилось отрицательное значение или нет. Условные переходы используют эти флаги. Например:

```
cmp eax, 5
je  equal_case
```

`cmp` сравнивает `eax` с 5 и выставляет флаги. `je` означает jump if equal - перейти, если равно. Сам `je` не сравнивает числа заново. Он смотрит на флаг, оставленный предыдущей инструкцией. Поэтому при чтении assembly нельзя разрывать сравнение и переход.

На низком уровне выполнение программы выглядит как бесконечный цикл:

1. Взять байты по адресу instruction pointer.
2. Декодировать их как инструкцию данной ISA.
3. Выполнить действие: поменять регистры, память или поток управления.
4. Обновить instruction pointer.
5. Повторить.

Это упрощение, потому что современные CPU имеют кэши, конвейеры, предсказание ветвлений, out-of-order execution и микрооперации. Но для первого уровня реверса полезно держать именно эту модель: адрес инструкции, состояние регистров, память, следующий шаг.

Разница архитектур хорошо видна на вызове функции:

```
; x86-64
call target       ; адрес возврата кладется на стек

; ARM
bl target         ; адрес возврата кладется в lr

; RISC-V
jal ra, target    ; адрес возврата кладется в ra
```

Идея одна: перейти в функцию и запомнить, куда вернуться. Механика разная. Вот почему реверсер не зубрит одну «правильную» форму, а учится видеть роль: где адрес возврата, где аргументы, где результат, кто обязан сохранить какие регистры.

Глава 4. Assembly: не язык демонов, а журнал изменений состояния

Assembly - ассемблер - это текстовое представление машинных инструкций. Машина исполняет байты, но человеку удобнее читать мнемоники: `mov`, `add`, `cmp`, `call`, `ret`, `ldr`, `str`, `b`, `jal`. Mnemonic - мнемоника, то есть короткое запоминаемое имя инструкции.

Новичок часто пытается переводить каждую строку assembly как отдельное предложение. Это быстро утомляет. Лучше читать иначе: каждая инструкция меняет состояние машины. Состояние машины - это регистры, память, флаги, instruction pointer и иногда состояние сопроцессоров или периферии.

Возьмем маленький фрагмент:

```
mov ecx, 10
call do_work
test eax, eax
jne  failed
```

Если знать Windows x64 calling convention, можно прочитать так:

`mov ecx, 10` кладет число 10 в `ecx`. В Windows x64 первый целочисленный аргумент функции передается через `rcx/ecx`. Значит, перед вызовом готовится первый аргумент.

`call do_work` вызывает функцию. После возврата результат часто лежит в `eax/rax`. Это не закон природы, а часть ABI - соглашения о бинарном интерфейсе.

`test eax, eax` проверяет, равен ли результат нулю. Команда делает побитовое AND значения с самим собой и выставляет флаги. Значение при этом не меняется.

`jne failed` означает jump if ne equal - перейти, если не равно. После `test eax,eax` это обычно читается как «если результат не ноль, перейти в ветку ошибки» или наоборот, в зависимости от соглашения функции.

Теперь важный момент: без знания calling convention этот фрагмент хуже читается. Ты увидишь `ecx`, но не поймешь, что это аргумент. Поэтому assembly никогда не читается в вакууме. Всегда нужен контекст: архитектура, ABI, формат файла, ОС, компилятор, оптимизации.

Addressing mode - режим адресации - это способ, которым инструкция находит операнд. Операнд - то, над чем инструкция работает. В `mov eax, 5` операнд 5 вписан прямо в инструкцию. Такое значение называется immediate - непосредственное значение. В `mov eax, [rbx]` значение берется из памяти по адресу, лежащему в `rbx`. Квадратные скобки в Intel-синтаксисе часто означают «прочитать память по адресу».

Сравни:

```
mov eax, 1234      ; положить число 1234
mov eax, [1234]    ; прочитать память по адресу 1234
lea rax, [rbx+8]   ; вычислить адрес rbx+8, но память не читать
```

`lea` часто путает новичков. Название означает load effective address - загрузить эффективный адрес. Но команда не читает память. Она вычисляет адресное выражение. Компиляторы любят использовать `lea` и для обычной арифметики, потому что она умеет быстро складывать регистр, индекс и смещение.

Branch - ветвление или переход. Программа редко идет ровной линией. У нее есть `if`, циклы, `switch`, обработка ошибок. В assembly это превращается в условные и безусловные переходы. Control flow graph, или CFG, - граф потока управления. Это схема, где вершины - базовые блоки инструкций, а стрелки - возможные переходы. Когда Ghidra или IDA рисует граф функции, она помогает увидеть структуру условий и циклов.

Читать assembly нужно с карандашом в голове:

- какие регистры входят в блок живыми;
- какие меняются;
- какие значения читаются из памяти;
- куда может пойти управление;
- что функция обещает вернуть;
- какие предположения декомпилятор сделал сам.

Декомпилятор полезен, но он не свидетель. Он переводчик. Иногда хороший, иногда слишком уверенный. Если псевдокод выглядит странно, возвращайся к assembly и проверяй факты.

Часть II. Функции, стек и ABI

Глава 5. Что такое функция, когда исходного кода уже нет

В исходном коде функция выглядит дружелюбно:

```
int add(int a, int b) {
    return a + b;
}
```

Есть имя, тип результата, параметры, фигурные скобки. В бинарнике почти все это может исчезнуть. Имя может пропасть. Типы могут пропасть. Граница функции может быть восстановлена инструментом приблизительно. Но сама идея функции остается: есть точка входа, есть способ получить аргументы, есть тело работы, есть способ вернуть результат и продолжить выполнение у вызывающего кода.

Caller - вызывающий код. Callee - вызываемая функция. Эти слова важны, потому что ABI распределяет обязанности между ними. ABI расшифровывается как Application Binary Interface - прикладной бинарный интерфейс. Если API - это соглашение на уровне исходного кода и функций, то ABI - соглашение на уровне уже скомпилированного кода: в каких регистрах идут аргументы, кто чистит стек, где лежит результат, какие регистры нужно сохранить, как выглядит стек при вызове.

Function prologue - пролог функции. Это первые инструкции, которые готовят функцию к работе: сохраняют нужные регистры, выделяют место на стеке, настраивают frame pointer, если он используется. Function epilogue - эпилог, конец функции: восстановить регистры, освободить стек, вернуться.

Но оптимизатор может убрать красивый пролог. Маленькая функция может быть inlined - встроена прямо в вызывающий код. Tail call - хвостовой вызов - может превратить конец функции в переход к другой функции без обычного возврата. Поэтому реверсер не должен ждать одного шаблона. Он ищет смысл.

Call graph - граф вызовов. Если функция A вызывает B и C, а B вызывает D, это можно нарисовать как связи. Call graph помогает понять крупную архитектуру программы. Но в реальном реверсе граф не всегда точный: виртуальные вызовы, function pointers, callbacks, jump tables и обфускация делают связи неочевидными.

Xref - cross-reference, перекрестная ссылка. В Ghidra, IDA и других инструментах xrefs показывают, кто ссылается на адрес, строку, функцию, таблицу. Если ты нашел строку `"license failed"`, xrefs к ней могут привести к функции проверки. Если нашел MMIO-адрес UART, xrefs могут привести к драйверу UART. Это один из главных приемов: идти от понятного островка к непонятному морю.

Глава 6. Стек: временная память, где программа помнит дорогу назад

Stack - стек - структура «последним пришел, первым ушел». Представь стопку листов: положил лист сверху, потом снял его первым. В программах стек используется для временных данных, связанных с вызовами функций.

Stack frame - кадр стека. Это участок стека, принадлежащий одному вызову функции. В нем могут лежать адрес возврата, сохраненные регистры, локальные переменные, временные значения, иногда аргументы. Когда функция вызывает другую функцию, появляется новый frame. Когда функция возвращается, ее frame исчезает.

На x86-64 стек обычно растет вниз: к меньшим адресам. Это звучит странно, но означает простую вещь: если `rsp` уменьшился, на стеке выделили место. Если `rsp` увеличился, место освободили.

Упрощенный вызов на x86-64:

```
caller готовит аргументы
call target
    CPU кладет return address на стек
    CPU переходит к target
target выделяет место под frame
target работает
target восстанавливает стек
ret
    CPU берет return address со стека
    CPU переходит обратно
```

Return address - адрес возврата. Это адрес инструкции, которая должна выполниться после завершения функции. Если адрес возврата поврежден, функция вернется не туда. Поэтому переполнение буфера на стеке исторически было опасным классом уязвимостей: данные могли затереть return address. Современные системы используют защиты: stack cookies, ASLR, DEP/NX, Control-flow Enforcement Technology и другие механизмы. Но фундаментальная механика остается важной.

Frame pointer - указатель кадра. На x86-64 часто это `rbp`, если компилятор его использует. Старый классический пролог:

```
push rbp
mov  rbp, rsp
sub  rsp, 0x40
```

`push rbp` сохраняет старое значение `rbp`. `mov rbp,rsp` делает новый ориентир для текущего frame. `sub rsp,0x40` выделяет 64 байта под локальные данные. Тогда локальная переменная может лежать по адресу `[rbp-0x10]`, а аргумент или сохраненное значение - по `[rbp+...]`.

Но в оптимизированном x64-коде `rbp` может быть обычным регистром, а frame pointer omitted - опущен. Тогда unwind и отладчик опираются не на цепочку `rbp`, а на metadata: специальные данные о том, как функция меняет стек. В Windows x64 это особенно важно для твоей темы ВКР: `.pdata` и `.xdata` хранят информацию, позволяющую восстановить состояние при исключениях и stack walking.

Stack trace - трасса стека, цепочка вызовов. Когда отладчик показывает:

```
main
parse_file
read_header
fail
```

он пытается восстановить, какие функции привели к текущему месту. Это не магический дневник. Это вывод из стековых данных, unwind metadata, символов и соглашений ABI. Если metadata неверна или стек поврежден, stack trace может лгать.

Глава 7. Windows x64 calling convention: договор Windows о вызове функций

Calling convention - соглашение о вызове. Это правила, по которым функции передают аргументы, возвращают значения и сохраняют регистры. В Windows x64 первые четыре целочисленных или pointer-аргумента обычно идут в регистрах:

```
1-й аргумент: RCX
2-й аргумент: RDX
3-й аргумент: R8
4-й аргумент: R9
результат:    RAX
```

Для floating-point значений используются XMM-регистры. Если аргументов больше четырех, остальные попадают на стек. Но даже для первых четырех аргументов caller обязан зарезервировать shadow space - 32 байта на стеке. Shadow space, или home space, - это место, куда callee при необходимости может сохранить регистровые аргументы. Слово shadow здесь не про тень в мистическом смысле, а про «резервное место рядом с вызовом».

Упрощенная картина перед вызовом:

```
RCX = первый аргумент
RDX = второй аргумент
R8  = третий аргумент
R9  = четвертый аргумент
на стеке есть 32 байта shadow space
stack alignment соблюден
```

Stack alignment - выравнивание стека. Для Windows x64 важно, чтобы стек был выровнен на 16 байт в нужных точках. Выравнивание нужно не для красоты, а для быстрой и корректной работы инструкций, особенно SIMD и ABI-зависимых операций.

Volatile registers - регистры, которые вызываемая функция может испортить. Nonvolatile registers - регистры, которые callee обязана сохранить, если использует. В Windows x64 nonvolatile включают, например, `rbx`, `rbp`, `rdi`, `rsi`, `r12-r15`, `xmm6-xmm15`. Для реверса это важно: если функция сохраняет `rbx` в прологе и восстанавливает в эпилоге, она соблюдает ABI.

Пример:

```
sub rsp, 28h
mov ecx, 10
call some_function
add rsp, 28h
ret
```

Почему `28h`, то есть 40 байт, а не ровно 32? Потому что кроме shadow space нужно учесть выравнивание после `call`, который положил return address на стек. Такие детали кажутся мелкими, но именно они отличают поверхностное чтение от настоящего понимания.

Windows x64 тесно связывает prologue/epilogue с unwind. Unwind - буквально «разматывание». В контексте исключений это восстановление предыдущего состояния стека и регистров, чтобы понять, куда передать управление или как построить stack trace. Если функция меняет `rsp`, сохраняет регистры и выделяет стек, Windows должна иметь описание этих действий. Поэтому у функций есть ограничения на форму пролога и эпилога, а данные в `.pdata` и `.xdata` помогают ОС пройти назад по стеку.

Для твоей ВКР это центральная связь: PE-файл хранит не только код, но и структуры, которые описывают, как этот код ведет себя при исключениях и размотке стека. Реверс здесь не заканчивается на инструкциях. Нужно читать формат файла, ABI и runtime-механику Windows вместе.

Глава 8. System V AMD64, ARM AAPCS и RISC-V ABI: тот же смысл, другие правила

Linux x86-64 обычно использует System V AMD64 ABI. Первые целочисленные аргументы идут в другом порядке регистров:

```
1-й: RDI
2-й: RSI
3-й: RDX
4-й: RCX
5-й: R8
6-й: R9
```

```
результат: RAX
```

Если ты читаешь Linux-бинарник как Windows x64, аргументы перепутаются. Поэтому первый вопрос реверсера: «какой ABI передо мной?». Не «какая команда мне нравится», а именно ABI. Он задает грамматику вызовов.

У System V есть red zone - область 128 байт ниже `rsp`, которую leaf-функция может использовать без изменения `rsp`. Leaf function - функция-лист: она не вызывает другие функции. В Windows x64 red zone нет. Поэтому один и тот же на вид код вокруг стека может иметь разные допустимые смыслы.

ARM AAPCS - Arm Architecture Procedure Call Standard. На 32-битном ARM первые аргументы обычно идут в `r0-r3`, результат - в `r0`, адрес возврата - в `lr` link register. Link register - регистр связи, место, куда команда вызова кладет адрес возврата. Если функция вызывает другую функцию, ей часто нужно сохранить `lr`, иначе адрес возврата затрется.

На Cortex-M исключения и прерывания автоматически кладут часть регистров на стек. Это называется exception entry. CPU сам сохраняет `r0-r3`, `r12`, `lr`, `pc`, `xPSR`, а потом переходит к handler. Handler - обработчик. Когда обработчик заканчивается, специальный exception return восстанавливает контекст. Поэтому embedded-реверс часто требует понимать не только C-функции, но и аппаратную модель исключений.

RISC-V ABI использует понятные имена ролей:

```
a0-a7  - argument registers, аргументы и результат
ra     - return address
sp     - stack pointer
t0-t6  - temporaries, временные регистры
s0-s11 - saved registers, сохраняемые регистры
```

Эта архитектура хороша для обучения, потому что имена регистров прямо подсказывают роль. Но роль все равно нужно подтверждать контекстом. Регистр `a0` может быть аргументом перед вызовом и результатом после вызова.

Общее правило для всех ABI:

- найди, где перед вызовом готовятся аргументы;
- найди инструкцию вызова;
- посмотри, где после возврата используется результат;
- проверь, какие регистры сохраняются и восстанавливаются;
- не переноси правила одной платформы на другую без проверки.

Часть III. Как исходный код превращается в бинарник

Глава 9. Компилятор, assembler, linker, loader: четыре разных существа

Когда программист нажимает build, кажется, что «компилятор сделал программу». На самом деле путь длиннее. Для C/C++ типичная цепочка такая:

```
source code -> preprocessor -> compiler -> assembler -> object files
object files + libraries -> linker -> executable/shared library
executable -> loader -> процесс в памяти
```

Preprocessor - препроцессор. Он обрабатывает `#include`, `#define`, условную компиляцию. Он еще не понимает программу глубоко. Он подставляет текст и готовит исходник для компилятора.

Compiler - компилятор. Он переводит высокоуровневый код в более низкое представление: анализирует синтаксис, типы, строит intermediate representation, оптимизирует и генерирует assembly или машинный код. Intermediate representation, IR, - промежуточное представление. Это удобная для компилятора форма программы, где можно делать оптимизации независимо от конкретной архитектуры.

Assembler - ассемблер. Он превращает текст assembly в object file. Object file - объектный файл: кусок будущей программы, где уже есть машинный код, но еще не все адреса известны. В нем могут быть relocations - записи о том, что некоторые места нужно поправить позже, когда будет известен окончательный адрес символа или секции.

Linker - линковщик. Он соединяет object files и libraries. Он решает, где будут лежать секции, какие символы на что ссылаются, какие relocations нужно применить, какие внешние функции останутся импортами. В embedded linker еще и раскладывает код по конкретной карте памяти микроконтроллера.

Loader - загрузчик. Это уже часть ОС или runtime-среды. Он берет готовый исполняемый файл, отображает его в память, назначает права страниц, подключает библиотеки, применяет relocations, разрешает imports, вызывает инициализацию и передает управление entry point.

Entry point - точка входа. Это адрес, с которого начинается выполнение модуля после загрузки. Важно: entry point не всегда равен `main`. В C/C++ перед `main` обычно работает runtime: инициализация глобальных объектов, настройка аргументов, подготовка окружения, иногда TLS callbacks. В embedded после reset сначала работает startup code: установка стека, копирование `.data` из Flash в RAM, очистка `.bss`, настройка clock, вызов `main`.

Почему это важно для реверса? Потому что бинарник хранит следы всей цепочки. Оптимизации компилятора меняют форму функций. Линкер создает таблицы. Загрузчик меняет адреса. Runtime добавляет код, который не писал автор приложения. Если принять весь код за «ручную бизнес-логику», можно утонуть. Нужно отличать пользовательскую логику от стандартной обвязки.

Глава 10. Оптимизации: почему бинарник не похож на исходник

Новичок часто ожидает: одна строка C - несколько очевидных инструкций. Иногда это верно в debug-сборке без оптимизаций. В release-сборке компилятор стремится сделать программу быстрее или меньше, а не удобнее для реверсера.

Inlining - встраивание функции. Если функция маленькая, компилятор может не делать `call`, а вставить ее тело прямо в место вызова. Тогда в бинарнике не будет отдельного вызова, хотя в исходнике он был. Для реверсера это значит: отсутствие функции не доказывает отсутствие

логики.

Constant folding - сворачивание констант. Выражение `2 + 3 * 4` может стать сразу `14`. Dead code elimination - удаление мертвого кода. Если результат вычисления нигде не используется, компилятор может убрать вычисление.

Register allocation - распределение регистров. Компилятор решает, какие переменные держать в регистрах, какие выгрузить в стек. Поэтому локальная переменная из исходника не обязана иметь постоянный адрес. Она может жить сначала в `eax`, потом в `ecx`, потом вообще исчезнуть.

Loop unrolling - разворачивание цикла. Вместо компактного цикла компилятор может повторить тело несколько раз, чтобы уменьшить количество переходов. Vectorization - векторизация - использование SIMD-инструкций, которые обрабатывают сразу несколько значений. Тогда простой цикл по массиву может стать набором операций над XMM/YMM-регистрами.

Tail call optimization - оптимизация хвостового вызова. Если функция в конце только возвращает результат другой функции, компилятор может заменить `call` и `ret` на `jmp`. В call graph это выглядит как переход, а не обычный вызов.

Пример:

```
int wrapper(int x) {
    return real_work(x);
}
```

В бинарнике это может стать:

```
jmp real_work
```

И это не ошибка. Это нормальная оптимизация.

Оптимизации не нужно бояться. Нужно перестать ждать буквального совпадения с исходником. Хороший вопрос не «где моя строка C?», а «какое наблюдаемое поведение сохранилось после оптимизации?». Аргументы все равно где-то попадают в вычисление. Проверка все равно влияет на переход. Доступ к файлу все равно приведет к syscall или API. Работа с MMIO все равно оставит запись по адресу регистра периферии.

Глава 11. Символы, debug information и почему им нельзя слепо верить

Symbol - символ - имя, связанное с адресом или объектом: функция, переменная, секция. В debug-сборке символов много. В release-сборке их могут удалить. Strip - удалить символы. Stripped binary - бинарник без символов.

Debug information - отладочная информация. В Linux и многих embedded-сборках часто используется DWARF. DWARF описывает типы, строки исходника, переменные, адресные диапазоны, правила восстановления стека. В Windows распространены PDB-файлы. PDB - Program Database, база отладочной информации Microsoft.

Если символы есть, жизнь проще. Но реверсер должен помнить: символы могут быть неполными, устаревшими, намеренно ложными или относиться к библиотеке, а не к интересующей логике. Декомпилятор тоже создает имена сам: `FUN_140001000`, `param_1`, `local_20`. Это не знание, а временные ярлыки.

Name recovery - восстановление имен - важная часть анализа. Имя должно быть не красивым, а честным. Плохое имя: `check_license`, если ты еще не доказал, что проверяется лицензия. Лучшее

временное имя: `reads_config_flag_and_returns_bool`. Оно длинное, зато показывает наблюдение и ограничение.

В embedded символы иногда можно получить из утекшего ELF, map-файла или прошивки с невычищенными таблицами. Map file - файл карты линковки, где линкер пишет, какие символы по каким адресам оказались. Если у тебя raw firmware без символов, map file от похожей версии может быть золотом.

Часть IV. Форматы: PE, ELF и raw firmware

Глава 12. PE-файл Windows: не просто EXE, а договор с загрузчиком

PE - Portable Executable. Portable здесь исторически означает переносимый между вариантами Windows NT и архитектурами, а не «запускается везде». PE используется для `.exe`, `.dll`, драйверов и некоторых системных файлов.

PE начинается с DOS header. Первые байты часто `MZ`. Это наследие старого DOS-формата. Внутри DOS header есть поле `e_lfanew` - file offset до PE signature. PE signature обычно `PE\0\0`. Реверсер читает это как цепочку:

```
offset 0x00: DOS header, magic MZ
e_lfanew: где начинается NT headers
NT headers: signature, file header, optional header
section table: список секций
```

Optional header называется optional исторически, но для executable он практически обязателен. В нем лежат image base, entry point RVA, section alignment, file alignment, subsystem, data directories. Data directories - каталог важных таблиц: imports, exports, resources, relocations, exception directory, TLS и другие.

Section - секция. Типичные имена: `.text` для кода, `.rdata` для read-only данных, `.data` для изменяемых данных, `.pdata` для exception directory на x64, `.xdata` для unwind info. Имя секции не является гарантией. Защищенные или упакованные файлы могут называть секции как угодно. Но права секции и содержимое часто выдают роль.

Import table - таблица импортов. Она говорит, какие функции из каких DLL нужны программе. Например, `CreateFileW` из `Kernel32.dll` намекает на работу с файлами, `RegOpenKeyExW` - на реестр, `WinHttpSendRequest` - на сеть. Но импорт не доказывает поведение: функция может быть не вызвана, может быть использована через wrapper, а часть API может загружаться динамически через `GetProcAddress`.

Export table - таблица экспортов. Для DLL она показывает, какие функции модуль предоставляет наружу. В реверсе DLL exports часто дают первые осмысленные имена.

Relocations - таблица перемещений. Если модуль не удалось загрузить по предпочтительному image base, адреса внутри него нужно поправить. ASLR - Address Space Layout Randomization - рандомизация расположения адресного пространства - делает такие перемещения обычной ситуацией.

Exception directory на Windows x64 связан с `.pdata`. Там лежат записи Runtime Function: диапазон кода функции и RVA unwind info. Unwind info в `.xdata` описывает, что функция делает в прологе: сколько выделяет стека, какие регистры сохраняет, есть ли exception handler. Поэтому анализ PE для твоей ВКР обязан соединять формат файла, ABI и механику исключений.

Глава 13. ELF Linux: сегменты для загрузчика, секции для инструментов

ELF - Executable и Linkable Format. Он используется в Linux и многих Unix- подобных системах, а также в embedded-разработке как промежуточный или отладочный формат.

ELF начинается с ELF header. Первые байты:

```
7F 45 4C 46
```

`45 4C 46` - ASCII `ELF`. Header сообщает архитектуру, endian, тип файла, entry point, положение program header table и section header table.

Program headers описывают сегменты для загрузчика. Segment - кусок, который нужно отобразить в память. Например, `PT_LOAD` говорит: возьми байты файла от такого offset, положи в память по такому virtual address, дай права read/execute или read/write.

Sections удобны для линковщика и анализа: `.text`, `.rodata`, `.data`, `.bss`, `.symtab`, `.dynsym`, `.rela.plt`. Но во время загрузки Linux главными являются program headers. Это важное отличие от наивного взгляда: секция не равна сегменту.

Dynamic linking - динамическая линковка. Программа может не содержать код `printf`, `malloc`, `open`, а ссылаться на shared libraries. Для этого есть dynamic section, GOT и PLT.

GOT - Global Offset Table. Таблица адресов глобальных объектов и функций. PLT - Procedure Linkage Table. Набор маленьких переходников для вызова динамических функций. При lazy binding адрес функции может разрешаться при первом вызове. Для реверсера это означает: вызов `puts@plt` - это не сама функция `puts`, а переходник к ней.

Relocations в ELF бывают разных типов и зависят от архитектуры. Они говорят: в этом месте нужно записать адрес символа, разницу адресов, смещение GOT и так далее. Если читаешь loader или unpacker, relocations становятся центральной темой.

В embedded ELF часто используют как удобный файл со всеми символами и секциями, а в устройство прошивают raw binary или HEX. Поэтому можно иметь:

- `.elf` для отладки;
- `.bin` как чистый образ байтов;
- `.hex` как текстовый формат с адресами и данными;
- `.map` как карту символов.

Если у тебя есть только `.bin`, ты потерял часть контекста. Если есть `.elf` и `.map`, анализ становится намного богаче.

Глава 14. Raw firmware, Intel HEX, S-record, UF2 и контейнеры прошивок

Raw firmware - сырой образ прошивки. Это просто байты без универсального заголовка. В отличие от PE или ELF, raw-файл сам не обязан говорить, где его base address. Если открыть `.bin`, ты не всегда знаешь, должен ли первый байт лежать по адресу `0x00000000`, `0x08000000`, `0x10000000` или где-то еще.

Base address - базовый адрес образа. Для Cortex-M STM32 часто Flash начинается с `0x08000000`, но это не мировой закон. У других микроконтроллеров и режимов загрузки адреса другие. Поэтому реверсер ищет подсказки:

- первые слова похожи на stack pointer и reset handler;
- адреса в таблице попадают в известный диапазон Flash;
- строки и таблицы выровнены разумно;
- документация на чип говорит, где находится boot memory;
- обновление прошивки содержит metadata с адресом загрузки.

Intel HEX - текстовый формат, где каждая строка содержит адрес, тип записи, данные и checksum. Он удобен тем, что хранит не только байты, но и адреса. Motorola S-record похож по идее. UF2 используется, например, в некоторых образовательных и microcontroller-платформах: файл разбит на блоки с адресами, его удобно копировать как накопитель.

Firmware container - контейнер прошивки. Производители часто заворачивают реальный образ в собственный формат: header, версия, модель устройства, размер, checksum или hash, подпись, сжатие, несколько разделов. Тогда задача реверсера делится:

1. Распознать контейнер.
2. Достать payload - полезную нагрузку.
3. Понять, сжат payload или зашифрован.
4. Найти base address и архитектуру.
5. Загрузить в Ghidra/IDA/Rizin с правильными параметрами.

Checksum - контрольная сумма. Она помогает обнаружить случайное повреждение. Hash - криптографический отпечаток, например SHA-256. Signature - цифровая подпись, которая подтверждает, что образ подписан доверенным ключом. Не путай эти слова: checksum не является защитой от злонамеренной подмены, а подпись может предотвращать установку измененной прошивки.

Часть V. Загрузка и выполнение

Глава 15. Что делает loader, когда программа стартует

Loader - загрузчик исполняемого файла. В Windows это часть механизма загрузки PE и подсистемы процесса. В Linux - kernel loader плюс dynamic linker, если программа динамическая. В embedded loader может быть boot ROM, bootloader или вообще отсутствовать как отдельный слой.

Упрощенно Windows loader делает:

- создает адресное пространство процесса;
- отображает PE sections в память;
- применяет base relocations, если image base изменился;
- разрешает imports через DLL;
- настраивает memory protections;
- обрабатывает TLS callbacks;
- передает управление entry point.

TLS - Thread Local Storage, локальное хранилище потока. TLS callbacks могут выполняться до привычного entry point. Поэтому вредоносное ПО и защитные оболочки иногда используют TLS callbacks, чтобы запустить код раньше, чем аналитик ожидает.

Linux для ELF делает похожие, но не идентичные вещи: читает program headers, создает mappings, подготавливает stack с аргументами и environment, передает управление интерпретатору динамической линковки, если он указан в `PT_INTERP`. Dynamic linker затем разрешает shared libraries и relocations.

В embedded старт после reset выглядит иначе. Cortex-M после сброса читает первое слово vector table как начальный stack pointer, второе слово как reset handler. Затем выполняется reset handler, обычно из startup code:

```
установить стек
скопировать .data из Flash в RAM
очистить .bss
настроить clock и runtime
вызвать main
```

Startup code - стартовый код. Это не «логика приложения», но без него приложение не заработает. Реверсер embedded должен уметь его узнавать, чтобы не тратить часы на стандартные циклы копирования памяти.

Глава 16. Runtime: невидимая обвязка вокруг твоего кода

Runtime - среда выполнения. Это слой кода и правил, который помогает программе жить после старта. В C это может быть CRT - C Runtime. В C++ - инициализация глобальных объектов, обработка исключений, RTTI. В Go, Rust, .NET, Java runtime еще заметнее.

CRT до `main` может разобрать аргументы командной строки, настроить heap, инициализировать глобальные переменные, вызвать constructors. После `main` он может вызвать destructors и завершить процесс через системный API.

Exception handling - обработка исключений. В C++ исключение не просто «прыгает» куда-то. Runtime должен найти подходящий handler, размотать стек, вызвать destructors для объектов, которые выходят из области видимости. Для этого нужны metadata. В Windows x64 это связано с

`.pdata/.xdata`, в Linux часто с DWARF unwind information и `.eh_frame`.

RTTI - Run-Time Type Information, информация о типах во время выполнения. В C++ она помогает `dynamic_cast` и `typeid`, а для реверсера может дать имена классов и структуру виртуальных таблиц. Vtable - virtual table, таблица виртуальных функций. Если ты видишь массив указателей на функции в read-only секции, это может быть vtable, но нужно проверять xrefs и паттерны создания объектов.

В embedded runtime может быть минимальным. Но даже там есть startup, инициализация `.data/.bss`, обработчики прерываний, иногда newlib, FreeRTOS, Zephyr или vendor HAL. HAL - Hardware Abstraction Layer, слой абстракции над железом. Он делает код переносимее, но в реверсе добавляет много функций- оберток.

Часть VI. Embedded: железо, прошивки и низкий уровень без ОС

Глава 17. Что такое embedded и почему Windows-привычек мало

Embedded system - встроенная система. Это компьютер внутри устройства: роутер, датчик, станок, автомобильный блок, медицинский прибор, умная лампа, контроллер двигателя, плата с ESP32. Иногда там Linux. Иногда RTOS. Иногда вообще bare metal - голое железо без операционной системы.

Bare metal означает, что программа работает напрямую на микроконтроллере. Нет процесса, нет DLL, нет привычной виртуальной памяти, нет файловой системы в обычном смысле. Есть Flash с кодом, RAM с данными, периферия, прерывания и документация на чип.

Microcontroller - микроконтроллер. Это чип, где CPU, память и периферия часто находятся в одном корпусе. Microprocessor - микропроцессор - обычно требует внешней памяти и сложнее связан с ОС. Граница не всегда важна, но для embedded-реверса микроконтроллер означает: смотри memory map и reference manual.

Reference manual - официальное руководство по конкретному семейству чипов. Datasheet - документ с электрическими характеристиками, pinout, основными режимами. Programming manual может описывать ядро или flash programming. Errata - список известных ошибок железа. Реверсер embedded читает эти документы не от корки до корки, а по вопросу: «что означает адрес 0x40021018?», «какой бит включает UART?», «какое прерывание номер 37?».

Главное отличие embedded: запись по адресу может менять физический мир. В обычном процессе Windows запись в память меняет байты процесса. В микроконтроллере запись в MMIO-регистр может включить питание периферии, дернуть ножку GPIO, отправить байт по UART, запустить DMA или стереть Flash.

Поэтому анализ embedded требует трех карт одновременно:

- карта памяти: Flash, SRAM, периферия;
- карта исполнения: reset, interrupts, tasks, main loop;
- карта железа: GPIO, UART, SPI, I2C, timers, DMA, clocks.

Глава 18. Cortex-M: vector table, reset handler и interrupts

Cortex-M - семейство ARM-ядер для микроконтроллеров. Его любят в embedded, потому что оно относительно простое, распространенное и хорошо документированное.

Vector table - таблица векторов. В начале образа Cortex-M обычно лежит массив адресов. Первый элемент - начальный stack pointer. Второй - reset handler. Дальше идут handlers исключений и прерываний.

Пример начала firmware little-endian:

```
00 40 00 20  81 01 00 08
```

Первое слово: `0x20004000`. Это похоже на конец SRAM и начальный стек. Второе слово: `0x08000181`. Младший бит равен 1. Для Cortex-M это признак Thumb-state. Реальный адрес инструкции часто читают как `0x08000180`, а бит 0 показывает режим. Если не знать эту деталь, можно решить, что адрес «кривой».

Interrupt - прерывание. Это событие, которое заставляет CPU временно остановить текущий поток и перейти к handler. Например, пришел байт по UART, сработал таймер, закончилась передача DMA. NVIC - Nested Vectored Interrupt Controller - контроллер прерываний Cortex-M. Он управляет приоритетами и доставкой interrupts.

Exception и interrupt в ARM-документации связаны. Exception - более общий механизм входа в особый обработчик. Interrupt - внешний или периферийный вид такого события. Для реверса важно: handler может выполниться в любой момент относительно main loop, поэтому данные могут меняться не только в явном линейном коде.

Volatile - ключевое слово C, которое говорит компилятору: значение может измениться вне обычного потока программы, не оптимизируй чтение/запись как обычную переменную. MMIO-регистры почти всегда должны быть volatile. Если видишь в исходниках embedded:

```
*(volatile uint32_t *)0x40021018 = 1;
```

это не странная прихоть. Это запись в аппаратный регистр.

Глава 19. Периферия: GPIO, UART, SPI, I2C, DMA и clocks

Peripheral - периферийный блок внутри микроконтроллера: GPIO, UART, SPI, I2C, таймер, ADC, DMA. Он управляется через registers - регистры периферии, обычно memory-mapped.

GPIO - General Purpose Input/Output. Универсальный вход-выход. Это ножки чипа, которые можно настроить как вход или выход. Чтобы зажечь светодиод, часто нужно:

1. Включить clock для порта.
2. Настроить pin как output.
3. Записать бит в output data register или bit set/reset register.

Clock - тактирование. Периферия часто выключена по умолчанию для экономии энергии. Если не включить clock, запись в регистры GPIO может ничего не дать. Поэтому в firmware перед работой с UART или GPIO часто есть обращения к RCC или похожему блоку clock control.

UART - последовательный интерфейс. В реверсе UART важен как канал логов, консоль загрузчика, debug shell. Если в прошивке есть строки вроде `"login:"`, `"AT+"`, `"baud"`, `"UART"` или обращения к регистрам USART, это хороший след.

SPI и I2C - шины общения с внешними микросхемами: flash, sensors, displays, radio modules. В бинарнике это проявляется через функции передачи байтов, таблицы команд, задержки, управление chip select.

DMA - Direct Memory Access, прямой доступ к памяти. DMA-контроллер может переносить данные между памятью и периферией без постоянного участия CPU. В реверсе DMA усложняет картину: данные «сами» появляются в буфере, потому что железо их туда положило. Если не учитывать DMA, можно искать запись в буфер в коде и не найти.

Как распознать драйвер периферии в прошивке:

- много обращений к одному диапазону MMIO;
- битовые маски и сдвиги;
- ожидание флага готовности в цикле;
- включение clock перед настройкой;

- обработчики прерываний с похожими именами или адресами;
- строки логов с названиями интерфейсов;
- таблицы конфигурации pinmux.

Глава 20. RTOS: задачи, планировщик и контекст

RTOS - Real-Time Operating System, операционная система реального времени. Она нужна, когда устройству нужно выполнять несколько задач с предсказуемыми задержками: обработка связи, датчиков, управления, логов.

Task - задача или поток RTOS. Scheduler - планировщик, код, который решает, какая задача выполняется сейчас. Context switch - переключение контекста: сохранить регистры текущей задачи, выбрать другую, восстановить ее регистры и продолжить.

В FreeRTOS у каждой задачи есть свой стек. Это важно: в embedded может быть не один стек, а много стеков задач плюс interrupt stack или main stack, зависит от архитектуры и RTOS. Если анализируешь дамп RAM, нужно понимать, какой stack frame относится к какой задаче.

Queue - очередь. Semaphore - семафор. Mutex - взаимное исключение. Event group - группа событий. Эти сущности помогают задачам обмениваться сигналами и данными. В бинарнике RTOS-приложение часто содержит вызовы `xQueueSend`, `xQueueReceive`, `xSemaphoreTake`, `vTaskDelay`, если символы не удалены. Если символов нет, паттерны все равно можно восстановить по структурам и вызовам kernel API.

Zephyr - современная RTOS с большим количеством подсистем: device tree, drivers, networking, Bluetooth, logging. Device tree - описание железа в структурированном виде. В реверсе Zephyr-прошивки строки и таблицы device names могут помочь понять устройство.

RTOS добавляет сложность, но дает и подсказки. Там есть стандартные функции, таблицы задач, имена потоков, структуры очередей, сообщения логов. Главное - понять, что main уже не единственный центр мира. Поведение распределено между задачами, interrupt handlers и callbacks.

Часть VII. Инструменты реверсера

Глава 21. Hex editor и strings: маленькие инструменты до тяжелой артиллерии

Hex editor показывает байты. Это самый честный инструмент, потому что он почти ничего не интерпретирует за тебя. Он может подсветить ASCII, показать offset, позволить сравнить файлы. Перед тем как верить декомпилятору, полезно иногда посмотреть на сырой материал.

Strings - поиск читаемых строк. Строки часто дают первые зацепки: сообщения об ошибках, пути, URL, имена команд, версии, debug-логи. Но строки тоже нужно проверять. Они могут быть неиспользуемыми, зашифрованными, сжатыми, сгенерированными библиотекой или специально ложными.

File carving - извлечение вложенных файлов по сигнатурам. В прошивке могут лежать gzip, squashfs, certificates, images, scripts. Binwalk помогает искать такие структуры. Но binwalk не оракул: найденная сигнатура может быть случайным совпадением. Всегда проверяй offset, размер, entropy, возможность распаковки и связь с контейнером.

Entropy - энтропия, грубо говоря мера «похожести на случайность». Сжатые и зашифрованные данные часто имеют высокую entropy. Но высокая entropy не говорит, сжатие это или шифрование. Нужны дополнительные признаки: заголовки, ключи, код распаковки, структура контейнера.

Глава 22. Ghidra, IDA, Rizin, Binary Ninja: что делает дизассемблер

Disassembler - дизассемблер. Он берет байты и пытается превратить их в assembly. Decompiler - декомпилятор. Он пытается поднять assembly в псевдокод, похожий на C. Это разные задачи. Декомпилятор стоит на плечах дизассемблера и анализа потока управления.

Ghidra хороша как бесплатная мощная платформа: поддерживает много архитектур, имеет decompiler, скрипты, data type manager. IDA исторически сильна в интерактивном анализе и экосистеме. Binary Ninja удобна современным API и промежуточными представлениями. Rizin и Cutter полезны как открытая экосистема для CLI/GUI-анализа.

Но любой инструмент делает предположения:

- где начинается код;
- где заканчивается функция;
- какая архитектура и endian;
- что является pointer;
- какие таблицы являются jump tables;
- какие типы у аргументов;
- где данные, а где код.

Твоя работа - не нажать кнопку decompile, а превратить предположения в проверенные выводы. Если Ghidra неправильно определила base address firmware, все xrefs могут быть бессмысленными. Если выбран ARM вместо Thumb, код развалится. Если PE-секция распаковывается только во время выполнения, статический анализ увидит оболочку, а не payload.

Практический порядок:

1. Сначала определить формат, архитектуру, endian и base address.

2. Проверить entry point и первые переходы.
3. Найти строки и xrefs.
4. Разметить известные API, MMIO, imports, vectors.
5. Переименовывать функции осторожно, по наблюдаемой роли.
6. Проверять спорные места в debugger или emulator.

Глава 23. Debugger: наблюдение живого поведения

Debugger - отладчик. Он позволяет запускать программу под контролем: ставить breakpoints, смотреть регистры, память, стек, инструкции, шагать по коду. Static analysis говорит, что может произойти. Dynamic analysis показывает, что происходит в конкретном запуске.

Breakpoint - точка останова. Software breakpoint на x86 часто ставится заменой байта инструкции на `int3` (`0xCC`). Когда CPU доходит до него, возникает exception, debugger получает управление. Hardware breakpoint использует специальные регистры CPU и может сработать на выполнение, чтение или запись адреса без изменения кода.

Single step - пошаговое выполнение. На x86 это связано с trap flag. На ARM и embedded механизм зависит от debug hardware. Watchpoint - точка наблюдения за памятью: остановиться, когда адрес читают или пишут.

WinDbg важен для Windows: умеет работать с symbols, exceptions, stack, PE-модулями, kernel/user debugging. x64dbg удобен для пользовательских программ. GDB широко используется в Linux и embedded. OpenOCD связывает GDB с JTAG/SWD-адаптером для микроконтроллеров.

Dynamic analysis опасен, если файл неизвестный. Для Windows malware нужен изолированный стенд: виртуальная машина, snapshot, отключенная или контролируемая сеть, мониторинг файлов/реестра/процессов. Для embedded нужна осторожность с устройством: неверная запись во Flash или fuse может его заблокировать.

Глава 24. Emulation: когда железа нет или запуск опасен

Emulation - эмуляция. Эмулятор воспроизводит поведение процессора или системы программно. QEMU умеет system emulation для многих архитектур и user mode emulation для запуска отдельных Linux-программ другой архитектуры.

Unicorn Engine дает CPU-emulation как библиотеку: можно исполнить маленький фрагмент кода, подставив память и регистры. Это удобно для unpacking, дешифровки строк, проверки функций без полного устройства.

Но эмуляция имеет границы. Прошивка может ждать конкретный MMIO-регистр, таймер, interrupt, датчик, radio chip. Если эмулятор этого не моделирует, код застрянет. Тогда нужно писать stubs - заглушки, которые возвращают ожидаемые значения, или переходить к hardware debugging.

Simulation, emulation и virtualization часто путают. Виртуализация запускает код почти напрямую на похожем CPU, изолируя ОС. Эмуляция может исполнять код другой архитектуры. Симуляция может моделировать поведение на более абстрактном уровне. Для реверса важно не слово само по себе, а точность: какие эффекты среды воспроизведены, а какие нет.

Часть VIII. Методика настоящего исследования

Глава 30. Как строить дальнейший путь

Первый маршрут - Windows reverse engineering. Углубляйся в PE/COFF, Windows Internals, WinDbg, ETW, exception handling, loaders, DLL injection как защитно-исследовательскую тему, kernel drivers только после уверенной базы.

Второй маршрут - Linux reverse engineering. ELF, System V ABI, glibc, dynamic linker, ptrace, seccomp, eBPF, kernel modules, containers, rootfs analysis.

Третий маршрут - embedded reverse engineering. Cortex-M, STM32, ESP32, RISC-V, JTAG/SWD, OpenOCD, UART, logic analyzer, bootloaders, RTOS, firmware formats, hardware security, fault injection как отдельная продвинутая ветка.

Четвертый маршрут - compiler и binary analysis. LLVM IR, optimization passes, data-flow analysis, symbolic execution, decompilation theory, lifting, intermediate representations.

Не распыляйся одинаково на все. Выбери одну практическую линию на месяц. Например: «каждую неделю разбираю один маленький PE и пишу отчет» или «каждую неделю беру один Cortex-M пример и размечаю MMIO». Универсальная база нужна не для хаоса, а для переноса мышления.

Часть X. Глубокие разборы: медленно, от начала до конца

Эта часть нужна, чтобы убрать главный страх новичка: «я вроде понял слова, но не понимаю, как оно все реально происходит». Ниже мы не добавляем новые модные термины ради терминов. Мы берем одну ситуацию и протаскиваем ее через слои: файл, формат, загрузчик, память, регистры, стек, инструкции, документация, инструмент, отчет.

Глава 31. Один байт проходит путь от файла до инструкции

Представь, что у нас есть PE-файл. Внутри секции `.text` лежит байт `E8`. В x86/x64 это часто opcode инструкции `call rel32`. Opcode - код операции, байт или несколько байтов, которые говорят процессору, какую инструкцию выполнять. Но сам по себе `E8` еще не «вызов функции». Он становится вызовом только если мы читаем его в правильной архитектуре, в правильном месте, как код, а не как данные.

Путь начинается на диске. Файл хранит байты подряд. Hex editor показывает file offset, например `0x1200`. Но когда Windows loader загружает PE в память, секция `.text` попадает по virtual address, например `0x140001000`. Если байт `E8` лежит на file offset `0x1200`, его runtime VA может быть `0x140001000`. Это не изменение байта. Это изменение системы координат.

Затем CPU получает instruction pointer `rip = 0x140001000`. Он читает байт `E8` и знает по ISA: дальше идут 4 байта signed displacement. Displacement - смещение, которое может быть положительным или отрицательным. Инструкция `call rel32` не хранит абсолютный адрес функции. Она хранит расстояние от следующей инструкции до target. Target - цель перехода.

Допустим, байты такие:

```
E8 2B 01 00 00
```

`2B 01 00 00` little-endian дает `0x12B`. Длина инструкции 5 байт. Адрес следующей инструкции:

```
0x140001000 + 5 = 0x140001005
```

Target:

```
0x140001005 + 0x12B = 0x140001130
```

Теперь команда `call` делает две вещи. Первое: кладет return address `0x140001005` на стек. Второе: меняет `rip` на `0x140001130`. Вот почему вызов функции - не просто «перейти». Нужно еще запомнить дорогу назад.

Если этот же байт `E8` лежит в `.rdata` как часть таблицы или строки, дизассемблировать его как код неправильно. Поэтому инструменты используют entry point, xrefs, секционные права, flow analysis и эвристики, чтобы решить, где код. Но решение инструмента можно и нужно проверять.

Отчетный вывод по такому месту должен быть точным:

```
По VA 0x140001000 находится инструкция call rel32.
Цель вычисляется как next_rip + displacement:
0x140001005 + 0x12B = 0x140001130.
Перед переходом CPU помещает return address 0x140001005 на стек.
```

В этом маленьком примере встретились сразу file offset, VA, opcode, little-endian, displacement, instruction length, `rip`, stack, return address. Если каждое слово разложено, фрагмент становится не страшным, а проверяемым.

Глава 32. Как PE-секция превращается в память процесса

PE-секция на диске имеет два размера, которые часто путают: `SizeOfRawData` и `VirtualSize`. `SizeOfRawData` - сколько байтов секции реально лежит в файле. `VirtualSize` - сколько места секция занимает в памяти после загрузки. Они могут отличаться.

Представь секцию `.data`:

```
PointerToRawData = 0x4000
SizeOfRawData    = 0x0800
VirtualAddress   = 0x9000
VirtualSize      = 0x1200
ImageBase        = 0x140000000
```

На диске секция начинается по file offset `0x4000` и занимает `0x800` байт. В памяти она начинается по VA:

```
ImageBase + VirtualAddress = 0x140000000 + 0x9000 = 0x140009000
```

Но в памяти ей нужно `0x1200` байт. Что с разницей между `0x800` и `0x1200`? Загрузчик дополняет хвост нулями. Это нормально. Так работают данные, которые должны существовать в памяти, но не обязаны занимать место в файле.

Теперь допустим, в data directory указан RVA `0x9500`. Как найти байты в файле? Сначала понять, в какую секцию попадает RVA. Секция `.data` покрывает RVA `0x9000..0xA200`. RVA `0x9500` внутри. Смещение внутри секции:

```
0x9500 - 0x9000 = 0x500
```

File offset:

```
PointerToRawData + 0x500 = 0x4000 + 0x500 = 0x4500
```

Это называется RVA-to-file-offset conversion. Новичок часто делает ошибку: прибавляет RVA к началу файла или ищет VA как file offset. Правильный путь: RVA -> найти секцию -> offset внутри секции -> PointerToRawData.

Почему это важно для unwind analyzer? Exception directory дает RVA таблицы Runtime Function. Чтобы прочитать ее из файла, нужно надежно превратить RVA в file offset и проверить границы. Если RVA указывает за пределы секций, если размер таблицы странный, если запись выходит за файл, инструмент должен не падать, а сообщать проблему.

В инженерном коде эта логика должна быть написана осторожно:

- использовать 64-битную арифметику для промежуточных сумм;
- проверять переполнения;
- проверять, что RVA попадает в секцию;
- проверять, что нужный диапазон умещается в raw data;
- помнить, что `VirtualSize` и `SizeOfRawData` различаются;
- не доверять именам секций.

Так форматы файлов превращаются из «таблиц ради таблиц» в защиту от ошибок. Каждое поле PE нужно не для зубрежки, а для ответа на вопрос: где эти байты будут в памяти и как безопасно найти их в файле?

Глава 33. Windows x64 unwind: зачем программе описание собственного стека

Unwind кажется странной темой, пока не представить ошибку в середине программы. Выполняется функция C. Она выделила 0x80 байт на стеке, сохранила `rbx` и `rsi`, вызвала функцию D, а внутри D произошло исключение. Windows нужно понять: какие функции сейчас активны, какие destructors вызвать, где handler, как восстановить регистры, куда передать управление.

Если бы все функции имели одинаковый frame pointer chain, ОС могла бы идти по `rbp`. Но x64 ABI не требует всегда использовать `rbp` как frame pointer. Оптимизатор может использовать `rbp` для обычных данных. Поэтому Windows использует unwind metadata.

В PE exception directory хранит массив Runtime Function entries. Каждая запись описывает диапазон кода:

```
BeginAddress
EndAddress
UnwindInfoAddress
```

Это RVA, не file offsets. Begin/End говорят, для какого диапазона инструкций действует запись. UnwindInfoAddress ведет в `.xdata`, где лежит UNWIND_INFO.

UNWIND_INFO описывает действия пролога: выделить стек, сохранить nonvolatile register, установить frame register и т.д. Важно: metadata описывает не весь код функции, а то, что нужно для восстановления состояния при движении назад.

Пример пролога:

```
push rbx
sub  rsp, 40h
```

Для unwind это означает:

- `rbx` сохранен на стеке;
- `rsp` уменьшен на 0x40;
- return address лежит выше текущего frame;
- чтобы размотать кадр, нужно обратить эти действия.

Unwind code - маленькая запись внутри UNWIND_INFO. Она говорит, какая операция была сделана и на каком offset пролога. Например, push nonvolatile register или allocate small stack. Эти записи упакованы компактно, поэтому парсер должен внимательно читать поля, выравнивание и count.

Почему это глубоко связано с реверсом? Потому что metadata иногда помогает найти границы функций даже там, где дизассемблер сомневается. Если Runtime Function говорит, что функция идет от RVA A до RVA B, это сильный структурный факт. Но он тоже требует проверки: файл может быть поврежден, защищен, собран нестандартно или намеренно запутан.

В твоей ВКР статический анализ `.pdata/.xdata` и динамическая проверка через Windows Debug API дополняют друг друга. Статика говорит: «по формату файла здесь должна быть такая unwind-запись». Динамика говорит: «при реальном выполнении и событиях отладчика стек и исключения ведут себя так». Сильный инструмент не выбирает одну реальность. Он сопоставляет их.

Глава 34. ELF-загрузка: почему sections не главное для kernel

Многие начинают ELF с section headers: `.text`, `.data`, `.bss`, `.rodata`. Это удобно для человека, но загрузчик Linux смотрит прежде всего на program headers. Program header типа `PT_LOAD` говорит: загрузить этот диапазон файла в память.

Пример:

```
Type    Offset  VirtAddr    FileSiz  MemSiz   Flags
LOAD    0x0000  0x400000    0x1000   0x1000   R E
LOAD    0x2000  0x402000    0x0800   0x1000   RW
```

Первый сегмент read/execute: код и read-only данные. Второй read/write: изменяемые данные. `FileSiz` может быть меньше `MemSiz`: хвост в памяти заполняется нулями, как `.bss`.

Dynamic linker подключается, если ELF содержит `PT_INTERP`, например путь к `/lib64/ld-linux-x86-64.so.2`. Тогда kernel загружает интерпретатор, а тот уже разрешает shared libraries, relocations, GOT/PLT.

Когда ты видишь вызов:

```
call puts@plt
```

это не обязательно прямой адрес `puts` в libc. PLT-entry может прыгать через GOT. При первом вызове dynamic linker узнает реальный адрес и записывает его в GOT. После этого следующие вызовы идут быстрее.

Для реверсера это означает: если хочешь понять реальный target динамической функции во время выполнения, смотри не только статический PLT, но и состояние GOT в процессе. GDB, `/proc/<pid>/maps`, `readelf -r`, `objdump -d` и динамический запуск вместе дают полную картину.

В statically linked ELF часть библиотечного кода уже внутри файла. Тогда бинарник больше, imports меньше, но analysis сложнее из-за большого объема runtime-кода. Нужно уметь отличать стандартные функции от логики программы.

Глава 35. От C-структуры к offset-ам в дизассемблере

Структура в C:

```
struct Packet {
    uint16_t type;
    uint16_t length;
    uint8_t  flags;
    uint8_t  reserved[3];
    uint32_t crc;
};
```

В исходнике она выглядит понятно. В бинарнике имени `Packet` может не быть. Останутся обращения по offset-ам:

```
movzx eax, word ptr [rcx]
movzx edx, word ptr [rcx+2]
movzx r8d, byte ptr [rcx+4]
mov   r9d, dword ptr [rcx+8]
```

Если Windows x64 ABI говорит, что `rcx` - первый аргумент, можно предположить: функция получает pointer на структуру. Обращения `[rcx]`, `[rcx+2]`, `[rcx+4]`, `[rcx+8]` похожи на поля. Но нужно объяснить padding - выравнивание.

Padding - дополнительные байты между полями, которые компилятор вставляет для выравнивания. В структуре выше после `flags` есть `reserved[3]`, поэтому `crc` красиво лежит по offset 8. Но даже без явного reserved компилятор мог бы добавить padding, чтобы `uint32_t` лежал

по адресу, кратному 4.

Data structure recovery делается постепенно:

1. Найти базовый pointer.
2. Собрать все offset-ы, которые читаются и пишутся.
3. Определить размер доступа: byte, word, dword, qword.
4. Проверить, есть ли строки, magic, ranges, checks.
5. Создать структуру в Ghidra/IDA.
6. Переименовать поля по роли, а не по фантазии.

Плохое имя поля: `license_flag`, если ты только видел bit check. Лучшее имя: `flags` или `status_flags`, пока не доказано больше. Потом, если xrefs показывают, что bit 2 ведет к сообщению `"license expired"`, можно уточнить.

В embedded та же логика работает для MMIO:

```
typedef struct {
    volatile uint32_t CR;
    volatile uint32_t CFGR;
    volatile uint32_t CIR;
    volatile uint32_t APB2RSTR;
    volatile uint32_t APB1RSTR;
    volatile uint32_t AHBENR;
    volatile uint32_t APB2ENR;
} RCC_TypeDef;
```

Если base address RCC известен из reference manual, обращение `[0x40021018]` можно назвать `RCC_APB2ENR`. Тогда код перестает быть мутным набором адресов и становится понятным включением clock.

Глава 36. Pointer: число, которое обещает быть адресом

Pointer - указатель. В C указатель хранит адрес объекта. Но в бинарнике любой pointer выглядит как число. Реверсер должен доказать, что число действительно используется как адрес.

Признаки pointer:

- значение попадает в диапазон памяти процесса или firmware;
- по нему читают или пишут через квадратные скобки в assembly;
- оно передается в функцию, которая ожидает buffer/string/struct;
- на него есть relocations;
- оно встречается в таблице адресов;
- xrefs показывают использование как target вызова или доступа.

False pointer - ложный указатель. Число может случайно быть похоже на адрес. Например, CRC, timestamp, float, hash fragment. Поэтому нельзя просто настроить Ghidra «делай pointer из всего, что похоже». Это может испортить анализ.

Null pointer - нулевой указатель, обычно `0`. Dereference - разыменование, чтение или запись по адресу, который хранится в pointer. Если программа делает dereference null pointer, обычно будет crash или fault. Но embedded может иметь память по адресу 0, например vector table, поэтому правила зависят от архитектуры.

Function pointer - указатель на функцию. Это число, по которому потом делают indirect call:

```
call qword ptr [rax+18h]
```

Такая инструкция не говорит статически, какая функция будет вызвана. Нужно понять, что лежит в `rax`, что за структура, что находится по offset `0x18`. Так восстанавливаются callbacks, virtual calls, driver tables, RTOS hooks.

Глава 37. Interrupt, fault и exception: три слова, которые нельзя мешать в кучу

В разных мирах эти слова используются по-разному, поэтому важно понимать роль, а не переводить механически.

Interrupt - событие, часто внешнее или периферийное. Таймер сработал, UART получил байт, GPIO изменил уровень. CPU переключается на handler.

Fault - ошибка выполнения или аппаратное исключение. Например, доступ к недопустимой памяти, деление на ноль, undefined instruction. В Cortex-M есть HardFault, MemManage Fault, BusFault, UsageFault. В Windows user mode похожую роль может играть access violation exception.

Exception - более общее «исключительное событие», которое требует особого пути обработки. В C++ exception - языковой механизм `throw/catch`. В CPU exception - аппаратное событие. В Windows SEH - structured exception handling - системный механизм обработки exceptions.

Почему это важно? Потому что слово handler может означать разные вещи:

- interrupt handler в vector table Cortex-M;
- exception handler Windows;
- C++ catch handler;
- signal handler Linux;
- callback в библиотеке.

Если в отчете написать «срабатывает exception», нужно уточнить слой. Это hardware fault? Windows exception? C++ exception? RTOS event? Без слоя фраза слишком мутная.

Глава 38. Cortex-M reset: полный путь от питания до main

Устройство получает питание. После reset ядро Cortex-M берет адрес vector table. Обычно после сброса таблица начинается с адреса 0, но микроконтроллер может remap-ить память так, что Flash видна с нуля или с `0x08000000`.

CPU читает первое слово: initial MSP. MSP - Main Stack Pointer. Это значение попадает в `sp`. Затем CPU читает второе слово: reset vector. Младший бит указывает Thumb-state, поэтому фактический адрес инструкции получается очисткой bit 0.

Дальше выполняется reset handler. В типичном startup он:

- копирует initialized data из Flash в RAM;
- очищает `.bss`;
- вызывает `SystemInit`;
- вызывает constructors C++, если они есть;
- вызывает `main`;

- если `main` вернулся, уходит в бесконечный цикл или reset.

Копирование `.data` выглядит как цикл:

```
load byte/word from flash_source
store byte/word to ram_destination
increment pointers
compare c end
branch if не done
```

Очистка `.bss` похожа, но вместо чтения из Flash записывает нули. Новичок может принять эти циклы за алгоритм приложения. Опытный реверсер узнает startup pattern и быстро проходит дальше.

Где взять адреса `.data` и `.bss`? В исходном проекте их задает linker script. В бинарнике подсказки могут быть в константах reset handler, в таблицах, в map file, в ELF symbols, если они есть. В raw firmware приходится восстанавливать.

Глава 39. MMIO-разбор по-настоящему: включаем GPIO

Пусть в firmware найден фрагмент:

```
ldr r0, =0x40021018
ldr r1, [r0]
orr r1, r1, #0x10
str r1, [r0]
```

Без документации это просто адрес и битовая операция. С документацией это может стать понятным действием. Допустим, reference manual говорит: `0x40021000` - RCC base, offset `0x18` - APB2ENR, bit 4 - IOPCEN, включение clock для GPIOC.

Тогда разбор:

```
0x40021018 = RCC base 0x40021000 + offset 0x18
чтение [r0] получает текущие enable bits
orr с 0x10 выставляет bit 4
запись [r0] включает clock GPIOC
```

Почему сначала читают, потом OR, потом пишут? Это read-modify-write. Нужно изменить один бит, не затерев остальные. Если просто записать `0x10`, можно случайно выключить другие включенные периферии.

Read-modify-write важен и в многопоточности, и в embedded. Иногда регистры имеют специальные set/reset registers, чтобы избежать опасности. Например, GPIO BSRR на STM32 позволяет выставить или сбросить отдельные биты без чтения текущего состояния output data register.

Отчетная формулировка:

```
Функция включает тактирование GPIOC через RCC_APB2ENR bit 4.
Основание: запись по MMIO address 0x40021018, соответствие reference manual,
маска 0x10, паттерн read-modify-write.
```

Так выглядит доказательный embedded reverse engineering: не «кажется, это GPIO», а цепочка адрес -> register -> bit -> операция -> эффект.

Глава 40. UART как след в прошивке

UART часто становится первым живым каналом общения с устройством. В прошивке он оставляет много следов:

- строки логов;
- форматные строки `printf`;

- функции `putchar`, `uart_write`, `HAL_UART_Transmit`;
- MMIO-адреса USART;
- настройки baud rate;
- interrupt handler приема;
- ring buffer.

Baud rate - скорость передачи, например 115200. В коде настройка baud rate может выглядеть как запись делителя в регистр. Чтобы понять ее, нужно знать частоту clock. Поэтому UART связан с clock tree: без понимания тактирования не всегда ясно, почему записано именно такое число.

Ring buffer - кольцевой буфер. Это массив, где head и tail бегают по кругу. Он часто используется для UART receive. В бинарнике можно узнать ring buffer по операциям modulo или маске размера, например `index & 0xFF` для буфера 256 байт.

Если устройство имеет скрытую консоль, команды могут лежать таблицей:

```
"help"  -> function pointer
"reboot" -> function pointer
"version" -> function pointer
```

В Ghidra это может выглядеть как чередование pointer на строку и pointer на функцию. Если разметить структуру command entry, весь shell станет читаемым.

Глава 41. DMA: почему код не записывает буфер, а данные появляются

DMA ломает простую картину «если данные изменились, значит CPU выполнил store». DMA-контроллер сам переносит данные. CPU только настраивает: source, destination, length, mode, interrupt on complete.

В firmware это может выглядеть так:

```
DMA channel source      = UART data register
DMA channel destination = rx_buffer
DMA channel length      = 256
DMA enable bit          = 1
```

После этого UART-прием может наполнять `rx_buffer` без инструкций записи в самом обработчике. Реверсер, который ищет `str` в этот buffer, может ничего не найти. Нужно искать настройку DMA.

Cache coherency - согласованность кэша - еще один слой. На Cortex-M7 или Cortex-A DMA и CPU cache могут видеть память по-разному, если не делать cache clean/invalidate. В реверсе это проявляется в вызовах функций очистки кэша перед/после DMA.

Глава 42. Embedded Linux firmware: когда прошивка внутри похожа на маленький Linux

Многие роутеры, камеры и IoT-устройства используют embedded Linux. Firmware update может содержать bootloader, kernel, device tree, root filesystem, configs, web interface, scripts, binaries.

Root filesystem - корневая файловая система. Часто это SquashFS, CramFS, ext, UBIFS. Binwalk может найти и извлечь ее. После извлечения реверс уже не только про assembly:

- `/etc/init.d` или systemd units показывают старт сервисов;
- `/etc/passwd`, configs, certificates показывают настройки;

- web files показывают интерфейс;
- binaries в `/bin`, `/sbin`, `/usr/bin` требуют ELF-анализа;
- kernel modules требуют понимания Linux kernel ABI;
- device tree blob описывает железо.

Device tree - дерево устройств. В Linux embedded оно сообщает kernel, какое железо есть на плате: UART, SPI, GPIO, flash partitions, clocks, interrupts. Если есть `.dtb`, его можно декомпилировать в `.dts` и узнать memory map и peripherals.

U-Boot часто встречается как bootloader. Его environment может содержать bootargs, адреса загрузки kernel, rootfs, partition layout. Строки вроде `bootcmd`, `console=ttyS0`, `mtdparts` дают сильные подсказки.

Глава 43. Packing, compression, encryption: почему видимый код не всегда главный

Packer - упаковщик. Он преобразует программу так, что исходный код или основной payload хранится сжатым/зашифрованным/запутанным, а при запуске маленький stub распаковывает его в память и передает управление. Stub - оболочка распаковки.

Compression - сжатие. Его цель уменьшить размер. Encryption - шифрование. Его цель скрыть содержимое без ключа. Obfuscation - запутывание, чтобы затруднить анализ. Эти механизмы могут сочетаться.

Признаки упаковки PE:

- необычные имена секций;
- мало imports;
- высокая entropy секции;
- entry point в странной секции;
- код выделяет память, пишет туда и прыгает;
- API вроде `VirtualAlloc`, `VirtualProtect`, `LoadLibrary`, `GetProcAddress`.

В firmware похожие признаки:

- контейнер с header и compressed payload;
- zlib/gzip/lzma magic;
- код распаковки перед основным образцом;
- encrypted partition;
- подпись и hash в metadata.

Важно: не называй все «зашифровано». Если binwalk распаковал gzip, это сжатие. Если payload выглядит случайным, но ключ и алгоритм не найдены, это только гипотеза шифрования. Точность языка - часть профессионализма.

Глава 44. Как читать официальную документацию, чтобы не утонуть

Официальные документы огромны. Intel SDM, Arm manuals, Microsoft PE/COFF, ABI specs, reference manuals STM32 - это сотни и тысячи страниц. Новичок пытается читать подряд и быстро сдается. Так делать не нужно.

Документацию читают вопросами.

Вопрос: «Где первый аргумент Windows x64 функции?» Идешь в Microsoft x64 calling convention, находишь register usage.

Вопрос: «Почему функция без rbp все равно unwinds?» Идешь в x64 exception handling и prolog/epilog docs.

Вопрос: «Что означает RVA в exception directory?» Идешь в PE format.

Вопрос: «Почему reset vector Cortex-M имеет младший бит 1?» Идешь в Arm Cortex-M architecture/manual и читаешь про Thumb state.

Вопрос: «Что такое `PT_LOAD`?» Идешь в ELF gABI.

Составляй собственные короткие карточки документации:

```
Тема: Windows x64 shadow space
Источник: Microsoft x64 calling convention
Правило: caller reserves 32 bytes for four register parameters
Где видно: sub rsp, 28h до call; stack layout
Ошибка новичка: считать эти 32 bytes локальными переменными callee
```

Такие карточки превращают гигантский manual в рабочий инструмент.

Глава 45. Как объяснять английские термины самому себе

Не зубри `relocation` как «релокация». Раскладывай:

`relocate` - переместить. Relocation в бинарнике - запись о месте, которое нужно исправить, если код/данные оказались по другому адресу. Она связывает формат файла, linker и loader.

Не зубри `offset` как «оффсет». Спрашивай: offset от чего? От начала файла? От начала секции? От base address? От `rbp`? От начала структуры? Один и тот же hex `0x20` может быть field offset, stack offset, RVA, file offset или MMIO register offset.

Не зубри `stack`. Спрашивай: какой стек? Стек потока Windows? Стек Linux thread? Main stack pointer Cortex-M? Process stack pointer Cortex-M? Стек задачи FreeRTOS? Стек bootloader? Один термин, разные контексты.

Не зубри `handler`. Спрашивай: кто вызывает handler? CPU при interrupt? Windows при exception? C++ runtime при throw? Библиотека при callback? RTOS при event? Роль определяется вызывающим механизмом.

Такой способ медленнее в первый месяц, но потом резко ускоряет обучение. Ты перестаешь бояться английских слов, потому что каждое слово становится дверью к конкретной механике.

Часть XI. Практикум с разбором мышления

Глава 46. Практикум: маленькая функция x64 до уровня регистров

Исходник:

```
int is_between(int x, int low, int high) {
    return x >= low && x <= high;
}
```

Возможный assembly Windows x64:

```
cmp ecx, edx
jl  false_case
cmp ecx, r8d
jg  false_case
mov eax, 1
ret
false_case:
xor eax, eax
ret
```

Windows x64 говорит: `ecx` - первый аргумент `x`, `edx` - второй `low`, `r8d` - третий `high`. `cmp ecx, edx` сравнивает `x` и `low`. `jl` - jump if less, переход если меньше для signed comparison. Если `x < low`, функция возвращает 0. Потом `cmp ecx,r8d`: если `x > high`, вернуть 0. Иначе положить 1 в `eax`.

Здесь нет стека. Функция leaf, локальных переменных нет, вызовов нет. Это не ошибка. Не каждая функция обязана иметь prologue. Если новичок ищет `push rbp` и не находит, он может решить, что это не функция. На самом деле границы функции видны по control flow и `ret`.

Если бы это была Linux System V функция, аргументы были бы в `edi`, `esi`, `edx`. Та же логика, другой ABI. Поэтому при переносе анализа между Windows и Linux нельзя автоматически читать `ecx` как первый аргумент.

Глава 47. Практикум: switch и jump table

Исходник:

```
switch (cmd) {
case 1: return start();
case 2: return stop();
case 3: return status();
default: return -1;
}
```

Компилятор может сделать цепочку сравнений, а может jump table - таблицу адресов переходов. Jump table - массив target-адресов, индексируемый значением `cmd`.

Паттерн:

```
cmp ecx, 3
ja  default_case
lea rax, [rip + table]
movsxd rcx, dword ptr [rax + rcx*4]
add rcx, rax
jmp rcx
```

Это выглядит сложнее, чем `if`. Но смысл: проверить диапазон, взять из таблицы смещение, прибавить base, перейти. Jump table часто лежит в read-only data рядом с кодом. Декомпилятор может восстановить `switch`, а может ошибиться, если таблица смешана с данными.

Для реверса командных обработчиков jump table важна. В firmware команды протокола, состояния автомата, menu entries часто превращаются в таблицы переходов или массивы function pointers. Не игнорируй data xrefs рядом с indirect jumps.

Глава 48. Практикум: строка как повод найти функцию

Строка `"invalid header"` сама по себе не доказывает формат. Но xref к ней ведет к месту, где программа сообщает ошибку. Если рядом видны проверки:

```
cmp word ptr [rcx], 5A4Dh
jne invalid_header
mov eax, [rcx+3Ch]
cmp dword ptr [rcx+rax], 00004550h
jne invalid_header
```

Это похоже на проверку PE: `MZ`, `e_lfanew`, `PE\0\0`. Теперь строка стала не просто текстом, а частью доказательства. Можно переименовать функцию осторожно: `validate_pe_headers` или `checks_mz_and_pe_signature`.

Если в коде дальше читается section table и data directories, уверенность растет. Если проверка только поверхностная, имя должно оставаться скромным. Хорошие имена отражают глубину доказательства.

Глава 49. Практикум: восстановление структуры по offset-ам

Допустим, функция получает `rcx` и делает:

```
cmp dword ptr [rcx], 46574152h
mov eax, [rcx+4]
mov edx, [rcx+8]
lea r8,  [rcx+10h]
```

`46574152h` little-endian в байтах будет `52 41 57 46`, ASCII `RAWF`. Это может быть magic header. `[rcx+4]` и `[rcx+8]` похожи на поля size/version или offset/length. `[rcx+0x10]` может быть начало payload.

В Ghidra можно создать структуру:

```
struct RawfHeader {
    uint32_t magic;
    uint32_t field_04;
    uint32_t field_08;
    uint32_t field_0c;
    uint8_t  payload[];
};
```

Пока поля не доказаны, не называй их слишком уверенно. После проверки xrefs и диапазонов `field_04` может стать `payload_size`, а `field_08` - `checksum`. Но сначала факты.

Глава 50. Практикум: маленький bootloader

Bootloader часто делает:

- проверить, есть ли валидное приложение;
- проверить checksum/signature;
- при необходимости принять update;
- записать Flash;
- передать управление application.

Для Cortex-M переход в application обычно требует:

1. Отключить interrupts или привести систему в безопасное состояние.
2. Установить MSP из первого слова application vector table.

3. Взять reset handler из второго слова.
4. Перейти по reset handler.

Псевдокод:

```
uint32_t app_sp = *(uint32_t *)APP_BASE;
uint32_t app_reset = *(uint32_t *)(APP_BASE + 4);
__set_MSP(app_sp);
((void (*)(void))app_reset)();
```

В assembly это может выглядеть как загрузка двух слов из Flash и indirect branch. Если `APP_BASE = 0x08008000`, адреса вокруг этой границы становятся важными. В отчете нужно указать не только «есть переход», но и как проверена валидность app image.

Глава 51. Практикум: FreeRTOS-задачи в бинарнике

Если символы не удалены, FreeRTOS легко узнается по именам вроде `xTaskCreate`, `vTaskStartScheduler`, `xQueueSend`. Если символов нет, ищем строки имен задач, размеры стеков, приоритеты, таблицы инициализации.

`xTaskCreate` обычно получает pointer на функцию задачи, имя, размер стека, параметр, priority и handle. В assembly это выглядит как подготовка нескольких аргументов и вызов одной kernel-функции. Если ABI известен, можно восстановить:

```
task function = sensor_task
name          = "sensor"
stack size    = 512
priority      = 3
```

Задача FreeRTOS часто содержит бесконечный цикл:

```
for (;;) {
    read_sensor();
    xQueueSend(...);
    vTaskDelay(...);
}
```

В реверсе это помогает отделить «главные циклы задач» от маленьких helper- функций. Поведение устройства складывается из нескольких таких циклов плюс interrupt handlers.

Глава 52. Практикум: распаковка embedded rootfs

Есть файл `update.bin`. Binwalk показывает:

```
0x00010000  uImage header
0x00200000  SquashFS filesystem
```

Это не конец анализа, а начало. Извлекаем rootfs и смотрим:

```
/etc/init.d/
/etc/config/
/bin/
/sbin/
/usr/bin/
/www/
```

В init scripts видно, какие сервисы стартуют. В web files видно API роутера. В binaries видны ELF-файлы под MIPS/ARM. В configs видны default passwords или network settings, если они есть. В kernel modules видна работа с железом.

Но аккуратно: найденный пароль в rootfs может быть hash, default placeholder или неиспользуемая строка. Нужна проверка, где он читается и как применяется.

Глава 53. Практикум: оформление вывода для ВКР

Плохой вывод:

```
Программа парсит PE и работает с unwind.
```

Хороший вывод:

```
Модуль `pe_reader` проверяет DOS signature `MZ`, читает `e_lfanew`,
проверяет PE signature `PE\0\0`, затем по Optional Header получает Data
Directory Exception Table. RVA таблицы переводится в file offset через
section table с проверкой границ. Далее каждая Runtime Function entry
сопоставляется с диапазоном функции, а `UnwindInfoAddress` используется для
чтения UNWIND_INFO из `.xdata`.
```

Почему второй лучше? Он показывает путь данных, термины, формат, проверки и связь с целью. Он не просто говорит «работает», он объясняет как.

Для embedded плохой вывод:

```
Прошивка использует UART.
```

Хороший вывод:

```
В firmware обнаружены обращения к MMIO-диапазону USART1, соответствующему
reference manual выбранного MCU. Перед настройкой USART включается clock
через RCC register. В коде присутствуют строки логов и ring buffer приема.
Обработчик прерывания USART читает data register и помещает байт в буфер.
```

Такой стиль можно защищать.

Часть XII. Частые тупики новичка и как из них выходить

Глава 54. «Я ничего не понимаю в Ghidra»

Обычно это значит не «я глупый», а «я загрузил файл без правильной рамки». Проверь:

- правильная архитектура;
- правильный endian;
- правильный base address;
- формат действительно распознан;
- entry point найден;
- секции/segments выглядят разумно;
- есть ли symbols;
- не упакован ли файл;
- не raw firmware ли это вместо executable.

Ghidra не может угадать физику мира, если ты не дал ей карту. Особенно для raw firmware. Если base address неправильный, xrefs на константы и функции будут ломаться. Если выбран ARM mode вместо Thumb, инструкции станут мусором.

Глава 55. «Декомпилятор показывает бред»

Декомпилятор строит псевдокод из предположений. Бред может означать:

- неправильно определена функция;
- в середине data, а не code;
- неверный calling convention;
- не размечены структуры;
- indirect jump не распознан как switch;
- код обфусцирован;
- оптимизации сделали форму непривычной;
- не заданы типы pointer-ов.

Лечение: вернуться к assembly, проверить границы функции, задать типы, создать структуры, разметить таблицы, посмотреть xrefs. Декомпилятор любит, когда ему дают правильные типы. Один восстановленный struct может превратить нечитаемый псевдокод в нормальную функцию.

Глава 56. «Я нашел адрес, но не знаю, что он значит»

Сначала определить систему координат:

- это file offset?
- RVA?
- VA?
- physical address?

- MMIO?
- stack offset?
- structure field offset?

Потом проверить диапазон. Адрес `0x140001000` похож на Windows x64 VA. Адрес `0x08001234` похож на Flash STM32. Адрес `0x20000020` похож на SRAM Cortex-M. Адрес `0x40021018` похож на MMIO STM32. Но «похож» не равно «доказано». Доказательство приходит из headers, memory map, reference manual и фактических обращений.

Глава 57. «Я боюсь официальной документации»

Бояться ее нормально. Она написана не как учебник, а как точный договор. Тебе не нужно читать все. Нужно научиться вытаскивать ответ.

Алгоритм:

1. Сформулировать один вопрос.
2. Найти раздел по оглавлению или поиску.
3. Выписать одно правило.
4. Найти, где это правило видно в бинарнике.
5. Записать ссылку на источник в заметки.

Через несколько недель документация перестает быть стеной. Она становится судьей в споре между твоей гипотезой и реальностью.

Глава 58. «Как понять, что я уже достаточно доказал?»

Достаточно - не значит абсолютно. В реверсе часто нет исходников и автора рядом. Достаточно значит: вывод опирается на несколько независимых признаков и честно ограничен.

Слабое доказательство: «имя похоже». Сильнее: имя + xrefs + аргументы + строки. Еще сильнее: статический анализ + динамический breakpoint. Для embedded: MMIO address + reference manual + bit mask + наблюдаемый эффект на UART/logic analyzer.

В отчете всегда оставляй раздел «ограничения». Например:

```
Не анализировалась криптографическая стойкость подписи.
Не проверялась работа на реальном устройстве.
Base address firmware определен по vector table и memory map, но vendor
linker script отсутствует.
```

Ограничение не ослабляет работу. Оно показывает зрелость.

Теоретическая редакция: что изменено

Эта версия перестраивает книгу вокруг фундаментальной теории. Практические мастерские, capstone-проекты, длинные экзаменационные блоки и рабочие тетради убраны из основного корпуса. Они полезны в отдельном практикуме, но здесь они мешали главной задаче: дать человеку широкую и глубокую базу, чтобы он понимал, что происходит под инструментом.

Основной принцип этой редакции: сначала теория слоев, потом термины, потом связи между Windows, Linux, embedded, mobile, managed runtime, kernel и hardware. Практическая проверка упоминается только как способ отличать факт от догадки, но книга больше не строится вокруг заданий.

Английские технические слова не удаляются, потому что официальная документация живет именно на них. Но каждое важное слово должно иметь русское объяснение: что это такое, зачем нужно, где существует, с чем связано и какую ошибку предотвращает.

Авторская опора на лучшие книги и документацию

Эта книга не копирует чужие главы, упражнения или формулировки. Она берет у сильных источников архитектуру мышления: где начинать, как углубляться, как соединять инструмент с низким уровнем, как строить лабораторию и как не продавать догадку как факт.

У хороших книг по реверсу есть общие черты. Они не бросают читателя сразу в море mnemonics; они дают карту. Они не делают из инструмента божество; они показывают, где инструмент ошибается. Они не учат только одной ОС; они учат переносить мышление. Они не прячут сложность, но вводят ее по одному слою. Именно этот подход используется здесь.

Eldad Eilam - Reversing: Secrets of Reverse Engineering

Что это за опора: классическая широкая школа reverse engineering: низкий уровень, Windows fundamentals, tools, file formats, malware, decompilation и program data.

Какой принцип берем: берем принцип широкого входа: реверс нельзя начинать с кнопок инструмента, сначала нужно объяснить ОС, память, функции, форматы и доказательность.

Как это применено здесь: в этой книге это превращается в ранние тома про байты, память, Windows, PE, функции и проверяемый стиль анализа.

Michael Sikorski, Andrew Honig - Practical Malware Analysis

Что это за опора: лабораторная модель: безопасная среда, basic static, basic dynamic, x86, IDA, Windows APIs, debugging, anti-analysis, packers, shellcode, C++ и 64-bit.

Какой принцип берем: берем не вредоносный фокус, а ритм обучения: сначала лаборатория, потом статическое наблюдение, потом динамическая проверка, потом отчет.

Как это применено здесь: в этой книге это становится практикумами, VM-дисциплиной, triage, debugging и разбором packing/anti-debug в защитной рамке.

Dennis Andriesse - Practical Binary Analysis

Что это за опора: современный взгляд на binary analysis: ELF/PE, loaders, instrumentation, data-flow, taint analysis, symbolic execution.

Какой принцип берем: берем идею инструментального мышления: реверсер должен понимать не только вывод Ghidra, но и путь данных, проверку и собственные маленькие инструменты.

Как это применено здесь: в этой книге это дает главы про ELF, loader, slicing, tracing, instrumentation и проверку гипотез.

Chris Eagle - The IDA Pro Book

Что это за опора: плотная школа интерактивного анализа: navigation, xrefs, data types, structures, graphs, scripting, loaders, processors, debugger.

Какой принцип берем: берем принцип: дизассемблер не оракул, а рабочая среда, где человек уточняет типы, имена, xrefs и структуру.

Как это применено здесь: в этой книге это переходит в восстановление структур, культуру переименования и работу с Ghidra/IDA/Rizin/Binary Ninja.

Chris Eagle, Kara Nance - The Ghidra Book

Что это за опора: современная Ghidra-школа: project workflow, disassembly, decompiler, data types, scripts, processors, loaders, collaboration.

Какой принцип берем: берем идею Ghidra как бесплатной профессиональной платформы и decompiler как помощника, а не истины.

Как это применено здесь: в этой книге Ghidra используется как учебная база, но каждый вывод проверяется байтами, ABI и документацией.

Windows Internals

Что это за опора: системная база Windows: процессы, threads, memory manager, loader, exceptions, handles, kernel/user boundary.

Какой принцип берем: берем runtime-контекст: PE-файл живет не в пустоте, его запускает Windows с правилами памяти, загрузки и исключений.

Как это применено здесь: в этой книге это особенно важно для ВКР: PE32+, Windows x64 ABI, exception directory, unwind metadata и debugger events.

Bryant и O'Hallaron - Computer Systems: A Programmer's Perspective

Что это за опора: сильная база о том, как программы выглядят с точки зрения машины: representation, machine code, linking, memory hierarchy, virtual memory.

Какой принцип берем: берем педагогическую честность: до malware и firmware нужно понять, как число становится адресом, а функция - машинным соглашением.

Как это применено здесь: в этой книге это фундамент про байты, C, assembly, stack, compiler, linker и loader.

The Hardware Hacking Handbook

Что это за опора: лабораторный подход к hardware и embedded security: интерфейсы, измерения, отладочные порты, firmware extraction, physical discipline.

Какой принцип берем: берем embedded-реальность: прошивка - это не только код, но и плата, UART, SWD/JTAG, Flash, питание и риск повредить устройство.

Как это применено здесь: в этой книге это дает Cortex-M, MMIO, UART, DMA, RTOS, OpenOCD, logic analyzer и hardware safety.

Rootkits и Bootkits

Что это за опора: низкоуровневая линия boot, firmware, UEFI, kernel и forensic thinking.

Какой принцип берем: берем идею анализа ниже обычного процесса: boot chain, early execution, persistence и trust boundaries.

Как это применено здесь: в этой книге это превращается в bootloader handoff, firmware containers, startup code и defensive analysis.

Официальные спецификации Microsoft, Intel, AMD, Arm, RISC-V, ELF gABI, DWARF, OpenOCD, QEMU

Что это за опора: первичные правила: ABI, instruction set, exception handling, binary formats, debug information и hardware interfaces.

Какой принцип берем: берем авторитет правил: если книга и инструмент расходятся со спецификацией, проверять нужно спецификацию и реальный артефакт.

Как это применено здесь: в этой книге каждый технический вывод привязан к правилу: format, ABI, ISA, memory map, debug standard или reference manual.

Как читать эту книгу после такой опоры

Думай о ней как о мосте. От Eilam берется широкий вход в reverse engineering, от Practical Malware Analysis - лабораторная дисциплина, от Practical Binary Analysis - современное понимание binary analysis как data-flow и instrumentation, от IDA/Ghidra-книг - интерактивная работа с xrefs/types/names, от Windows Internals - реальная жизнь процесса, от CSAPP - машинная база, от hardware-hacking книг - embedded и физический слой.

Главное отличие этой книги: она написана на русском, для читателя, который может прийти вообще не из IT, но хочет в итоге понимать и Windows ВКР, и будущий embedded reverse. Поэтому каждое английское слово должно быть не маркером элитарности, а рабочей ручкой, за которую можно взять понятие.

Часть I. Большая теоретическая база

Эта часть читается как обычная книга, а не как список заданий. Главы идут от самых нижних понятий к системам и платформам.

Теоретическая глава 1. Что такое реверс-инжиниринг как наука о слоях

Реверс-инжиниринг часто ошибочно представляют как набор инструментов: открыл файл в Ghidra, увидел псевдокод, переименовал функции. Это слабое представление. Настоящий фундамент начинается раньше: с идеи, что любой бинарный объект состоит из слоев. Есть физический слой хранения: биты на диске, во Flash, в памяти или в сетевом пакете. Есть слой представления: числа, строки, таблицы, адреса, инструкции. Есть слой соглашений: формат файла, ABI, ISA, протокол, memory map. Есть слой исполнения: CPU, loader, runtime, операционная система или микроконтроллер. Есть слой смысла: зачем программа это делает.

Новичок тонет, когда пытается сразу прыгнуть на слой смысла. Он видит строку `error`, рядом условный переход и пишет: "это проверка". Но строка еще не доказывает поведение. Нужно спуститься по слоям: где строка лежит, кто на нее ссылается, какая функция получает управление, какие данные сравниваются, что происходит при каждой ветке, какой внешний эффект появляется. Только после этого смысл становится выводом, а не впечатлением.

Поэтому фундаментальный реверс - это не "угадай назначение функции". Это дисциплина переходов между слоями. Когда ты читаешь PE, ты переходишь от байтов к полям заголовка, от полей к правилам loader-а, от loader-а к карте памяти процесса, от карты памяти к выполнению инструкций. Когда читаешь firmware, ты переходишь от raw bytes к base address, от base address к vector table, от vector table к reset handler, от reset handler к инициализации памяти и периферии. Везде одна и та же мыслительная форма: слой, правило, следствие.

В этой книге теория важнее кнопок. Инструмент может помочь увидеть структуру, но не может заменить понимание. Если инструмент ошибся в base address, вся карта firmware поедет. Если decompiler неверно угадал тип, псевдокод станет красивым, но ложным. Если debugger показывает регистр, но ты не знаешь ABI, значение в регистре остается числом без роли. Теория дает роль.

Теоретическая глава 2. Информация: бит, байт, число, знак и порядок байтов

Бит - это минимальное различение: ноль или единица. Сам по себе бит почти ничего не говорит, но цепочка битов превращается в число, символ, адрес, флаг, инструкцию или кусок сжатых данных. Байт обычно состоит из восьми битов и дает 256 возможных значений. В реверсе байт удобнее видеть в hex, потому что две шестнадцатеричные цифры ровно описывают один байт. `FF` не страшнее числа 255; это просто форма записи, которая близка к машинному представлению.

Важно понять: байты не имеют единственного смысла. Последовательность `4D 5A` может быть строкой `MZ`, может быть 16-битным числом, может быть частью другой структуры. Смысл появляется только после выбора правила чтения. Это правило может прийти из формата файла, ABI, архитектуры процессора, reference manual микроконтроллера или из восстановленной структуры данных.

Endianness - порядок байтов в многобайтовом числе. В little-endian младший байт записывается первым, поэтому байты `00 40 00 20` читаются как `0x20004000`. Для человека это сначала выглядит перевернуто, но для CPU это обычная договоренность. Ошибка с endian меняет адреса,

размеры, длины пакетов, идентификаторы команд и значения MMIO-регистров. Поэтому хороший реверсер не "видит число", а сначала спрашивает: каким правилом это число прочитано?

Signedness - вопрос о том, считать ли число знаковым. Один и тот же байт `FF` может означать 255 как unsigned или -1 как signed 8-bit. В машинном коде это проявляется через инструкции расширения знака, тип сравнения, условные переходы и арифметику. Ошибка со знаком может превратить проверку границ в уязвимость, а в реверсе - в неверную реконструкцию условия. Поэтому термин "число" всегда неполон: нужно знать размер, endian, signedness и контекст использования.

Теоретическая глава 3. Модель CPU: состояние, инструкция и переход времени

CPU можно представить как машину состояния. В каждый момент у нее есть набор регистров, счетчик команд, флаги, доступная память и правила выполнения инструкций. Инструкция берет текущее состояние, читает операнды, выполняет действие и создает новое состояние. Это звучит сухо, но именно эта модель спасает от мистики assembly.

Fetch-decode-execute - учебная схема, но она полезна. CPU берет байты инструкции из памяти, декодирует их как команду конкретной ISA, читает регистры или память, выполняет операцию и решает, где будет следующая инструкция. В реальном процессоре есть pipeline, prediction, cache, out-of-order execution и микрокод, но реверсер обычно начинает с архитектурно видимого состояния: какие регистры изменились, какая память прочитана, какой переход выбран.

Регистры - не просто быстрые переменные. Их роль задается архитектурой и ABI. Один и тот же `rax` в x86-64 может быть временным регистром, возвращаемым значением функции, частью умножения или системного вызова. `rsp` почти всегда связан со стеком, но даже стек нужно читать через соглашение: где return address, где shadow space, где сохраненные регистры, где локальные данные.

Флаги - маленькие признаки результата операции. После `cmp` CPU не хранит "результат сравнения" как человеческую фразу. Он выставляет флаги: zero, carry, sign, overflow и другие. Условный переход читает эти флаги. Поэтому понимать ветвление - значит понимать, какие флаги выставила предыдущая инструкция и какой смысл у signed/unsigned перехода.

Для embedded особенно важны исключения и interrupts. CPU может временно оставить текущий поток инструкций и перейти в handler. Это не обычный `call`: аппаратно сохраняется часть состояния, выбирается адрес обработчика, меняется режим или контекст. Поэтому firmware нельзя читать только линейно от `main`; часть поведения живет в обработчиках событий.

Теоретическая глава 4. Память: адресные пространства, страницы, стек и куча

Память в реверсе нельзя понимать как одну длинную полку. Есть файл на диске, виртуальная память процесса, физическая память, память микроконтроллера, регистры периферии, memory-mapped файлы и кэши. Все они выглядят как адреса и байты, но правила у них разные.

В обычной ОС программа работает с virtual address. Процессор и ОС через MMU переводят виртуальные страницы в физические. Это дает изоляцию процессов, защиту страниц, lazy loading, shared libraries, ASLR и copy-on-write. Когда debugger показывает адрес `0x7ff...`, это не "номер микросхемы памяти", а координата внутри виртуального города конкретного процесса.

Стек - область, тесно связанная с вызовами функций. При `call` обычно сохраняется return address, затем функция может выделить место под локальные данные и сохранить регистры. Но форма стека зависит от ABI, оптимизаций и исключений. На Windows x64 есть shadow space, на System V AMD64 другие правила аргументов, на Cortex-M interrupt создает аппаратный exception frame. Поэтому фраза "посмотрим стек" без ABI мало что значит.

Heap - динамическая память. Она важна для структур, строк, объектов, буферов и жизненного цикла данных. В реверсе heap часто виден через allocator calls, custom allocators, pointer graphs, use-after-free patterns и сериализованные структуры. Но heap не имеет такой простой формы, как учебный массив: allocator добавляет metadata, выравнивание, корзины размеров и защитные проверки.

В embedded memory map может быть еще жестче: Flash по одному диапазону, SRAM по другому, периферия по третьему. Адрес `0x40021018` на STM32 похож на memory access, но фактически может быть регистром RCC. Запись туда меняет железо, а не обычную ячейку RAM. Поэтому embedded-реверс требует постоянно спрашивать: этот адрес - память, Flash, boot ROM или MMIO?

Теоретическая глава 5. Функции и ABI: договор, без которого регистры молчат

Функция в бинарнике - не просто красивый прямоугольник в graph view. Это участок кода, который соблюдает договор: откуда берет аргументы, где оставляет результат, какие регистры имеет право портить, какие обязан восстановить, как использует стек и как возвращает управление. Этот договор называется ABI, если говорить шире, и calling convention, если говорить именно о вызовах.

На Windows x64 первые четыре целочисленных аргумента обычно идут через `rcx`, `rdx`, `r8`, `r9`, а вызывающий код резервирует shadow space. На System V AMD64 используются `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`. Если перепутать эти два мира, функция будет прочитана неверно: то, что было первым аргументом в Linux, в Windows может быть совсем другой ролью.

Prologue и epilogue - не ритуальные куски кода, а механика подготовки и восстановления состояния. Prologue может уменьшить stack pointer, сохранить nonvolatile-регистры, установить frame pointer. Epilogue возвращает состояние. Но оптимизатор может убрать frame pointer, превратить функцию в tail call, встроить функцию внутрь другой или разнести сохранение регистров необычным образом. Поэтому границы функции - гипотеза, которую инструмент строит по признакам.

Unwind - отдельная теория восстановления стека. На Windows x64 `.pdata/.xdata` описывают, как разматывать стек без надежной цепочки `rbp`. В DWARF CFI похожая идея выражена через правила CFA и сохранения регистров. Для реверса это критично: stack trace не появляется из воздуха. Его строят правила, metadata, symbols и фактическое состояние памяти.

Для C++ и объектных языков добавляются `this`, vtable, constructors, destructors, RTTI, virtual calls. Все это тоже договоры. Virtual call - не "магия ООП", а загрузка указателя из объекта, затем загрузка адреса функции из таблицы и непрямой вызов. Когда видишь такую цепочку в assembly, ты видишь машинную форму позднего связывания.

Теоретическая глава 6. Компилятор: почему бинарник не обязан быть похожим на исходник

Компилятор - не переводчик слово в слово. Он строит представление программы, проверяет правила языка, преобразует код, оптимизирует его, выбирает инструкции, распределяет регистры и подчиняется ABI. Поэтому бинарник - это не фотография исходника, а результат множества законных преобразований.

Front-end языка превращает исходный текст в синтаксические и семантические структуры. Затем часто появляется intermediate representation, более удобное для анализа и оптимизаций. На этом уровне compiler может удалить мертвый код, свернуть константы, встроить маленькие функции, заменить цикл, переупорядочить вычисления, объединить переменные или доказать, что ветка невозможна.

Undefined behavior в C и C++ особенно важен. Если программа нарушает правила языка, compiler может делать предположения, которые выглядят странно на уровне машинного кода. Реверсер, не знающий этого, иногда думает, что видит хитрую защиту, хотя перед ним следствие оптимизации.

Register allocation превращает логические переменные в физические регистры и stack slots. Одна исходная переменная может жить в разных регистрах в разные моменты, а один register может последовательно хранить разные значения. Поэтому имена вроде `local_20` или `uVar3` в decompiler output - временные догадки, а не настоящие имена.

Linker соединяет object files, разрешает symbols, применяет relocations и раскладывает секции. В embedded linker script может напрямую задавать, что попадет во Flash, что в SRAM, где будет vector table и где начнется stack. Поэтому понять binary без toolchain невозможно: compiler и linker оставляют следы не хуже автора программы.

Теоретическая глава 7. Форматы бинарных файлов: договор между producer и loader

PE, ELF, Mach-O, DEX, class file и firmware container - это договоры. Producer создает файл по правилам, consumer читает его по правилам. Producer может быть compiler, linker, packer, build system или vendor update tool. Consumer может быть OS loader, dynamic linker, runtime, bootloader, verifier, debugger или updater.

В PE важны DOS header, NT headers, section table, data directories, imports, exports, relocations, TLS, resources и exception directory. Но все эти поля нужно читать через вопрос: кто их использует? Loader строит image, применяет relocations, подготавливает imports, задает protection страниц и передает управление entry point. Debugger и exception machinery используют дополнительные metadata.

В ELF важнейшее разделение - sections и segments. Sections удобны для linker-а и анализа, но загрузка процесса в память идет через program headers. Dynamic section, relocations, GOT, PLT и interpreter связывают файл с dynamic linker. Если читать ELF только через section names, можно получить красивую, но неполную картину.

Mach-O строится вокруг header и load commands. Там важны segments, dylib dependencies, binding/rebasing information, dyld, code signature и platform policy. На iOS binary не живет отдельно от bundle, sandbox, entitlements и подписи. Это хороший пример того, как формат файла связан не только с кодом, но и с политикой платформы.

Raw firmware часто вообще не имеет самодокументирующего заголовка. Тогда формат приходится восстанавливать из признаков: vector table, strings, таблицы указателей, повторяющиеся структуры, сжатые блоки, checksums, signatures, partition layout. Здесь теория форматов превращается в теорию поиска границ и правил чтения.

Теоретическая глава 8. Loader, runtime и операционная система

Loader - механизм, который превращает файл в исполняемую реальность. Он не просто копирует bytes. Он создает отображения памяти, выбирает base address, применяет relocations, разрешает imports, подготавливает TLS, выставляет permissions и передает управление стартовому коду. Runtime затем готовит язык и библиотечную среду: инициализирует CRT, глобальные объекты, heap, обработку исключений, иногда GC или JIT.

На Windows важно различать слои Win32 API, Native API, ntdll, kernel boundary, handles, objects, threads, SEH/VEH и loader lock. Import table может показать намерения, но не доказывает, что функция реально вызывается. Delay imports, GetProcAddress и indirect calls меняют картину.

На Linux процесс создается через kernel, ELF loader и часто dynamic linker `ld.so`. `/proc/<pid>/maps` показывает живую карту процесса, syscalls задают границу user/kernel, signals создают асинхронные входы, а shared objects и relocations связывают адреса с реальными библиотеками.

В managed runtime loader другой природы. CLR или JVM читает metadata, проверяет типы, загружает assemblies/classes, компилирует или интерпретирует bytecode, управляет исключениями и памятью. Поэтому managed reverse нельзя свести к native disassembly: главный смысл часто лежит в metadata и bytecode.

В bare-metal firmware классического OS loader-а может не быть. Reset handler и startup code выполняют роль подготовки мира: копируют `.data` из Flash в RAM, обнуляют `.bss`, настраивают clock tree, иногда включают FPU и вызывают constructors. Это тоже loader-like работа, но она живет в самом firmware.

Теоретическая глава 9. Данные и типы: структуры, указатели, массивы и объекты

Реверс часто начинается с кода, но заканчивается данными. Программа принимает решения не абстрактно: она читает поля структур, сравнивает длины, индексирует массивы, разыменовывает указатели, выбирает handler из таблицы, интерпретирует packet или command descriptor.

Struct layout - это теория смещений, размеров и выравнивания. Поле `[rcx+8]` не становится `field_8` просто потому, что decompiler так написал. Нужно собрать все обращения: где читается byte, где dword, где pointer, где signed compare, где write. Только после этого можно предположить тип поля. Padding тоже важен: compiler может вставлять пустые байты для alignment, а protocol format может быть packed.

Pointer - не любое похожее число. Candidate pointer должен попадать в допустимый диапазон, разыменовываться как адрес, иметь связь с relocation или xrefs, соответствовать alignment и не противоречить memory map. В firmware candidate pointer особенно зависит от base address: если base выбран неверно, все указатели станут мусором.

Arrays и tables часто выдают архитектуру программы. Jump table показывает switch или dispatcher. Function pointer table показывает callbacks. Vector table в Cortex-M показывает exception handlers. Vtable показывает объектную модель. Import/export table показывает внешнюю границу. Уметь

видеть таблицу - значит видеть не строку кода, а форму организации поведения.

Types в decompiler-е - инструмент мышления. Когда ты задаешь правильную структуру, псевдокод становится яснее. Но тип - это вывод, а не подарок. Его нужно обосновать обращениями, размерами, control flow и контекстом.

Теоретическая глава 10. Control flow, data flow и смысл программы

Control flow отвечает на вопрос, куда может пойти выполнение. Это ветки, циклы, calls, returns, indirect jumps, exceptions, interrupts, callbacks и scheduler decisions. Data flow отвечает на вопрос, откуда берется значение и куда оно влияет. Настоящее понимание появляется, когда control flow и data flow соединяются.

Basic block - участок кода без внутренних развилок: вошли сверху, вышли одним из концов. Control-flow graph соединяет basic blocks переходами. Но graph view не равен пониманию. Нужно знать, какие условия управляют переходами, какие значения доходят до условий и какие side effects происходят на каждой ветке.

Data flow особенно важен для parser-ов, crypto, protocol reverse и vulnerability research. Если поле `length` читается из файла, потом расширяется со знаком, потом используется в allocation или copy, нужно провести цепочку. Ошибка в одном звене меняет вывод о безопасности и поведении.

SSA, slicing, taint analysis и symbolic execution - это формальные способы думать о зависимостях. SSA дает каждой версии значения отдельное имя. Slicing оставляет только инструкции, влияющие на интересующий результат. Taint analysis отмечает данные из источника и смотрит, куда они текут. Symbolic execution заменяет конкретные значения символами и исследует условия путей. Даже если ты не используешь эти инструменты каждый день, теория помогает задавать правильные вопросы.

В embedded control flow часто асинхронный: interrupt handler меняет buffer, DMA пишет данные без участия CPU, RTOS scheduler переключает task, callback вызывается драйвером. Поэтому линейное чтение `main` почти всегда неполно.

Теоретическая глава 11. Windows-фундамент для твоей ВКР: PE, x64 ABI и unwind

Windows-часть твоей ВКР не должна быть набором слов `.pdata`, `.xdata` и `UNWIND_INFO`. Ее фундамент таков: PE-файл содержит структурный договор с loader-ом; Windows x64 ABI задает договор вызовов; exception handling и unwind metadata задают договор восстановления стека. Проект EH/Unwind Analyzer находится ровно на пересечении этих договоров.

PE32+ image имеет ImageBase, sections, data directories и exception directory. `.pdata` содержит RUNTIME_FUNCTION entries: диапазоны функций и ссылки на unwind info. `.xdata` хранит UNWIND_INFO: сведения о prologue, stack allocation, сохраненных регистрах, chained info и handler data. Эти структуры нужны не для красоты: Windows должна уметь размотать стек при exception, stack walking и debugging.

Windows x64 ABI объясняет, почему unwind metadata выглядит именно так. Если функция меняет nonvolatile registers или выделяет stack space, это состояние нужно восстановить при unwinding. Если compiler убрал frame pointer, metadata становится главным источником правды о frame. Поэтому анализ unwind без ABI невозможен.

RtlVirtualUnwind - концептуально не "черная магия Windows", а алгоритм, который берет control PC, function table entry, context record и применяет unwind codes, чтобы восстановить caller context. Твой проект статически извлекает и оценивает признаки вокруг этих структур, поэтому теоретическая база ВКР - PE parsing, RVA mapping, x64 ABI, exception directory, prologue semantics и доказательная отчетность.

Важная граница: наличие unwind metadata не означает, что функция безопасна или вредоносна. Это структурный факт. Эвристика может оценить необычность, несовпадение, повреждение, плотность или подозрительные сочетания, но вывод должен оставаться связанным с признаками.

Теоретическая глава 12. Linux и ELF: другой мир, те же фундаментальные роли

Linux/ELF полезен не только сам по себе, но и как противоядие от Windows-центричности. Когда видишь второй мир, лучше понимаешь первый. В PE есть sections и data directories, в ELF есть program headers, section headers, dynamic section, GOT, PLT и relocations. Роли похожи, но правила разные.

ELF header задает тип файла, architecture, entry point и offsets к таблицам. Program headers говорят loader-у, какие segments отображать в память, с какими permissions, какого размера на диске и в памяти. Разница между file size и memory size объясняет zero-fill вроде `.bss`. Section headers удобны для linker-а и инструментов, но в stripped binary могут быть менее важны для исполнения.

Dynamic linker разрешает shared libraries и relocations. GOT хранит адреса, которые могут быть заполнены во время загрузки или при lazy binding. PLT дает stub-ы для внешних вызовов. Поэтому `puts@plt` - не сама функция `puts`, а механизм перехода к ней.

Syscall - граница между user-mode и kernel. В Linux это видно более прямо, чем в Windows userland API: `openat`, `read`, `write`, `mmap`, `ioctl`, `clone` и другие вызовы дают карту внешних эффектов. Но libc wrappers, vDSO, seccomp и namespaces усложняют картину.

DWARF добавляет debug info и call frame information. Но debug info - не исходник, а набор описаний: types, line info, locations, CFI. Он может быть отсутствующим, неполным или не соответствовать оптимизациям. Теория DWARF учит не верить именам слепо, а читать, что именно описывает metadata.

Теоретическая глава 13. Embedded-фундамент: firmware, memory map, MMIO, interrupts, DMA

Embedded reverse начинается с другой картины мира. В desktop ОС у тебя есть процесс, virtual memory, loader, threads, APIs. В bare-metal firmware часто есть микроконтроллер, Flash, SRAM, vector table, reset handler, периферия, interrupts и reference manual. Это не проще и не сложнее; это другой слой реальности.

Raw firmware не обязан иметь заголовок. Первые слова Cortex-M firmware часто дают initial stack pointer и reset handler. Нечетный адрес reset handler не ошибка: младший bit показывает Thumb state. Но этот вывод работает только при правильной architecture и base address. Если base неверный, дизассемблер построит ложные xrefs и функции.

MMIO - центральная идея embedded. CPU выполняет load/store по адресу, но за адресом может стоять регистр периферии. Bit field может включить clock, настроить GPIO mode, запустить UART, очистить interrupt flag, включить DMA channel. Поэтому запись в `*(volatile uint32_t *)addr` - не просто запись числа, а событие в железе.

Interrupts ломают линейное чтение. UART receive handler может наполнять ring buffer, timer handler может менять state machine, DMA complete interrupt может запускать обработку пакета. RTOS добавляет tasks, queues, semaphores, mutexes, priorities и context switch. Если искать всю логику в `main`, embedded-программа будет казаться пустой.

Embedded Linux соединяет оба мира: внутри firmware-образа может быть bootloader, kernel, device tree, root filesystem, init scripts, services, web interface и native binaries. Здесь reverse становится системным: нужно понимать не только один binary, но и устройство как набор слоев.

Теоретическая глава 14. Managed и mobile: байткод, metadata, runtime и платформа

Не весь реверс начинается с native instructions. В .NET, JVM и Android главный смысл часто живет в bytecode, metadata, manifest, class descriptors, resources, permissions и runtime behavior. Native code все еще важен, особенно через JNI или JIT, но читать только assembly - значит пропустить верхний договор.

.NET assembly содержит metadata tables, IL, manifest, references, attributes и resources. CLR загружает assembly, проверяет типы, компилирует IL в native code, управляет exceptions и memory через GC. Reflection может перенести часть поведения из явных calls в строки и metadata. Obfuscator может испортить имена, но metadata-структура все равно остается частью истины.

JVM class file хранит constant pool, methods, descriptors, attributes и bytecode для stack-based virtual machine. JVM verifier и class loader задают ограничения, JIT превращает hot paths в native code. Поэтому Java reverse - это не "легкий C"; это чтение виртуальной машины и ее модели типов.

Android APK соединяет ZIP-пакет, manifest, resources, DEX, ART, permissions, Binder IPC, SELinux policy и native libraries. Метод Java может вызвать JNI, native `.so` может делать syscalls, компонент может общаться с system service через Binder. Android требует держать в голове managed, native и OS-policy одновременно.

Apple-мир добавляет Mach-O, dyld, Objective-C runtime, Swift metadata, entitlements, sandbox, code signing и dyld shared cache. iOS binary нельзя честно понять только как "ARM64-код": он живет внутри подписанной платформенной модели.

Теоретическая глава 15. Kernel, drivers и UEFI: код ниже обычного процесса

Kernel-mode и firmware-уровень важны для широкой базы, даже если ВКР сейчас user-mode. Здесь меняется главный вопрос: кто вызывает код? В обычной программе мы ищем entry point и поток выполнения. В driver или UEFI module нужно искать registration, callbacks, dispatch tables, request structures и privilege boundaries.

UEFI boot chain состоит из фаз и сервисов. PEI, DXE, BDS, boot services, runtime services, ACPI и Secure Boot задают контекст. UEFI image может быть PE-похожим, но исполняться до обычной ОС, с другими правилами памяти и доверия. Поэтому знакомый формат файла не означает знакомую среду исполнения.

Windows driver может иметь DriverEntry, device objects, dispatch routines, IRP handling, IOCTL interface, unload routine. IOCTL особенно интересен как граница между user-mode и kernel-mode: есть код операции, input/output buffers, access rules и method of buffering. Анализировать driver как обычный EXE неправильно.

Linux kernel module имеет init/exit, file operations, sysfs/debugfs entries, ioctl handlers, netfilter hooks, platform driver callbacks, eBPF-related boundaries. Kernel code работает с другим уровнем доверия, memory allocation, locking, interrupt context и user copy rules.

Теоретическая ценность этого слоя в том, что он учит не искать `main` там, где его нет. Поведение появляется через регистрацию ролей и обратные вызовы. Это знание потом помогает и в user-mode reverse: callbacks, event loops, virtual dispatch и handlers становятся понятнее.

Теоретическая глава 16. Обфускация, упаковка и автоматизированный анализ как теория препятствий

Обфускация - не магия, а набор способов разрушить прямую связь между байтами и смыслом. Packer может хранить payload в сжатом или зашифрованном виде и создавать исполняемый код во время runtime. Control-flow flattening превращает понятные ветки в dispatcher. Opaque predicate создает условие, результат которого известен автору, но мешает анализу. String encryption убирает очевидные строки. VM-based obfuscation исполняет собственный bytecode внутри виртуальной машины.

Anti-debug и anti-analysis проверяют среду наблюдения: debugger flags, timing, exceptions, process names, breakpoints, hooks, unusual hardware state. Теоретически это важно не как список трюков, а как принцип: наблюдение может менять поведение объекта. Это общая научная проблема, просто в реверсе она имеет машинную форму.

YARA, capa, Frida, fuzzing, symbolic execution и taint analysis - не замена мышлению, а разные способы получать сигналы. YARA описывает признаки. capa сопоставляет возможности. Frida наблюдает runtime на выбранных точках. Fuzzing исследует пространство входов. Symbolic execution рассуждает о путях при заданной модели. Taint analysis отслеживает влияние данных.

Фундаментальная граница: инструментальный сигнал не равен объяснению. Match правила, hook event, crash, path constraint или taint flow должны быть вписаны в теорию объекта: какой слой наблюдается, какие правила действуют, что доказано, что не доказано. Поэтому автоматизация - часть теории доказательств, а не просто набор утилит.

Теоретическая глава 17. Аппаратный слой: сигналы, шины, питание и физическая проверяемость

Embedded reverse в будущем почти неизбежно выводит к железу. На плате есть питание, reset, clock, Flash, RAM, микроконтроллер, шины, debug ports, датчики, radio modules, secure elements. Бинарник в таком мире - только одна часть системы. Поведение может зависеть от напряжения, timing, конфигурации pins, fuse bits и внешних устройств.

UART, SPI, I2C, CAN и другие интерфейсы - это не просто провода. У них есть электрический уровень, framing, timing, roles, error handling и протокольные соглашения. Logic analyzer помогает увидеть цифровую последовательность, oscilloscope - форму сигнала, но интерпретация требует знания протокола и setup-а.

JTAG и SWD дают debug-доступ, но не всегда включены. OpenOCD связывает adapter, target, reset strategy, flash algorithm и debug commands. Chip-off и SPI Flash dump дают bytes, но без понимания partition layout, compression, encryption и signatures эти bytes еще не становятся firmware model.

Side-channel и fault injection требуют особенно строгого мышления. Side-channel наблюдает косвенные утечки: время, питание, электромагнитные признаки. Fault injection пытается нарушить вычисление: glitch clock, voltage, laser или другой физический фактор. Один красивый trace ничего

не доказывает без повторяемости, setup-а, контроля переменных и честной границы вывода.

Теория аппаратного слоя нужна даже тем, кто начинает с Windows. Она учит уважать среду. Программа не висит в пустоте: всегда есть машина, правила исполнения и способ наблюдения.

Часть II. Живой фундамент реверса

Эта часть написана как большая объяснительная книга, а не как конспект. Здесь почти нет заданий. Главная цель - связать в одну картину то, что обычно разорвано по разным книгам: устройство машины, память, CPU, ABI, компиляцию, загрузку, форматы, операционные системы, embedded, mobile, managed runtime, kernel, firmware и hardware.

Глава 1. Машина не понимает программу как человек

Человек смотрит на программу и думает словами: окно, файл, сеть, проверка, ошибка, команда, пользователь. Машина так не думает. Для нее есть состояние и правила перехода в следующее состояние. В одном месте лежат байты, в другом находятся регистры, где-то есть память, где-то стоит счетчик команд, где-то выставлены флаги. Инструкция не "понимает намерение автора", она просто берет текущее состояние и меняет его по правилам архитектуры.

Reverse engineering начинается в тот момент, когда мы принимаем эту разницу всерьез. Мы не пытаемся сразу угадать человеческий смысл. Мы сначала спрашиваем: что именно изменила машина? Какие байты прочитаны? Какой register изменился? Какой address использован? Какая branch выбрана? Какой handler получил управление? Какой file format заставил loader сделать именно такую memory map?

Это похоже на чтение незнакомого языка, но с одной важной разницей: грамматика машины описана документами. ISA говорит, что делают инструкции. ABI говорит, как функции передают аргументы и сохраняют регистры. PE, ELF и Mach-O говорят, как файл превращается в образ памяти. Reference manual микроконтроллера говорит, какой bit в каком register включает периферию. Поэтому реверс - не гадание, а медленное восстановление правил.

Если человек без IT-базы открывает дизассемблер, он видит лес: `mov`, `call`, `jmp`, `rsp`, `RVA`, `GOT`, `MMIO`, `handler`. Но этот лес можно превратить в карту. `mov` - это перенос данных между местами. `call` - передача управления с обещанием вернуться. `rsp` - указатель на вершину стека. `RVA` - адрес относительно image base в PE. `GOT` - таблица адресов в ELF, которую помогает заполнить dynamic linker. `MMIO` - доступ к регистрам устройства через адресное пространство. `handler` - функция, которую вызывает не ты напрямую, а событие, runtime, OS или hardware.

Главное фундаментальное движение такое: мы переводим непонятное слово в роль, роль в механизм, механизм в наблюдаемое состояние. Пока слово не прошло этот путь, оно остается украшением. Когда прошло - становится инструментом мышления.

Глава 2. Байт как кирпич, но не как смысл

Байт часто называют "единицей данных", но для реверса этого мало. Байт

- это маленький контейнер возможностей. Он может быть частью числа,

символа, машинной инструкции, поля структуры, адреса, compressed stream, encrypted block, checksum, флага или просто выравнивания. Поэтому вопрос "что значит этот байт?" всегда неполон. Правильный вопрос: "какое правило сейчас читает этот байт?"

Возьмем два байта `4D 5A`. В Windows PE они выглядят как ASCII-строка `MZ`, потому что старый DOS header начинается именно с такой подписи. Но если читать эти же байты как little-endian 16-bit number, получится другое представление. Если сдвинуться на один байт, смысл снова изменится. Байт не лжет и не говорит правду. Он просто лежит. Смысл появляется только при наложении формата.

Hex - шестнадцатеричная запись - нужен не для того, чтобы выглядеть "хакерски". Он удобен, потому что один hex digit описывает четыре бита, а два hex digits описывают один байт. `FF` сразу видно как все восемь битов, выставленные в единицу. `00` - все нули. `80` - старший bit выставлен. Для flags, masks, addresses и binary formats такая запись естественнее десятичной.

Endianness надо разжевать особенно спокойно. Представь число как слово из нескольких букв. В big-endian старшая часть числа лежит первой, как привычная запись слева направо. В little-endian младшая часть лежит первой. Поэтому bytes `78 56 34 12` могут означать число `0x12345678`. Это не "перевернутость ради сложности", а договор архитектуры и формата. Если забыть endian, можно перепутать длину пакета, адрес handler-a, размер секции или значение регистра.

Signedness - еще одна ловушка. Машина часто хранит просто bits, а человек решает, читать ли их как signed или unsigned. Byte `FF` может быть 255 или -1. В assembly это проявляется через sign extension, zero extension и разные условные переходы. Когда parser проверяет length, ошибка signedness может стать уязвимостью. Когда реверсер читает сравнение, ошибка signedness становится неверным выводом.

Так фундамент начинается не с "выучи hex", а с более глубокой мысли: данные не имеют смысла без правил чтения. Реверсер всю жизнь ищет эти правила, проверяет их и уточняет.

Глава 3. Адреса: почему одно число может указывать на разные миры

Address по-русски обычно переводят как адрес, но в реверсе это слово слишком опасно без уточнения. File offset, virtual address, RVA, physical address, MMIO address, stack offset и field offset - все они похожи на "число места", но живут в разных картах.

File offset - смещение от начала файла. Hex editor мыслит именно так: пройти от начала файла столько-то байтов. Virtual address - адрес в виртуальном адресном пространстве процесса. Его видит работающая программа. RVA, relative virtual address, - адрес относительно image base; в PE это один из главных способов ссылаться на части image. Physical address - адрес в физической памяти, который обычно скрыт от user-mode процесса. MMIO address - адрес, за которым находится не RAM, а register устройства. Stack offset - смещение относительно stack pointer или frame pointer. Field offset - смещение поля внутри структуры.

Новичок часто берет число из PE header и прибавляет его к началу файла. Иногда это случайно работает, но чаще ломает анализ. RVA надо переводить через section table: найти секцию, в диапазон которой попадает RVA, вычислить смещение внутри секции и прибавить PointerToRawData. Это не формальность. Это разница между дисковым файлом и образом памяти.

В Linux похожая ловушка возникает между sections и segments. Section удобна инструменту и linker-у, а program header важен loader-у. В firmware raw dump может вообще не сказать, с какого base address его надо отображать. Тогда адреса из vector table и pointer tables помогут выбрать гипотезу, но пока base не обоснован, xrefs остаются шаткими.

Адрес - это не абсолютная истина, а координата внутри выбранной карты. Поэтому хороший отчет должен писать не просто "по адресу 0x1000", а "file offset 0x1000", "RVA 0x1000", "VA 0x140001000", "MMIO address 0x40021018" или "stack offset -0x20". Эта точность кажется мелкой, пока не спасет анализ от полностью неверного вывода.

Глава 4. CPU как театр маленьких причин

Процессор не исполняет C, C++, Rust или Python. Он исполняет инструкции своей architecture. Architecture, или ISA - instruction set architecture, задает видимые программисту правила: какие есть регистры, какие инструкции, как адресуется память, какие флаги меняются, какие исключения возможны. Реальный процессор внутри может быть гораздо сложнее, но реверсер начинает с этой видимой модели.

Instruction - инструкция - маленький приказ машине. `mov` переносит данные, `add` складывает, `cmp` сравнивает через выставление flags, `jmp` меняет поток управления, `call` сохраняет путь назад и прыгает в функцию, `ret` берет адрес возврата и продолжает выполнение. Даже эти слова надо объяснять через состояние. `call` не "вызывает функцию" в человеческом смысле. Он меняет instruction pointer и обычно кладет return address на stack. Человеческое понятие функции появляется выше.

Register - маленькая быстрая ячейка внутри CPU. Но register получает смысл только в контексте. `rax` после функции на Windows x64 часто содержит return value. Перед `call` `rcx` может быть первым аргументом Windows x64 calling convention. В System V AMD64 первым аргументом будет `rdi`. На ARM и RISC-V будут свои регистры и свои правила.

Flags - это память о результате операции. После сравнения CPU не хранит фразу "a меньше b". Он выставляет bits состояния. Условный переход читает эти bits. Поэтому `jl` и `jb` отличаются не буквой, а смыслом: один относится к signed comparison, другой к unsigned comparison.

В embedded CPU еще чаще встречается с exceptions и interrupts. Interrupt - событие, которое может прервать обычный поток и отправить CPU в handler. Handler - обработчик. Он не обязательно вызван обычным `call`; его может вызвать периферия, timer, DMA complete event или fault. Поэтому прошивка похожа не на одну дорогу, а на город с перекрестками событий.

Глава 5. Функция, стек и ABI как социальный договор машинного кода

ABI - application binary interface - один из самых важных терминов для реверсера. Его можно перевести как бинарный договор приложения. Он говорит отдельно собранным кускам кода, как жить вместе: где аргументы, где return value, кто сохраняет какие registers, как выровнен stack, как выглядят structures, как работают exceptions и linking.

Calling convention - часть ABI, отвечающая за вызовы функций. Если функция принимает пять аргументов, машина не видит "пять красивых переменных". Она видит registers и memory slots. На Windows x64 первые четыре integer/pointer arguments идут через `rcx`, `rdx`, `r8`, `r9`, а вызывающий код резервирует shadow space. Shadow space - 32 bytes на stack, которые caller обязан выделить для callee. На System V AMD64 первые аргументы идут через `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`. Это не детали для зануд. Это то, что определяет смысл регистров в disassembly.

Stack - не просто "место для переменных". Это память дороги назад. `call` оставляет return address, callee может сохранить registers, выделить locals, подготовить alignment. Prologue - начало функции, где она готовит frame. Epilogue - конец, где она возвращает состояние. Но compiler может убрать frame pointer, объединить stack slots, сделать tail call или inline. Поэтому красивый учебный stack frame - только начальная модель.

Unwind - разматывание стека - объясняет, как вернуться по цепочке вызовов, особенно когда обычная цепочка frame pointer отсутствует. В Windows x64 для этого есть `.pdata` и `.xdata`: таблицы, где описано, как функция меняла stack и сохраняла registers. В DWARF есть CFI - call

frame information. Debugger, exception dispatcher и stack walker не угадывают прошлое, они применяют правила. Твоя ВКР про EH/Unwind Analyzer как раз стоит на этой теории.

Глава 6. Компилятор не переводит, а преобразует

Compiler часто представляют как переводчик с C на assembly. Это слишком бедная картинка. Compiler строит модель программы и меняет ее, сохраняя допустимый смысл по правилам языка. Он может удалить лишнее, встроить функцию, переставить вычисления, заменить ветку арифметикой, свернуть константу, развернуть цикл, объединить переменные, выбрать registers и даже использовать undefined behavior как основание для смелых предположений.

Intermediate representation, или IR, нужен compiler-у как внутренний язык для анализа. Исходный код уже разобран, но машинные инструкции еще не выбраны. На этом уровне удобно делать data-flow analysis, optimization passes, inlining, constant propagation и dead code elimination. Реверсер не всегда видит IR напрямую, но видит следы: исчезнувшие функции, странные ветки, объединенные переменные.

Register allocation решает, какие значения будут жить в registers, а какие уйдут в stack. Поэтому одна исходная переменная может не иметь одного постоянного места. Decompiler вынужден придумывать временные имена: `uVar1`, `local_20`, `param_1`. Это не авторские имена, а попытка инструмента восстановить удобную форму.

Linker делает следующий большой шаг. Object files содержат sections, symbols и relocations. Symbol - имя, связанное с адресом или сущностью. Relocation - запись "когда станет известно реальное место, исправь вот это значение". Linker соединяет object files, раскладывает sections, разрешает symbols и готовит binary. В embedded linker script может сказать: `.text` во Flash, `.data` загрузить из Flash, но выполнить в RAM, stack положить в конец SRAM. Поэтому компиляция - это не один шаг, а целая история превращений.

Глава 7. Loader и runtime: кто оживляет файл

Исполняемый файл сам по себе не "работает". Его должен оживить loader. Loader читает формат, отображает sections или segments в memory, задает permissions, применяет relocations, разрешает imports, готовит TLS, иногда подключает dynamic libraries и передает управление entry point. Entry point - не обязательно `main`; часто это стартовый код runtime, который только потом вызовет пользовательскую логику.

Runtime - среда выполнения вокруг языка и библиотек. В C/C++ runtime готовит CRT, global constructors, exception handling, heap. В .NET CLR загружает metadata, проверяет types, JIT-компилирует IL, управляет GC. В JVM class loader и verifier подготавливают class files, а JIT может позже превратить bytecode в native code. В bare-metal firmware startup code копирует `.data`, обнуляет `.bss`, настраивает clock и вызывает `main`.

Поэтому reverse engineering всегда должен спрашивать: это логика автора или служебная обвязка? Новичок часто тратит силы на startup, allocator, exception wrappers, thunk functions, PLT stubs, import trampolines и runtime initialization. Это полезно распознать, но нельзя путать с предметным поведением программы.

Loader также объясняет, почему file view и memory view отличаются. File view - это файл на диске, как он лежит в байтах. Memory view - это образ после загрузки, где уже сработали правила loader-а. В PE file offset не равен RVA. В ELF sections не равны loaded segments. В Mach-O load commands управляют отображением. В Android APK вообще является package, где DEX, resources, manifest и native libraries вместе формируют приложение. Файл становится поведением только

через loader и runtime.

Глава 8. PE, ELF и Mach-O как разные грамматики исполняемости

PE - Portable Executable - формат Windows. Его нельзя понимать как "просто EXE". Это договор с Windows loader. DOS header с `MZ`, NT headers с `PE`, COFF header, Optional Header, section table, data directories, imports, exports, relocations, resources, TLS и exception directory - все это разные части договора. Optional Header на самом деле не optional для image; название историческое.

RVA в PE - relative virtual address, адрес относительно ImageBase. ImageBase - предпочитаемая база загрузки image. SectionAlignment и FileAlignment объясняют, почему расположение на диске и в памяти не одинаковое. Import Address Table помогает loader-у записать реальные адреса внешних функций. Base relocations позволяют image жить не по preferred base, что важно для ASLR.

ELF - Executable и Linkable Format - живет в Linux и других Unix-like системах. ELF header говорит базовые свойства. Program headers важны для загрузки: какие segments положить в memory. Section headers важны для linker-а и инструментов. Dynamic section, GOT, PLT и relocations создают связь с dynamic linker. GOT - global offset table, таблица адресов данных и функций. PLT - procedure linkage table, переходники для вызовов внешних функций.

Mach-O - формат Apple. Его грамматика построена вокруг load commands. dyld - dynamic loader Apple - использует эти команды, shared cache, binding, rebasing и code signing context. Objective-C runtime добавляет classes, selectors, method lists. Swift metadata добавляет свои descriptors. На iOS формат нельзя отделить от sandbox, entitlements и подписи. Так становится видно: file format всегда связан с культурой платформы.

Глава 9. Decompiler, disassembler и опасность красивого псевдокода

Disassembler переводит bytes в инструкции. Decompiler пытается поднять инструкции в похожий на C текст. Оба инструмента невероятно полезны, но оба строят модель. Function boundaries, types, variables, structures, switch recovery, calling convention, signedness - многое приходится угадывать. Угадывание может быть очень умным, но оно остается угадыванием, пока не подтверждено механизмом.

Decompiler красив тем, что возвращает человеку знакомую форму. Но эта красота опасна. Псевдокод может скрыть flags, side effects, volatile access, exact width, sign extension, undefined behavior, aliasing, indirect calls. Он может показать `if (x < y)`, но в assembly было unsigned comparison. Он может показать структуру, но поля еще не доказаны. Он может объединить переменные, которые в исходнике были разными, или разделить то, что было одним понятием.

Правильное отношение такое: decompiler - переводчик, а не свидетель. Он помогает читать, но свидетельство лежит ниже: instruction semantics, ABI, memory access, data flow, xrefs, runtime behavior, documentation. Когда вывод важен, возвращайся к нижнему слою. Не потому что decompiler плохой, а потому что фундамент должен стоять на правилах.

Data flow и control flow помогают не утонуть. Control flow говорит, куда может пойти выполнение. Data flow говорит, откуда пришло значение и куда оно влияет. Если строка ведет к функции, это только начало. Нужно понять, какое значение сравнивается, какая branch выбирается, какой side effect возникает, где находится external boundary.

Глава 10. Embedded как мир, где адрес может включить железо

Embedded reverse часто начинается с шока: нет привычного EXE, нет процесса, нет imports, нет красивого loader-а ОС. Есть firmware image, Flash, SRAM, vector table, reset handler, startup code, peripheral registers и reference manual. Но это не хаос. Это другая система правил.

Cortex-M firmware часто начинается с vector table. Первое word - initial stack pointer, второе - reset handler address. Если reset handler нечетный, это не ошибка, а Thumb bit. Reset handler запускает initialization: копирует `.data`, обнуляет `.bss`, вызывает SystemInit, настраивает clocks и переходит к main или scheduler. Уже здесь видно: startup code - не предметная логика, но без него программа не оживет.

MMIO - memory-mapped input/output - ключ к периферии. CPU думает, что читает или пишет memory address, но за адресом стоит hardware register. GPIO mode register меняет режим ножки. UART status register говорит, принят ли byte. DMA control register запускает перенос. RCC register включает clock. Поэтому `volatile` в C нужен, чтобы compiler не решил, будто это обычная память, которую можно кэшировать или удалить.

Interrupts и DMA делают поведение нелинейным. UART может принять byte и вызвать interrupt. DMA может записать buffer без выполнения инструкций CPU на каждый byte. Timer может менять state machine. RTOS может переключить task. Поэтому embedded reverse - это не чтение функции сверху вниз, а восстановление сети событий.

Глава 11. Managed runtime и mobile: когда смысл живет выше CPU

.NET, JVM, Android и Apple-платформы показывают, что native assembly - не всегда главный слой. Managed code - код под управлением runtime. В .NET это IL, metadata, assemblies, CLR, JIT и GC. IL - intermediate language, байткод .NET. Metadata описывает types, methods, references, attributes. JIT - just-in-time compiler, превращающий IL в native code во время выполнения. GC - garbage collector, управляющий памятью объектов.

JVM похожа идеей, но другая правилами. Class file содержит constant pool, methods, descriptors, attributes и bytecode для stack-based virtual machine. Operand stack JVM - не системный stack CPU, а абстрактный стек виртуальной машины. Verifier проверяет безопасность bytecode перед выполнением. JIT может превратить hot methods в native instructions. Поэтому Java reverse должен читать class file и runtime model, а не только native dump.

Android соединяет managed и native. APK - package, внутри которого manifest, resources, DEX, native libraries. DEX - Dalvik Executable, формат bytecode для Android runtime. ART - Android Runtime, современная среда выполнения. JNI - Java Native Interface, мост между Java/Kotlin кодом и native `.so`. Binder - IPC-механизм Android, через который процессы общаются с services. SELinux задает policy. Тут один feature может проходить через manifest, DEX method, JNI call, ELF library, syscall и Binder transaction.

Apple-мир тоже многоуровневый. Mach-O binary живет с dyld, code signing, entitlements, sandbox, Objective-C runtime, Swift metadata, dyld shared cache и иногда pointer authentication. Поэтому mobile reverse - это теория платформы, а не просто "открыть ARM64".

Глава 12. Kernel, drivers и UEFI: там, где нет обычного main

В user-mode программе мы привыкли искать entry point, затем main, затем вызовы. В kernel driver или UEFI module такая привычка ломается. Поведение часто начинается не с main, а с регистрации обработчиков. Код говорит системе: вот мои dispatch routines, вот мои callbacks, вот мой IOCTL handler, вот мои protocol interfaces. Потом система вызывает их при событиях.

Kernel mode - режим с повышенными правами. Ошибка там может повлиять на всю систему. Driver object, device object, IRP, IOCTL, dispatch table - это не украшения Windows-драйвера, а форма общения с kernel. IRP - I/O request packet, структура запроса ввода-вывода. IOCTL - control code, через который user-mode или другой компонент может попросить driver выполнить операцию. Чтобы читать driver, нужно понять не только instructions, но и модель запросов.

Linux kernel module имеет свои entry points: init, exit, file operations, sysfs/debugfs entries, ioctl, netfilter hooks, platform driver callbacks. eBPF добавляет загружаемые программы с verifier-ом и строгими ограничениями. Тут снова видно: reverse - это изучение договоров между слоями.

UEFI живет еще раньше обычной ОС. PEI, DXE, boot services, runtime services, ACPI, Secure Boot и firmware volumes создают boot chain. UEFI image может быть PE-like, но среда исполнения другая. Secure Boot добавляет цепочку доверия. ACPI tables описывают железо для ОС. В этом мире формат файла, firmware policy и hardware initialization связаны особенно тесно.

Глава 13. Обфускация и упаковка как намеренное разрушение карты

В обычном бинарнике мы надеемся, что структура хоть немного помогает: функции похожи на функции, строки лежат строками, imports показывают внешние зависимости, control flow читается графом. Obfuscation ломает эту надежду. Она не отменяет правила машины, но делает путь от bytes к смыслу длиннее.

Packer хранит полезный код в сжатом, зашифрованном или преобразованном виде, а маленький stub восстанавливает его во время runtime. Поэтому static view показывает не payload, а упаковочную оболочку. Высокая entropy может быть признаком packing или encryption, но сама по себе ничего не доказывает. Нужно понять, где memory становится executable, где появляются новые bytes, куда передается control.

Control-flow flattening превращает понятный graph в dispatcher и state variable. Opaque predicate добавляет условие, которое выглядит неизвестным, но фактически имеет предсказуемый результат. String encryption убирает obvious strings. VM-based obfuscation превращает исходную логику в bytecode собственной виртуальной машины. Anti-debug проверяет среду наблюдения. Все это разные способы увеличить расстояние между наблюдением и смыслом.

Но даже сильная обфускация не отменяет фундамент. CPU все равно исполняет инструкции, память все равно меняется, loader и runtime все равно имеют правила, данные все равно куда-то текут. Теория помогает не паниковать: если не читается высокий слой, возвращаемся ниже, к состояниям, переходам, памяти и событиям.

Глава 14. Hardware security: когда программа становится сигналом

На аппаратном уровне программа перестает быть только файлом. Она становится поведением устройства: потреблением питания, сигналом на линии, обменом по шине, реакцией на reset, записью во Flash, ответом secure element. Поэтому hardware reverse требует другой честности. Нужно описывать setup, wiring, voltage, clock, sample rate, adapter, target, firmware version. Без этого наблюдение невозможно повторить.

UART, SPI, I2C, CAN - это протоколы связи. У каждого есть framing, timing, roles, errors. Logic analyzer показывает цифровую историю линий, но не знает смысл без protocol decoder и контекста. Oscilloscope показывает форму сигнала, но интерпретация требует понимания электричества. JTAG и SWD дают debug-доступ, если они включены и не заблокированы. OpenOCD связывает adapter и target через правила конкретного чипа.

Side-channel analysis смотрит не на прямой результат, а на косвенные признаки: время, питание, электромагнитное излучение. Fault injection пытается нарушить работу: voltage glitch, clock glitch, laser fault. Это не "магические атаки", а физическое вмешательство в вычисление. Но вывод здесь особенно хрупкий: одно удачное измерение без повторения и контроля не доказывает устойчивый механизм.

Hardware layer важен даже для будущего embedded reverse как культура: устройство реально. Его можно повредить. Его поведение зависит от среды. Его сигналы требуют измерения. Это дисциплина, которой нет в чистом desktop reverse.

Глава 15. Доказательность как теоретический принцип, а не практика

Доказательство в реверсе - это не оформление отчета в конце. Это способ думать с первой минуты. Каждый вывод должен знать свой слой. "Эта функция проверяет ключ" - слабая фраза, если не сказано, какие данные входят, какое сравнение происходит, какая branch зависит от результата, какой side effect наблюдается. "Это UART" - слабая фраза, если не показаны MMIO addresses, register layout, baud rate setup, interrupt или data register.

Факт - то, что наблюдается: bytes, instruction, register value, table entry, string, call, memory mapping, trace event. Правило - то, что объясняет, как читать факт: specification, ABI, ISA, reference manual, runtime documentation, protocol description. Вывод - связь факта и правила. Ограничение - честная граница: что не проверено, какой version неизвестен, какой dynamic path не покрыт, какой hardware setup отсутствует.

Это не бюрократия. Это защита от самообмана. Реверс-инженер постоянно видит неполные картины. Инструменты подсказывают, decompiler сглаживает, строки соблазняют, знакомые patterns заставляют торопиться. Теория доказательности возвращает спокойствие: сначала слой, потом правило, потом вывод.

Поэтому фундаментальная книга по реверсу должна учить не только байтам и регистрам. Она должна учить интеллектуальной честности. Без нее даже огромный набор терминов превращается в шум.

Часть III. Связная теория понятий

В этой части термины не идут как мастерские или практические задания. Они вплетены в теоретические тома. Каждый том показывает, как понятия работают вместе и почему одно понятие нельзя понимать без соседних.

Том 1. Том I. Грамотность байтов и форматов

Этот том учит видеть бинарный объект как набор слоев, а не как страшный файл без смысла.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 1. байт

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - general reverse engineering. байт - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть байт можно не вообще где-то, а в конкретных следах: Ищи байт в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне байт должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. байт не живет один. Главные соседи для байт: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: запомнить слово, но не знать, где оно живет и как доказывается. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У байт есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда байт попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для байт полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин байт встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории байт нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя байт должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary байт редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если байт связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и

hardware-мире байт напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда байт объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 2. число

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое general reverse engineering это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. число - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для заголовок файла сначала ищут не красивую интерпретацию, а признаки существования: Ищи число в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне число должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг число выглядит так: Главные соседи для число: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: запомнить слово, но не знать, где оно живет и как доказывается. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. число нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для число важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке число часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить число как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР число ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У число есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении число важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает число, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу число может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь

эти углы сведены в один фундамент. Когда фундамент построен, `число` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 3. hex

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте general reverse engineering эту потребность закрывает именно это понятие. hex - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы hex таковы: Ищи hex в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне hex должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи для hex: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг hex обычно такая: запомнить слово, но не знать, где оно живет и как доказывается. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя hex должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary hex редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если hex связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире hex напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда hex объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для hex полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если hex кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста hex вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность hex, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не

научится. Поэтому hex здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 4. endian

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое general reverse engineering это выглядит так: `endian` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `таблица полей` проявления находятся здесь: Ищи `endian` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне `endian` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи для `endian`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: запомнить слово, но не знать, где оно живет и как доказывается. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `endian` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `endian` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `endian`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `endian` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `endian` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `endian` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `endian` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `endian` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа,

которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `endian` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `endian` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `endian` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `endian` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `endian` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `endian` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `endian` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `endian` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 5. кодировка

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В general reverse engineering без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. кодировка - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи кодировка в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне кодировка должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи кодировка: Главные соседи для кодировка: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для кодировка полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если кодировка кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста кодировка вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность кодировка, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому кодировка здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У кодировка есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда кодировка попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для кодировка полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин кодировка встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В

долгой учебной траектории кодировка нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 6. выравнивание

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя general reverse engineering. выравнивание - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `заголовок файла` это понятие проявляется через места, которые можно указать пальцем: Ищи выравнивание в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под выравнивание устроен так: На низком уровне выравнивание должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у выравнивание такие: Главные соседи для выравнивание: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить выравнивание в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на выравнивание со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для выравнивание особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают выравнивание через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же выравнивание часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера выравнивание должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда выравнивание попадает в отчет или объяснение, его нельзя оставлять голым английским словом.

Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У выравнивание почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина выравнивание - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности выравнивание часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять выравнивание в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, выравнивание помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. выравнивание нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для выравнивание важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке выравнивание часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить выравнивание как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР выравнивание ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 7. checksum

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - general reverse engineering. `checksum` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `checksum` можно не вообще где-то, а в конкретных следах: Ищи `checksum` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне `checksum` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `checksum` не живет один. Главные соседи для `checksum`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: запомнить слово, но не знать, где оно живет и как доказывается. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `checksum` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `checksum` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `checksum` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `checksum` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `checksum` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `checksum` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `checksum` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `checksum` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что

позволяют правила и наблюдения. В embedded и hardware-мире `checksum` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `checksum` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 8. entropy

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое general reverse engineering это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. entropy - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для таблица полей сначала ищут не красивую интерпретацию, а признаки существования: Ищи entropy в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне entropy должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг entropy выглядит так: Главные соседи для entropy: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: запомнить слово, но не знать, где оно живет и как доказывается. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. entropy нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для entropy важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке entropy часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить entropy как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР entropy ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У entropy есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении entropy важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает entropy, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу entropy может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis -

защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `entropy` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 9. container

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте general reverse engineering эту потребность закрывает именно это понятие. `container` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `container` таковы: Ищи `container` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне `container` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи для `container`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `container` обычно такая: запомнить слово, но не знать, где оно живет и как доказывается. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `container` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `container` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `container` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `container` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `container` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `container` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `container` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `container` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `container`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные

различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `container` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 10. signature

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое general reverse engineering это выглядит так: `signature` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `заголовок файла` проявления находятся здесь: Ищи `signature` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне `signature` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи для `signature`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: запомнить слово, но не знать, где оно живет и как доказывается. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `signature` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `signature` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `signature`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `signature` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `signature` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `signature` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `signature` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `signature` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа,

которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `signature` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `signature` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `signature` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `signature` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `signature` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `signature` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `signature` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `signature` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 11. offset

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В general reverse engineering без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `offset` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `offset` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне `offset` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `offset`: Главные соседи для `offset`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `offset` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `offset` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `offset` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `offset`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `offset` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `offset` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `offset` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `offset` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `offset` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной

траектории `offset` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 12. evidence

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя general reverse engineering. `evidence` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `таблица` полей это понятие проявляется через места, которые можно указать пальцем: Ищи `evidence` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `evidence` устроен так: На низком уровне `evidence` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `evidence` такие: Главные соседи для `evidence`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `evidence` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `evidence` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `evidence` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `evidence` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `evidence` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `evidence` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `evidence` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское

раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `evidence` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `evidence` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `evidence` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `evidence` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `evidence` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `evidence` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `evidence` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `evidence` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `evidence` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `evidence` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 1

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: байт, число, hex, endian, кодировка, выравнивание, checksum, entropy, container, signature, offset, evidence. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 2. Том II. C, память и данные как материал реверса

Здесь язык C рассматривается не как язык программирования для новичка, а как источник машинных форм.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 13. pointer

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - general reverse engineering. `pointer` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `pointer` можно не вообще где-то, а в конкретных следах: Ищи `pointer` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне `pointer` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `pointer` не живет один. Главные соседи для `pointer`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: запомнить слово, но не знать, где оно живет и как доказывается. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `pointer` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `pointer` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `pointer` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `pointer` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `pointer` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `pointer` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `pointer` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `pointer` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что

позволяют правила и наблюдения. В embedded и hardware-мире `pointer` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `pointer` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 14. array

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое general reverse engineering это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `array` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `локальный` буфер сначала ищут не красивую интерпретацию, а признаки существования: Ищи `array` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне `array` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `array` выглядит так: Главные соседи для `array`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: запомнить слово, но не знать, где оно живет и как доказывается. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `array` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `array` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `array` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `array` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `array` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `array` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `array` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `array`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `array` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь

эти углы сведены в один фундамент. Когда фундамент построен, `array` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 15. struct

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте general reverse engineering эту потребность закрывает именно это понятие. `struct` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `struct` таковы: Ищи `struct` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне `struct` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи для `struct`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `struct` обычно такая: запомнить слово, но не знать, где оно живет и как доказывается. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `struct` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `struct` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `struct` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `struct` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `struct` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `struct` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `struct` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `struct` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `struct`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали,

читатель утонет; если оставить только определение, он ничему не научится. Поэтому `struct` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 16. union

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое general reverse engineering это выглядит так: `union` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `allocator trace` проявления находятся здесь: Ищи `union` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне `union` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи для `union`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: запомнить слово, но не знать, где оно живет и как доказывается. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `union` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `union` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `union`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `union` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `union` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `union` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `union` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `union` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент

его отобразил. В Windows, Linux и embedded один и тот же `union` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `union` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `union` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `union` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `union` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `union` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `union` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `union` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 17. padding

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В general reverse engineering без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `padding` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `padding` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне `padding` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `padding`: Главные соседи для `padding`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `padding` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `padding` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `padding` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `padding`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `padding` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `padding` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `padding` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `padding` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `padding` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой

учебной траектории `padding` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 18. signedness

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя general reverse engineering. `signedness` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `локальный буфер` это понятие проявляется через места, которые можно указать пальцем: Ищи `signedness` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `signedness` устроен так: На низком уровне `signedness` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `signedness` такие: Главные соседи для `signedness`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `signedness` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `signedness` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `signedness` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `signedness` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `signedness` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `signedness` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `signedness` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское

раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `signedness` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `signedness` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `signedness` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `signedness` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `signedness` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `signedness` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `signedness` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `signedness` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `signedness` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `signedness` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 19. cast

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - general reverse engineering. `cast` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `cast` можно не вообще где-то, а в конкретных следах: Ищи `cast` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне `cast` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `cast` не живет один. Главные соседи для `cast`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: запомнить слово, но не знать, где оно живет и как доказывается. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `cast` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `cast` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `cast` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `cast` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `cast` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `cast` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `cast` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `cast` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и

hardware-мире `cast` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `cast` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 20. heap

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое general reverse engineering это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. heap - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `allocator trace` сначала ищут не красивую интерпретацию, а признаки существования: Ищи heap в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне heap должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг heap выглядит так: Главные соседи для heap: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: запомнить слово, но не знать, где оно живет и как доказывается. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. heap нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для heap важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке heap часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить heap как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР heap ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У heap есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении heap важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает heap, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу heap может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один

фундамент. Когда фундамент построен, heap перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 21. stack object

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте general reverse engineering эту потребность закрывает именно это понятие. `stack object` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `stack object` таковы: Ищи `stack object` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне `stack object` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи для `stack object`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `stack object` обычно такая: запомнить слово, но не знать, где оно живет и как доказывается. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `stack object` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `stack object` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `stack object` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `stack object` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `stack object` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `stack object` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `stack object` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `stack object` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `stack object`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние

понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `stack object` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 22. function pointer

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое general reverse engineering это выглядит так: `function pointer` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `локальный буфер` проявления находятся здесь: Ищи `function pointer` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне `function pointer` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи для `function pointer`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: запомнить слово, но не знать, где оно живет и как доказывается. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `function pointer` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `function pointer` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `function pointer`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `function pointer` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `function pointer` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `function pointer` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `function pointer` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `function pointer` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser

может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `function pointer` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `function pointer` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `function pointer` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `function pointer` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `function pointer` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `function pointer` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `function pointer` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `function pointer` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 23. undefined behavior

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В general reverse engineering без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `undefined behavior` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `undefined behavior` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне `undefined behavior` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `undefined behavior`: Главные соседи для `undefined behavior`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `undefined behavior` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `undefined behavior` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `undefined behavior` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `undefined behavior`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `undefined behavior` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `undefined behavior` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `undefined behavior` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `undefined behavior` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `undefined behavior` встречается в книге, рядом должен появиться не

только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `undefined behavior` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 24. buffer

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя general reverse engineering. `buffer` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `allocator trace` это понятие проявляется через места, которые можно указать пальцем: Ищи `buffer` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `buffer` устроен так: На низком уровне `buffer` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `buffer` такие: Главные соседи для `buffer`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `buffer` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `buffer` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `buffer` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `buffer` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `buffer` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `buffer` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `buffer` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это,

зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `buffer` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `buffer` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `buffer` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `buffer` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `buffer` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `buffer` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `buffer` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `buffer` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `buffer` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `buffer` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 2

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: pointer, array, struct, union, padding, signedness, cast, heap, stack object, function pointer, undefined behavior, buffer. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 3. Том III. CPU, assembly и исполнение

Этот том связывает инструкции, регистры, память и переходы в единую модель выполнения.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 25. fetch

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - general reverse engineering. `fetch` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `fetch` можно не вообще где-то, а в конкретных следах: Ищи `fetch` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне `fetch` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `fetch` не живет один. Главные соседи для `fetch`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: запомнить слово, но не знать, где оно живет и как доказывается. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `fetch` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `fetch` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `fetch` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `fetch` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `fetch` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `fetch` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `fetch` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `fetch` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и

hardware-мире `fetch` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `fetch` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 26. decode

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое CPU/architecture это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. decode относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для обработчик ARM сначала ищут не красивую интерпретацию, а признаки существования: Ищи decode в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг decode выглядит так: Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. decode нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для decode важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке decode часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить decode как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР decode ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У decode есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении decode важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает decode, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу decode может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных

и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `decode` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 27. register

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте CPU/architecture эту потребность закрывает именно это понятие. `register` относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `register` таковы: Ищи `register` в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `register` обычно такая: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `register` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `register` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `register` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `register` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `register` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `register` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `register` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `register` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `register`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом

исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `register` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 28. flags

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое tools/workflow это выглядит так: `flags` - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `трасса debugger` проявления находятся здесь: Ищи `flags` в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `flags` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `flags` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `flags`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `flags` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `flags` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `flags` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `flags` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `flags` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило

платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `flags` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `flags` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `flags` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `flags` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `flags` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `flags` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `flags` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `flags` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 29. addressing

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В general reverse engineering без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `addressing` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `addressing` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне `addressing` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `addressing`: Главные соседи для `addressing`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `addressing` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `addressing` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `addressing` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `addressing`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `addressing` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `addressing` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `addressing` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `addressing` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `addressing` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще

понадобился в устройстве программы. В долгой учебной траектории `addressing` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 30. branch

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя CPU/architecture. branch относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте обработчик ARM это понятие проявляется через места, которые можно указать пальцем: Ищи branch в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под branch устроен так: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у branch такие: Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить branch в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на branch со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для branch особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают branch через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же branch часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера branch должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда branch попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть

русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У branch почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина branch - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности branch часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять branch в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, branch помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. branch нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для branch важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке branch часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить branch как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР branch ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 31. call

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - CPU/architecture. `call` относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `call` можно не вообще где-то, а в конкретных следах: Ищи `call` в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `call` не живет один. Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `call` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `call` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `call` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `call` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `call` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `call` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `call` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `call` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что

позволяют правила и наблюдения. В embedded и hardware-мире `call` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `call` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 32. return

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое tools/workflow это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `return` - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `трасса debugger` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `return` в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `return` выглядит так: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `return` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `return` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `return` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `return` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `return` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `return` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `return` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `return`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `return` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы,

malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `return` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 33. interrupt

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте general reverse engineering эту потребность закрывает именно это понятие. `interrupt` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `interrupt` таковы: Ищи `interrupt` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне `interrupt` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи для `interrupt`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `interrupt` обычно такая: запомнить слово, но не знать, где оно живет и как доказывается. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `interrupt` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `interrupt` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `interrupt` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `interrupt` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `interrupt` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `interrupt` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `interrupt` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `interrupt` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `interrupt`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные

различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `interrupt` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 34. fault

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое hardware security это выглядит так: `fault` переносит реверс из файла в физический мир. Здесь доказательство может быть не только инструкцией, но и сигналом на линии, формой импульса, дампом памяти, реакцией платы или записью логического анализатора. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `hardware target` проявления находятся здесь: Ищи `fault` в board photos, pinout, datasheets, bus traces, SPI flash dumps, JTAG/SWD configuration, oscilloscope captures, boot ROM behavior, TrustZone transitions и secure element communication. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне устройство - не абстрактная программа. Питание, reset, clock, timing на шине и protection fuses определяют, что вообще можно наблюдать. Side-channel видит косвенную утечку, fault injection пытается нарушить вычисление, а hardware trace требует калибровки до интерпретации. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: voltage, clock, reset, bus protocol, timing, flash layout, debug port, threat model, safety и repeatability. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: считать единичный trace доказательством или игнорировать риск повредить устройство. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `fault` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `fault` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `fault`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `fault` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `fault` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `fault` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `fault` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `fault` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace,

parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `fault` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `fault` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `fault` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `fault` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `fault` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `fault` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `fault` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `fault` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 35. cache

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В CPU/architecture без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. cache относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи cache в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи cache: Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для cache полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если cache кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста cache вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность cache, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому cache здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У cache есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда cache попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для cache полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин cache встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной

траектории `cache` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 36. atomic

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя tools/workflow. `atomic` - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `трасса debugger` это понятие проявляется через места, которые можно указать пальцем: Ищи `atomic` в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `atomic` устроен так: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `atomic` такие: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `atomic` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `atomic` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `atomic` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `atomic` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `atomic` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `atomic` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `atomic` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет,

какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `atomic` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `atomic` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `atomic` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `atomic` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `atomic` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `atomic` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `atomic` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `atomic` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `atomic` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `atomic` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 3

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: fetch, decode, register, flags, addressing, branch, call, return, interrupt, fault, cache, atomic. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 4. Том IV. Компиляция, линковка и runtime

Здесь показывается, почему бинарник не похож на исходник и какие следы оставляет toolchain.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 37. preprocessor

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - managed runtime. `preprocessor` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `preprocessor` можно не вообще где-то, а в конкретных следах: Ищи `preprocessor` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `preprocessor` не живет один. Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `preprocessor` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `preprocessor` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `preprocessor` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `preprocessor` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `preprocessor` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `preprocessor` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `preprocessor` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события.

Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `preprocessor` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `preprocessor` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `preprocessor` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 38. compiler IR

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое managed runtime это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `compiler IR` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `map file` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `compiler IR` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `compiler IR` выглядит так: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `compiler IR` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `compiler IR` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `compiler IR` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `compiler IR` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `compiler IR` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `compiler IR` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `compiler IR` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `compiler IR`, это начало разговора. Нужно

спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `compiler IR` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `compiler IR` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 39. object file

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте managed runtime эту потребность закрывает именно это понятие. `object file` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `object file` таковы: Ищи `object file` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `object file` обычно такая: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `object file` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `object file` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `object file` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `object file` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `object file` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `object file` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `object file` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `object file` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function

- ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `object file`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `object file` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 40. symbol

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое Android это выглядит так: `symbol` в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `startup code` проявления находятся здесь: Ищи `symbol` в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `symbol` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `symbol` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `symbol`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `symbol` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `symbol` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `symbol` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `symbol` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `symbol` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому

фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `symbol` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `symbol` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `symbol` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `symbol` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `symbol` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `symbol` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `symbol` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `symbol` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 41. relocation

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В managed runtime без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `relocation` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `relocation` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `relocation`: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `relocation` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `relocation` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `relocation` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `relocation`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `relocation` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `relocation` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `relocation` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `relocation` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница

часто объясняет больше, чем отдельная инструкция. Если термин `relocation` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `relocation` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 42. linker script

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя managed runtime. `linker script` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `map file` это понятие проявляется через места, которые можно указать пальцем: Ищи `linker script` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `linker script` устроен так: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `linker script` такие: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `linker script` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `linker script` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `linker script` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `linker script` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `linker script` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `linker script` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где

проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `linker script` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `linker script` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `linker script` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `linker script` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `linker script` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `linker script` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `linker script` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `linker script` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `linker script` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `linker script` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `linker script` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 43. static library

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - toolchain. `static library` относится к дороге от исходного текста до исполняемого артефакта. Reverse engineering видит не исходник, а следы решений compiler, assembler, linker и runtime. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `static library` можно не вообще где-то, а в конкретных следах: Ищи `static library` в object files, symbols, relocation records, map files, debug info, startup code, optimized functions, library calls и generated metadata. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне compiler удаляет лишнее, объединяет похожее, встраивает функции, меняет порядок и специализирует code. Linker соединяет object files, разрешает symbols, применяет relocations, размещает sections и может выбросить неиспользованные части. Runtime готовит memory и language environment до того, как логика программы станет видимой. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `static library` не живет один. Главные соседи: translation unit, IR, object file, symbol, relocation, section, linker script, runtime init и optimization. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, a bytecode без runtime. Главная ловушка: считать every missing call доказательством исчезновения логики, хотя compiler мог встроить функцию или заменить вызов на tail-call. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `static library` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `static library` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, a hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `static library` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-a и начинается ответственность loader-a? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `static library` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `static library` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `static library` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `static library` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно

наблюдение поддерживает другое. Если `static library` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `static library` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `static library` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 44. dynamic library

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Android это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `dynamic library` в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `startup code` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `dynamic library` в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `dynamic library` выглядит так: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `dynamic library` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `dynamic library` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `dynamic library` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `dynamic library` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `dynamic library` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `dynamic library` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `dynamic library` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `dynamic library`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах

по реверсу `dynamic library` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `dynamic library` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 45. optimization

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте managed runtime эту потребность закрывает именно это понятие. `optimization` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `optimization` таковы: Ищи `optimization` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `optimization` обычно такая: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `optimization` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `optimization` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `optimization` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `optimization` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `optimization` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `optimization` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `optimization` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `optimization` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function

- ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `optimization`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `optimization` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 46. debug info

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое managed runtime это выглядит так: `debug info` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `map file` проявления находятся здесь: Ищи `debug info` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `debug info` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `debug info` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `debug info`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `debug info` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `debug info` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `debug info` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `debug info` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime.

Инструменты показывают `debug info` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `debug info` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `debug info` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `debug info` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `debug info` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `debug info` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `debug info` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `debug info` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `debug info` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 47. runtime

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В toolchain без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `runtime` относится к дороге от исходного текста до исполняемого артефакта. Reverse engineering видит не исходник, а следы решений compiler, assembler, linker и runtime. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `runtime` в object files, symbols, relocation records, map files, debug info, startup code, optimized functions, library calls и generated metadata. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне compiler удаляет лишнее, объединяет похожее, встраивает функции, меняет порядок и специализирует code. Linker соединяет object files, разрешает symbols, применяет relocations, размещает sections и может выбросить неиспользованные части. Runtime готовит memory и language environment до того, как логика программы станет видимой. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `runtime`: Главные соседи: translation unit, IR, object file, symbol, relocation, section, linker script, runtime init и optimization. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: считать every missing call доказательством исчезновения логики, хотя compiler мог встроить функцию или заменить вызов на tail-call. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `runtime` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `runtime` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `runtime` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `runtime`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `runtime` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `runtime` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `runtime` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `runtime` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `runtime` встречается в книге, рядом должен появиться не только

перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `runtime` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 48. startup

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя Android. `startup` в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `Android application` это понятие проявляется через места, которые можно указать пальцем: Ищи `startup` в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `startup` устроен так: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `startup` такие: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `startup` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `startup` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `startup` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `startup` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `startup` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `startup` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `startup` попадает в отчет или объяснение, его нельзя оставлять голым

английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `startup` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `startup` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `startup` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `startup` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `startup` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `startup` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `startup` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `startup` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `startup` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `startup` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 4

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: preprocessor, compiler IR, object file, symbol, relocation, linker script, static library, dynamic library, optimization, debug info, runtime, startup. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 5. Том V. Windows reverse engineering

Этот том углубляет Windows-ветку: PE, x64 ABI, unwind, отладка и системное окружение.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 49. DOS header

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Windows/PE. DOS header в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть DOS header можно не вообще где-то, а в конкретных следах: Ищи DOS header в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. DOS header не живет один. Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У DOS header есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда DOS header попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для DOS header полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин DOS header встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории DOS header нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя DOS header должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary DOS header редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table

entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `DOS header` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `DOS header` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `DOS header` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 50. PE headers

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Windows/PE это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `PE headers` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `DLL` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `PE headers` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `PE headers` выглядит так: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `PE headers` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `PE headers` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `PE headers` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `PE headers` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `PE headers` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `PE headers` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `PE headers` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `PE headers`, это начало разговора. Нужно спросить, откуда инструмент

взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `PE headers` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `PE headers` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 51. section table

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Windows/PE эту потребность закрывает именно это понятие. `section table` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `section table` таковы: Ищи `section table` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `section table` обычно такая: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `section table` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `section table` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `section table` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `section table` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `section table` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `section table` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `section table` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `section table`

вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `section table`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `section table` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 52. imports

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое Windows/PE это выглядит так: `imports` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `unwind table` проявления находятся здесь: Ищи `imports` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `imports` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `imports` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `imports`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `imports` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `imports` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `imports` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `imports` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто

появляется только после понимания runtime. Инструменты показывают `imports` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `imports` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `imports` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `imports` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `imports` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `imports` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `imports` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `imports` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `imports` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 53. exports

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Windows/PE без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. exports в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи exports в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи exports: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для exports полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если exports кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста exports вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность exports, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому exports здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У exports есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда exports попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, a hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для exports полезна привычка смотреть на границы. Где заканчивается ответственность compiler-a и начинается ответственность loader-a? Где user-mode переходит в kernel-mode? Где bytecode

превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `exports` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `exports` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 54. relocations

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя Windows/PE. `relocations` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `DLL` это понятие проявляется через места, которые можно указать пальцем: Ищи `relocations` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `relocations` устроен так: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `relocations` такие: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `relocations` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `relocations` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `relocations` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `relocations` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `relocations` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `relocations` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто

исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `relocations` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `relocations` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `relocations` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `relocations` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `relocations` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `relocations` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `relocations` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `relocations` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `relocations` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `relocations` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `relocations` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 55. TLS

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Windows/PE. TLS в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть TLS можно не вообще где-то, а в конкретных следах: Ищи TLS в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. TLS не живет один. Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, a bytecode без runtime. Главная ловушка: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У TLS есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда TLS попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, a hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для TLS полезна привычка смотреть на границы. Где заканчивается ответственность compiler-a и начинается ответственность loader-a? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин TLS встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории TLS нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя TLS должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary TLS редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не

список терминов, а сеть, где одно наблюдение поддерживает другое. Если TLS связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире TLS напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда TLS объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 56. resources

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Windows/PE это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `resources` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `unwind table` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `resources` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `resources` выглядит так: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `resources` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `resources` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `resources` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `resources` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `resources` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `resources` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `resources` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `resources`, это начало разговора. Нужно спросить, откуда инструмент взял

уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `resources` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `resources` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 57. SEH

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Windows/PE эту потребность закрывает именно это понятие. SEH в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы SEH таковы: Ищи SEH в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг SEH обычно такая: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя SEH должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary SEH редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если SEH связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире SEH напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда SEH объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для SEH полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если SEH кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста SEH вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack;

loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность SEH, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому SEH здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 58. x64 ABI

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое general reverse engineering это выглядит так: `x64 ABI` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `DLL` проявления находятся здесь: Ищи `x64 ABI` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне `x64 ABI` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи для `x64 ABI`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: запомнить слово, но не знать, где оно живет и как доказывается. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `x64 ABI` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `x64 ABI` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `x64 ABI`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `x64 ABI` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `x64 ABI` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `x64 ABI` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `x64 ABI` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `x64 ABI` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент

его отобразил. В Windows, Linux и embedded один и тот же `x64 ABI` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `x64 ABI` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `x64 ABI` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `x64 ABI` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `x64 ABI` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `x64 ABI` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `x64 ABI` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `x64 ABI` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 59. unwind

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Windows/PE без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. unwind в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи unwind в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи unwind: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для unwind полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если unwind кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста unwind вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность unwind, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому unwind здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У unwind есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда unwind попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для unwind полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в

native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `unwind` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `unwind` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 60. PDB

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя Windows/PE. PDB в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `unwind table` это понятие проявляется через места, которые можно указать пальцем: Ищи PDB в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под PDB устроен так: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у PDB такие: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить PDB в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на PDB со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для PDB особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают PDB через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же PDB часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера PDB должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где

проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда PDB попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У PDB почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина PDB - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности PDB часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять PDB в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, PDB помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. PDB нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для PDB важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке PDB часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить PDB как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР PDB ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 5

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: DOS header, PE headers, section table, imports, exports, relocations, TLS, resources, SEH, x64 ABI, unwind, PDB. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 6. Том VI. Linux reverse engineering

Этот том показывает Linux-мир: ELF, динамический загрузчик, syscalls, DWARF и процессы.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 61. ELF header

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Linux/ELF. `ELF header` в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `ELF header` можно не вообще где-то, а в конкретных следах: Ищи `ELF header` в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `ELF header` не живет один. Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `ELF header` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `ELF header` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `ELF header` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `ELF header` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `ELF header` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `ELF header` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `ELF header` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где

одно наблюдение поддерживает другое. Если `ELF header` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `ELF header` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `ELF header` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 62. program header

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Linux/ELF это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `program header` в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `shared object` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `program header` в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `program header` выглядит так: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `program header` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `program header` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `program header` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `program header` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `program header` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `program header` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `program header` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `program header`, это начало разговора. Нужно спросить, откуда инструмент взял

уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `program header` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `program header` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 63. section header

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Linux/ELF эту потребность закрывает именно это понятие. `section header` в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `section header` таковы: Ищи `section header` в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `section header` обычно такая: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `section header` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `section header` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `section header` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `section header` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `section header` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `section header` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `section header` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `section header` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function

- ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `section header`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `section header` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 64. GOT

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое Linux/ELF это выглядит так: GOT в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `process maps` проявления находятся здесь: Ищи GOT в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У GOT есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении GOT важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает GOT, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу GOT может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, GOT перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на GOT со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для GOT особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают GOT через свои модели. Ghidra может назвать

функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же GOT часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера GOT должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда GOT попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У GOT почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина GOT - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности GOT часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять GOT в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, GOT помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 65. PLT

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Linux/ELF без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. PLT в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи PLT в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи PLT: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для PLT полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если PLT кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста PLT вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность PLT, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому PLT здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У PLT есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда PLT попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для PLT полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин PLT встречается в книге, рядом должен появиться не только перевод, но

и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории PLT нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 66. ld.so

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя Linux/ELF. `ld.so` в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `shared object` это понятие проявляется через места, которые можно указать пальцем: Ищи `ld.so` в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `ld.so` устроен так: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `ld.so` такие: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `ld.so` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `ld.so` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `ld.so` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `ld.so` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `ld.so` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `ld.so` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы

связывают термин с остальной теорией. Когда `ld.so` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `ld.so` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `ld.so` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `ld.so` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `ld.so` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `ld.so` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `ld.so` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `ld.so` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `ld.so` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `ld.so` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `ld.so` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 67. syscall

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Linux/ELF. `syscall` в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `syscall` можно не вообще где-то, а в конкретных следах: Ищи `syscall` в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `syscall` не живет один. Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `syscall` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `syscall` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `syscall` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `syscall` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `syscall` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `syscall` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `syscall` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если

`syscall` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `syscall` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `syscall` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 68. signal

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Linux/ELF это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `signal` в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `process maps` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `signal` в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `signal` выглядит так: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `signal` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `signal` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `signal` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `signal` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `signal` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `signal` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `signal` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `signal`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool

перестает быть авторитетом и становится помощником. В разных книгах по реверсу `signal` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `signal` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 69. procfs

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Linux/ELF эту потребность закрывает именно это понятие. `procfs` в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `procfs` таковы: Ищи `procfs` в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `procfs` обычно такая: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `procfs` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `procfs` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `procfs` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `procfs` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `procfs` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `procfs` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `procfs` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `procfs` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть

фундамент. В книге для новичка важно не прятать сложность `procfs`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `procfs` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 70. ptrace

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое Linux/ELF это выглядит так: `ptrace` в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `shared object` проявления находятся здесь: Ищи `ptrace` в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `ptrace` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `ptrace` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `ptrace`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `ptrace` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `ptrace` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `ptrace` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `ptrace` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `ptrace` через свои модели. Ghidra может назвать

функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `ptrace` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `ptrace` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `ptrace` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `ptrace` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `ptrace` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `ptrace` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `ptrace` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `ptrace` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 71. DWARF

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Linux/ELF без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. DWARF в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи DWARF в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи DWARF: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для DWARF полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если DWARF кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста DWARF вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность DWARF, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому DWARF здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У DWARF есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда DWARF попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для DWARF полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин DWARF встречается в книге, рядом должен появиться не

только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории DWARF нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 72. core dump

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя Linux/ELF. `core dump` в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `process maps` это понятие проявляется через места, которые можно указать пальцем: Ищи `core dump` в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `core dump` устроен так: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `core dump` такие: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `core dump` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `core dump` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `core dump` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `core dump` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `core dump` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `core dump` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode,

между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `core dump` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `core dump` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `core dump` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `core dump` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `core dump` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `core dump` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `core dump` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `core dump` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `core dump` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `core dump` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `core dump` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 6

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: ELF header, program header, section header, GOT, PLT, ld.so, syscall, signal, procfs, ptrace, DWARF, core dump. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 7. Том VII. Embedded bare metal

Этот том переводит мышление из мира процессов в мир микроконтроллеров, регистров и железа.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 73. memory map

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - embedded/bare metal. `memory map` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `memory map` можно не вообще где-то, а в конкретных следах: Ищи `memory map` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `memory map` не живет один. Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `memory map` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `memory map` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `memory map` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `memory map` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `memory map` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `memory map` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `memory map` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где

одно наблюдение поддерживает другое. Если `memory map` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `memory map` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `memory map` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 74. vector table

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое embedded/bare metal это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `vector table` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `STM32 проект` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `vector table` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `vector table` выглядит так: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `vector table` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `vector table` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `vector table` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `vector table` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `vector table` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `vector table` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `vector table` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `vector table`, это начало разговора. Нужно спросить, откуда инструмент взял

уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `vector table` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `vector table` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 75. reset handler

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте embedded/bare metal эту потребность закрывает именно это понятие. `reset handler` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `reset handler` таковы: Ищи `reset handler` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `reset handler` обычно такая: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `reset handler` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `reset handler` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `reset handler` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `reset handler` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `reset handler` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `reset handler` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `reset handler` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `reset handler` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context;

bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `reset handler`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `reset handler` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 76. MMIO

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое embedded/bare metal это выглядит так: `MMIO` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `лог UART` проявления находятся здесь: Ищи `MMIO` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `MMIO` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `MMIO` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `MMIO`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `MMIO` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `MMIO` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `MMIO` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `MMIO` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `MMIO` через свои модели. Ghidra может назвать функцию, IDA может построить graph,

debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `MMIO` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `MMIO` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `MMIO` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `MMIO` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `MMIO` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `MMIO` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `MMIO` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `MMIO` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 77. GPIO

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В embedded/bare metal без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. GPIO в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи GPIO в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи GPIO: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для GPIO полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если GPIO кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста GPIO вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность GPIO, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому GPIO здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У GPIO есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда GPIO попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для GPIO полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин GPIO встречается в книге, рядом должен появиться не только перевод, но

и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `GPIO` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 78. UART

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя embedded/bare metal. UART в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `Android application` это понятие проявляется через места, которые можно указать пальцем: Ищи UART в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под UART устроен так: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у UART такие: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить UART в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на UART со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для UART особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают UART через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же UART часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера UART должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и

native code? Такие вопросы связывают термин с остальной теорией. Когда UART попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У UART почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина UART - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности UART часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять UART в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, UART помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. UART нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для UART важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке UART часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить UART как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР UART ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 79. SPI

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - embedded/bare metal. `SPI` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `SPI` можно не вообще где-то, а в конкретных следах: Ищи `SPI` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `SPI` не живет один. Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `SPI` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `SPI` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `SPI` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `SPI` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `SPI` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `SPI` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `SPI` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `SPI` связан с безопасностью, нужно быть особенно аккуратным.

Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `SPI` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `SPI` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 80. I2C

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое embedded/bare metal это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `I2C` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `лог UART` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `I2C` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `I2C` выглядит так: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `I2C` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `I2C` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `I2C` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `I2C` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `I2C` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `I2C` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `I2C` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `I2C`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и

становится помощником. В разных книгах по реверсу `I2C` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `I2C` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 81. timer

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте embedded/bare metal эту потребность закрывает именно это понятие. `timer` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `timer` таковы: Ищи `timer` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `timer` обычно такая: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `timer` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `timer` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `timer` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `timer` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `timer` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `timer` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `timer` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `timer` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть

фундамент. В книге для новичка важно не прятать сложность `timer`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `timer` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 82. DMA

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое embedded/bare metal это выглядит так: DMA в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта STM32 проект проявления находятся здесь: Ищи DMA в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У DMA есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении DMA важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает DMA, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу DMA может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, DMA перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на DMA со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для DMA особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают DMA через свои модели. Ghidra может назвать

функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же DMA часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера DMA должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда DMA попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У DMA почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина DMA - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности DMA часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять DMA в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, DMA помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 83. Flash

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В embedded/bare metal без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `Flash` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `Flash` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `Flash`: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `Flash` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `Flash` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `Flash` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `Flash`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `Flash` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `Flash` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `Flash` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `Flash` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `Flash` встречается в книге, рядом должен появиться не только перевод,

но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `Flash` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 84. clock tree

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя embedded/bare metal. `clock tree` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `лог UART` это понятие проявляется через места, которые можно указать пальцем: Ищи `clock tree` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `clock tree` устроен так: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `clock tree` такие: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `clock tree` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `clock tree` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `clock tree` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `clock tree` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `clock tree` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `clock tree` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode,

между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `clock tree` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `clock tree` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `clock tree` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `clock tree` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `clock tree` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `clock tree` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `clock tree` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `clock tree` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `clock tree` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `clock tree` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `clock tree` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 7

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: memory map, vector table, reset handler, MMIO, GPIO, UART, SPI, I2C, timer, DMA, Flash, clock tree. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 8. Том VIII. Архитектуры: x86-64, ARM, RISC-V, Xtensa

Этот том учит переносить мышление между архитектурами без слепого копирования правил.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 85. x86-64

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - CPU/architecture. x86-64 относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть x86-64 можно не вообще где-то, а в конкретных следах: Ищи x86-64 в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. x86-64 не живет один. Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У x86-64 есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда x86-64 попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для x86-64 полезна привычка смотреть на границы. Где заканчивается ответственность compiler-a и начинается ответственность loader-a? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин x86-64 встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории x86-64 нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя x86-64 должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary x86-64 редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если x86-64 связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория

учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире x86-64 напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда x86-64 объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 86. AArch64

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое CPU/architecture это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. AArch64 относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для листинг ARM сначала ищут не красивую интерпретацию, а признаки существования: Ищи AArch64 в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг AArch64 выглядит так: Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. AArch64 нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для AArch64 важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке AArch64 часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить AArch64 как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР AArch64 ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У AArch64 есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении AArch64 важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает AArch64, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу AArch64 может появляться под разным углом. Системные книги объясняют слой ОС, binary

analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `AArch64` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 87. Cortex-M

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте embedded/bare metal эту потребность закрывает именно это понятие. `Cortex-M` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `Cortex-M` таковы: Ищи `Cortex-M` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `Cortex-M` обычно такая: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `Cortex-M` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `Cortex-M` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `Cortex-M` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `Cortex-M` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `Cortex-M` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `Cortex-M` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `Cortex-M` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `Cortex-M` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и

verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность Cortex-M, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому Cortex-M здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 88. Thumb

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое embedded/bare metal это выглядит так: `Thumb` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `ESP32 firmware` проявления находятся здесь: Ищи `Thumb` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `Thumb` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `Thumb` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `Thumb`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `Thumb` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `Thumb` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `Thumb` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `Thumb` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `Thumb` через свои модели. Ghidra может назвать

функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же Thumb часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера Thumb должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда Thumb попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У Thumb почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина Thumb - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности Thumb часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять Thumb в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, Thumb помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 89. RISC-V

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В CPU/architecture без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. RISC-V относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи RISC-V в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи RISC-V: Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для RISC-V полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если RISC-V кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста RISC-V вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность RISC-V, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому RISC-V здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У RISC-V есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда RISC-V попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для RISC-V полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин RISC-V встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В

долгой учебной траектории `RISC-V` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 90. Xtensa

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя CPU/architecture. Xtensa относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте листинг ARM это понятие проявляется через места, которые можно указать пальцем: Ищи Xtensa в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под Xtensa устроен так: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у Xtensa такие: Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить Xtensa в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на Xtensa со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для Xtensa особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают Xtensa через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же Xtensa часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера Xtensa должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда Xtensa попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это,

зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У Xtensa почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина Xtensa - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности Xtensa часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять Xtensa в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, Xtensa помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. Xtensa нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для Xtensa важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке Xtensa часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить Xtensa как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР Xtensa ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 91. MIPS

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - CPU/architecture. MIPS относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть MIPS можно не вообще где-то, а в конкретных следах: Ищи MIPS в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. MIPS не живет один. Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У MIPS есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда MIPS попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для MIPS полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин MIPS встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории MIPS нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя MIPS должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary MIPS редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если MIPS связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что

позволяют правила и наблюдения. В embedded и hardware-мире MIPS напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда MIPS объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 92. calling convention

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое embedded/bare metal это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `calling convention` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `ESP32 firmware` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `calling convention` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `calling convention` выглядит так: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `calling convention` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `calling convention` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `calling convention` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `calling convention` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `calling convention` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `calling convention` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `calling convention` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `calling`

`convention`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `calling convention` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `calling convention` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 93. privileged mode

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте managed runtime эту потребность закрывает именно это понятие. `privileged mode` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `privileged mode` таковы: Ищи `privileged mode` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `privileged mode` обычно такая: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `privileged mode` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `privileged mode` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `privileged mode` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `privileged mode` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `privileged mode` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `privileged mode` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `privileged mode` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `privileged mode` вызывает не одну

ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `privileged mode`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `privileged mode` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 94. endianness

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое CPU/architecture это выглядит так: `endianness` относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `листинг ARM` проявления находятся здесь: Ищи `endianness` в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `endianness` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `endianness` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `endianness`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `endianness` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `endianness` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `endianness` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `endianness` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `endianness` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через

допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `endianness` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `endianness` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `endianness` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `endianness` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `endianness` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `endianness` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `endianness` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `endianness` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 95. SIMD

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В CPU/architecture без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. SIMD относится к архитектуре - набору правил, по которым процессор понимает инструкции, регистры, адреса, исключения и иногда privilege levels. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи SIMD в disassembly, register view, ABI docs, exception vectors, instruction manual, compiler output и emulator traces. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне CPU снова и снова берет instruction bytes, декодирует их, читает registers/memory, выполняет операцию, обновляет state и выбирает следующий address. Разные ISA families решают одни и те же роли по-разному, поэтому сильный реверсер сравнивает роли, а не только mnemonics. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи SIMD: Главные соседи: instruction set, register file, flags/status register, memory addressing, ABI, exception model, privilege и endianness. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: искать x86-pattern в ARM или читать Thumb/AArch64/RISC-V без проверки mode и ABI. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для SIMD полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если SIMD кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста SIMD вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность SIMD, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому SIMD здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У SIMD есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда SIMD попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для SIMD полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин SIMD встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной

траектории `SIMD` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 96. DSP

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя embedded/bare metal. DSP в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте ESP32 firmware это понятие проявляется через места, которые можно указать пальцем: Ищи DSP в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под DSP устроен так: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует .data и очищает .bss, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у DSP такие: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить DSP в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на DSP со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для DSP особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают DSP через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же DSP часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера DSP должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с

остальной теорией. Когда DSP попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У DSP почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина DSP - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности DSP часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять DSP в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, DSP помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. DSP нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для DSP важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке DSP часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить DSP как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР DSP ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 8

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: x86-64, AArch64, Cortex-M, Thumb, RISC-V, Xtensa, MIPS, calling convention, privileged mode, endianness, SIMD, DSP. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 9. Том IX. Firmware, hardware tools и лабораторная работа с устройствами

Этот том добавляет физический слой: как добывают, запускают и проверяют прошивки.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 97. JTAG

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - embedded/bare metal. `JTAG` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `JTAG` можно не вообще где-то, а в конкретных следах: Ищи `JTAG` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `JTAG` не живет один. Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `JTAG` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `JTAG` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `JTAG` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `JTAG` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `JTAG` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `JTAG` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `JTAG` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `JTAG` связан с безопасностью, нужно быть особенно аккуратным.

Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `JTAG` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `JTAG` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 98. SWD

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое embedded/bare metal это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. SWD в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для роутер сначала ищут не красивую интерпретацию, а признаки существования: Ищи SWD в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг SWD выглядит так: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. SWD нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для SWD важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке SWD часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить SWD как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР SWD ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У SWD есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении SWD важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает SWD, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и

становится помощником. В разных книгах по реверсу SWD может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, SWD перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 99. OpenOCD

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Windows/PE эту потребность закрывает именно это понятие. OpenOCD в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы OpenOCD таковы: Ищи OpenOCD в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг OpenOCD обычно такая: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя OpenOCD должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary OpenOCD редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если OpenOCD связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире OpenOCD напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда OpenOCD объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для OpenOCD полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если OpenOCD кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста OpenOCD вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map;

function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность OpenOCD, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому OpenOCD здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 100. logic analyzer

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое embedded/bare metal это выглядит так: `logic analyzer` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `отладочный адаптер` проявления находятся здесь: Ищи `logic analyzer` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `logic analyzer` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `logic analyzer` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `logic analyzer`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `logic analyzer` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `logic analyzer` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `logic analyzer` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `logic analyzer` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime.

Инструменты показывают `logic analyzer` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `logic analyzer` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `logic analyzer` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `logic analyzer` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `logic analyzer` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `logic analyzer` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `logic analyzer` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `logic analyzer` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `logic analyzer` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 101. UART console

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В embedded/bare metal без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `UART console` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `UART console` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `UART console`: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `UART console` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `UART console` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `UART console` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `UART console`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `UART console` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `UART console` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `UART console` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `UART console` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная

инструкция. Если термин `UART console` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `UART console` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 102. flash dump

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя embedded/bare metal. `flash dump` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `роутер` это понятие проявляется через места, которые можно указать пальцем: Ищи `flash dump` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `flash dump` устроен так: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `flash dump` такие: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `flash dump` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `flash dump` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `flash dump` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `flash dump` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `flash dump` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `flash dump` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode,

между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `flash dump` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `flash dump` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `flash dump` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `flash dump` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `flash dump` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `flash dump` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `flash dump` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `flash dump` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `flash dump` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `flash dump` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `flash dump` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 103. chip-off

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - embedded/bare metal. `chip-off` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `chip-off` можно не вообще где-то, а в конкретных следах: Ищи `chip-off` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `chip-off` не живет один. Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `chip-off` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `chip-off` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `chip-off` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `chip-off` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `chip-off` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `chip-off` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `chip-off` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое.

Если `chip-off` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `chip-off` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `chip-off` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 104. binwalk

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое embedded/bare metal это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `binwalk` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `отладочный адаптер` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `binwalk` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `binwalk` выглядит так: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `binwalk` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `binwalk` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `binwalk` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `binwalk` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `binwalk` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `binwalk` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `binwalk` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `binwalk`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть

авторитетом и становится помощником. В разных книгах по реверсу `binwalk` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `binwalk` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 105. filesystem

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте embedded/bare metal эту потребность закрывает именно это понятие. `filesystem` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `filesystem` таковы: Ищи `filesystem` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `filesystem` обычно такая: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `filesystem` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `filesystem` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `filesystem` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `filesystem` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `filesystem` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `filesystem` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `filesystem` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `filesystem` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и

permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `filesystem`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `filesystem` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 106. QEMU

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое embedded/bare metal это выглядит так: QEMU в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта роутер проявления находятся здесь: Ищи QEMU в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У QEMU есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении QEMU важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает QEMU, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу QEMU может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, QEMU перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на QEMU со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для QEMU особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают QEMU через свои модели. Ghidra может назвать функцию, IDA может построить graph,

debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же QEMU часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера QEMU должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда QEMU попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У QEMU почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина QEMU - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности QEMU часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять QEMU в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, QEMU помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 107. Unicorn

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В embedded/bare metal без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `Unicorn` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `Unicorn` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `Unicorn`: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `Unicorn` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `Unicorn` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `Unicorn` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `Unicorn`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `Unicorn` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `Unicorn` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `Unicorn` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `Unicorn` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `Unicorn` встречается в книге, рядом

должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `Unicorn` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 108. hardware safety

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя embedded/bare metal. `hardware safety` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `отладочный адаптер` это понятие проявляется через места, которые можно указать пальцем: Ищи `hardware safety` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `hardware safety` устроен так: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `hardware safety` такие: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `hardware safety` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `hardware safety` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `hardware safety` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `hardware safety` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `hardware safety` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `hardware safety` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между

файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `hardware safety` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `hardware safety` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `hardware safety` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `hardware safety` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `hardware safety` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `hardware safety` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `hardware safety` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `hardware safety` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `hardware safety` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `hardware safety` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `hardware safety` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 9

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: JTAG, SWD, OpenOCD, logic analyzer, UART console, flash dump, chip-off, binwalk, filesystem, QEMU, Unicorn, hardware safety. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 10. Том X. Инструменты и рабочий процесс

Этот том учит пользоваться инструментами как увеличительным стеклом, а не как кнопкой истины.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 109. Ghidra

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - tools/workflow. `Ghidra` - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `Ghidra` можно не вообще где-то, а в конкретных следах: Ищи `Ghidra` в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `Ghidra` не живет один. Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `Ghidra` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `Ghidra` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `Ghidra` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `Ghidra` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `Ghidra` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `Ghidra` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `Ghidra` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `Ghidra` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория

учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `Ghidra` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `Ghidra` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 110. IDA

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое tools/workflow это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. IDA - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для сессия debugger сначала ищут не красивую интерпретацию, а признаки существования: Ищи IDA в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг IDA выглядит так: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. IDA нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для IDA важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке IDA часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить IDA как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР IDA ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У IDA есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении IDA важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает IDA, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу IDA может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis -

защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `IDA` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 111. Rizin

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте tools/workflow эту потребность закрывает именно это понятие. `Rizin` - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `Rizin` таковы: Ищи `Rizin` в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `Rizin` обычно такая: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `Rizin` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `Rizin` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `Rizin` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `Rizin` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `Rizin` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `Rizin` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `Rizin` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `Rizin` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `Rizin`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали,

читатель утонет; если оставить только определение, он ничему не научится. Поэтому `Rizin` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 112. Binary Ninja

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое tools/workflow это выглядит так: `Binary Ninja` - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `исследовательский отчет` проявления находятся здесь: Ищи `Binary Ninja` в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `Binary Ninja` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `Binary Ninja` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `Binary Ninja`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `Binary Ninja` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `Binary Ninja` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `Binary Ninja` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `Binary Ninja` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `Binary Ninja` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может

вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `Binary Ninja` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `Binary Ninja` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `Binary Ninja` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `Binary Ninja` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `Binary Ninja` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `Binary Ninja` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `Binary Ninja` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `Binary Ninja` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 113. x64dbg

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В tools/workflow без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. x64dbg - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи x64dbg в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи x64dbg: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для x64dbg полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если x64dbg кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста x64dbg вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность x64dbg, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому x64dbg здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У x64dbg есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда x64dbg попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для x64dbg полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин x64dbg встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В

долгой учебной траектории x64dbg нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 114. WinDbg

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя tools/workflow. `WinDbg` - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `сессия debugger` это понятие проявляется через места, которые можно указать пальцем: Ищи `WinDbg` в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `WinDbg` устроен так: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `WinDbg` такие: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `WinDbg` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `WinDbg` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `WinDbg` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `WinDbg` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `WinDbg` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `WinDbg` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `WinDbg` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет,

какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У WinDbg почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина WinDbg - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности WinDbg часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять WinDbg в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, WinDbg помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. WinDbg нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для WinDbg важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке WinDbg часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить WinDbg как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР WinDbg ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 115. GDB

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - tools/workflow. GDB - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть GDB можно не вообще где-то, а в конкретных следах: Ищи GDB в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. GDB не живет один. Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, a bytecode без runtime. Главная ловушка: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У GDB есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда GDB попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, a hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для GDB полезна привычка смотреть на границы. Где заканчивается ответственность compiler-a и начинается ответственность loader-a? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин GDB встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории GDB нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя GDB должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary GDB редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если GDB связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и

hardware-мире GDB напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда GDB объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 116. Python scripting

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое tools/workflow это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. Python scripting - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для исследовательский отчет сначала ищут не красивую интерпретацию, а признаки существования: Ищи Python scripting в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг Python scripting выглядит так: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. Python scripting нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для Python scripting важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке Python scripting часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить Python scripting как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР Python scripting ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У Python scripting есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении Python scripting важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает Python scripting, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится

помощником. В разных книгах по реверсу `Python scripting` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `Python scripting` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 117. types

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Windows/PE эту потребность закрывает именно это понятие. `types` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `types` таковы: Ищи `types` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `types` обычно такая: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `types` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `types` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `types` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `types` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `types` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `types` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `types` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `types` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map;

function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `types`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `types` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 118. signatures

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое tools/workflow это выглядит так: `signatures` - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `сессия debugger` проявления находятся здесь: Ищи `signatures` в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `signatures` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `signatures` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `signatures`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `signatures` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `signatures` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `signatures` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `signatures` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `signatures` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через

допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `signatures` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `signatures` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `signatures` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `signatures` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `signatures` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `signatures` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `signatures` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `signatures` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 119. notes

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В tools/workflow без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. notes - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи notes в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи notes: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для notes полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если notes кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста notes вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность notes, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому notes здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У notes есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда notes попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для notes полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин notes встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной

траектории `notes` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 120. report

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя tools/workflow. `report` - это часть рабочего процесса. Инструмент помогает видеть и проверять, но он не несет ответственности за вывод вместо исследователя. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `исследовательский отчет` это понятие проявляется через места, которые можно указать пальцем: Ищи `report` в disassembler project, debugger session, script output, type database, signature matches, trace logs и report tables. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `report` устроен так: На низком уровне tool строит модель из bytes: дизассемблирует, угадывает функции, распространяет типы, разрешает references и рисует представления. Каждый шаг может быть неполным. Задача исследователя - улучшать модель и сохранять цепочку от наблюдения к выводу. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `report` такие: Главные соседи: hypothesis, xrefs, types, decompiler assumptions, breakpoint, trace, notes, reproducibility и evidence table. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: написать «Ghidra показала» вместо объяснения bytes, references и правил, на которых держится claim. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `report` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `report` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `report` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `report` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `report` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `report` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `report` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем

существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `report` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `report` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `report` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `report` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `report` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `report` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `report` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `report` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `report` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `report` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 10

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: Ghidra, IDA, Rizin, Binary Ninja, x64dbg, WinDbg, GDB, Python scripting, types, signatures, notes, report. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 11. Том XI. Защитный анализ, malware и уязвимости

Здесь реверс соединяется с безопасностью, но без превращения книги в руководство по злоупотреблениям.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 121. triage

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Windows/PE. `triage` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `triage` можно не вообще где-то, а в конкретных следах: Ищи `triage` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `triage` не живет один. Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, a bytecode без runtime. Главная ловушка: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `triage` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `triage` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, a hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `triage` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-a и начинается ответственность loader-a? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `triage` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `triage` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `triage` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `triage` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит

не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `triage` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `triage` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `triage` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 122. sandbox

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое general reverse engineering это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. sandbox - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для сетевой лог сначала ищут не красивую интерпретацию, а признаки существования: Ищи sandbox в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне sandbox должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг sandbox выглядит так: Главные соседи для sandbox: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: запомнить слово, но не знать, где оно живет и как доказывается. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. sandbox нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для sandbox важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке sandbox часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить sandbox как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР sandbox ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У sandbox есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении sandbox важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает sandbox, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу sandbox может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis -

защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `sandbox` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 123. packer

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Apple/Mach-O эту потребность закрывает именно это понятие. `packer` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `packer` таковы: Ищи `packer` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `packer` обычно такая: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `packer` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `packer` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `packer` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `packer` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `packer` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `packer` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `packer` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `packer` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка

важно не прятать сложность `packer`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `packer` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 124. anti-debug

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое defensive analysis это выглядит так: `anti-debug` - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `defensive analysis lab sample` проявления находятся здесь: Ищи `anti-debug` в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: выдавать tool result за окончательный вывод или путать detection с explanation. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `anti-debug` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `anti-debug` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `anti-debug`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `anti-debug` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `anti-debug` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `anti-debug` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `anti-debug` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `anti-debug` через свои модели. Ghidra может назвать функцию, IDA

может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `anti-debug` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `anti-debug` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `anti-debug` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `anti-debug` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `anti-debug` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `anti-debug` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `anti-debug` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `anti-debug` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 125. crypto constants

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Windows/PE без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `crypto constants` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `crypto constants` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `crypto constants`: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `crypto constants` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `crypto constants` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `crypto constants` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `crypto constants`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `crypto constants` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `crypto constants` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `crypto constants` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `crypto constants` полезна привычка смотреть на границы. Где заканчивается

ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `crypto constants` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `crypto constants` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 126. protocol

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя general reverse engineering. `protocol` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `сетевой лог` это понятие проявляется через места, которые можно указать пальцем: Ищи `protocol` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `protocol` устроен так: На низком уровне `protocol` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `protocol` такие: Главные соседи для `protocol`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `protocol` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `protocol` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `protocol` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `protocol` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `protocol` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `protocol` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `protocol` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское

раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `protocol` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `protocol` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `protocol` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `protocol` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `protocol` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `protocol` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `protocol` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `protocol` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `protocol` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `protocol` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 127. persistence

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Windows/PE. `persistence` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `persistence` можно не вообще где-то, а в конкретных следах: Ищи `persistence` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `persistence` не живет один. Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, a bytecode без runtime. Главная ловушка: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `persistence` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `persistence` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, a hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `persistence` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-a и начинается ответственность loader-a? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `persistence` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `persistence` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `persistence` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `persistence` редко встречается в одиночку. Он цепляется за constants,

strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `persistence` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `persistence` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `persistence` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 128. network behavior

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое general reverse engineering это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `network behavior` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `уязвимый parser` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `network behavior` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне `network behavior` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `network behavior` выглядит так: Главные соседи для `network behavior`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: запомнить слово, но не знать, где оно живет и как доказывается. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `network behavior` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `network behavior` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `network behavior` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `network behavior` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `network behavior` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `network behavior` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `network behavior` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `network behavior`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по

реверсу `network behavior` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `network behavior` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 129. rootkit concept

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Windows/PE эту потребность закрывает именно это понятие. `rootkit concept` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `rootkit concept` таковы: Ищи `rootkit concept` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `rootkit concept` обычно такая: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `rootkit concept` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `rootkit concept` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `rootkit concept` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `rootkit concept` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `rootkit concept` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `rootkit concept` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `rootkit concept` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого

абстракция становится рабочей. У сильного специалиста `rootkit concept` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `rootkit concept`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `rootkit concept` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 130. memory forensics

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое general reverse engineering это выглядит так: `memory forensics` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `сетевой лог` проявления находятся здесь: Ищи `memory forensics` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне `memory forensics` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи для `memory forensics`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: запомнить слово, но не знать, где оно живет и как доказывается. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `memory forensics` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `memory forensics` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `memory forensics`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `memory forensics` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `memory forensics` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `memory forensics` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `memory forensics` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `memory forensics` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser

может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `memory forensics` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `memory forensics` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `memory forensics` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `memory forensics` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `memory forensics` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `memory forensics` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `memory forensics` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `memory forensics` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 131. vulnerability

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Apple/Mach-O без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `vulnerability` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `vulnerability` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `vulnerability`: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `vulnerability` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `vulnerability` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `vulnerability` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `vulnerability`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `vulnerability` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `vulnerability` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `vulnerability` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `vulnerability` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная

инструкция. Если термин `vulnerability` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `vulnerability` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 132. disclosure

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя general reverse engineering. `disclosure` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `уязвимый parser` это понятие проявляется через места, которые можно указать пальцем: Ищи `disclosure` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `disclosure` устроен так: На низком уровне `disclosure` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `disclosure` такие: Главные соседи для `disclosure`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `disclosure` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `disclosure` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `disclosure` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `disclosure` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `disclosure` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `disclosure` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `disclosure` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское

раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `disclosure` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `disclosure` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `disclosure` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `disclosure` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `disclosure` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `disclosure` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `disclosure` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `disclosure` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `disclosure` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `disclosure` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 11

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: triage, sandbox, packer, anti-debug, crypto constants, protocol, persistence, network behavior, rootkit concept, memory forensics, vulnerability, disclosure. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 12. Том XII. Сквозные проекты и защита результатов

Последний том собирает базу в большие исследования с отчетами и проверками.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 133. PE parser

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Windows/PE. `PE parser` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `PE parser` можно не вообще где-то, а в конкретных следах: Ищи `PE parser` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `PE parser` не живет один. Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `PE parser` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `PE parser` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `PE parser` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `PE parser` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `PE parser` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `PE parser` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `PE parser` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register

roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `PE parser` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `PE parser` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `PE parser` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 134. unwind analyzer

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Windows/PE это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `unwind analyzer` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `лабораторный отчет` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `unwind analyzer` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `unwind analyzer` выглядит так: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `unwind analyzer` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `unwind analyzer` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `unwind analyzer` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `unwind analyzer` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `unwind analyzer` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `unwind analyzer` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `unwind analyzer` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент

показывает `unwind analyzer`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `unwind analyzer` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `unwind analyzer` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 135. ELF loader view

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Linux/ELF эту потребность закрывает именно это понятие. `ELF loader view` в Linux надо читать через разницу между файлом ELF и живым процессом. ELF хранит материал для загрузки, связывания и отладки, но поведение процесса появляется только после того, как kernel и dynamic linker построят отображения памяти. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `ELF loader view` таковы: Ищи `ELF loader view` в ELF header, program headers, dynamic section, relocation tables, GOT/PLT, `.eh_frame`, DWARF sections, `/proc/<pid>/maps`, strace/logs или core dump. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне kernel загружает loadable segments, dynamic linker разрешает зависимости, relocations меняют адреса, GOT становится таблицей фактических целей, PLT служит переходным механизмом, а syscalls переводят выполнение из user-mode в kernel boundary. Sections часто полезны для анализа, но loader живет прежде всего program headers. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: segment, section, relocation, symbol, dynamic linker, shared object, syscall, signal, CFI и process memory map. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `ELF loader view` обычно такая: принять section layout за точную карту памяти процесса или перепутать PLT-stub с настоящей library function. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `ELF loader view` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `ELF loader view` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `ELF loader view` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `ELF loader view` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `ELF loader view` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `ELF loader view` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `ELF loader view` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `ELF loader view` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function

- ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `ELF loader view`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `ELF loader view` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 136. shell command table

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое general reverse engineering это выглядит так: `shell command table` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта план развития проявления находятся здесь: Ищи `shell command table` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне `shell command table` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи для `shell command table`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: запомнить слово, но не знать, где оно живет и как доказывается. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `shell command table` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `shell command table` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `shell command table`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `shell command table` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `shell command table` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `shell command table` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `shell command table` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `shell command table` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace,

parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `shell command table` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `shell command table` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `shell command table` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `shell command table` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `shell command table` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `shell command table` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `shell command table` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `shell command table` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 137. vector recovery

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В embedded/bare metal без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `vector recovery` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `vector recovery` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `vector recovery`: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `vector recovery` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `vector recovery` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `vector recovery` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `vector recovery`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `vector recovery` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `vector recovery` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `vector recovery` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `vector recovery` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к

RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `vector recovery` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `vector recovery` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 138. STM32 GPIO

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя embedded/bare metal. STM32 GPIO в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте лабораторный отчет это понятие проявляется через места, которые можно указать пальцем: Ищи STM32 GPIO в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под STM32 GPIO устроен так: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у STM32 GPIO такие: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить STM32 GPIO в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на STM32 GPIO со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для STM32 GPIO особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают STM32 GPIO через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же STM32 GPIO часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера STM32 GPIO должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode,

между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда STM32 GPIO попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У STM32 GPIO почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина STM32 GPIO - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности STM32 GPIO часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять STM32 GPIO в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, STM32 GPIO помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. STM32 GPIO нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для STM32 GPIO важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке STM32 GPIO часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить STM32 GPIO как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР STM32 GPIO ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 139. UART protocol

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - embedded/bare metal. `UART protocol` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `UART protocol` можно не вообще где-то, а в конкретных следах: Ищи `UART protocol` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `UART protocol` не живет один. Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `UART protocol` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `UART protocol` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `UART protocol` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `UART protocol` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `UART protocol` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `UART protocol` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `UART protocol` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не

список терминов, а сеть, где одно наблюдение поддерживает другое. Если `UART protocol` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `UART protocol` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `UART protocol` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 140. RTOS tasks

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое embedded/bare metal это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `RTOS tasks` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `план развития` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `RTOS tasks` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `RTOS tasks` выглядит так: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `RTOS tasks` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `RTOS tasks` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `RTOS tasks` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `RTOS tasks` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `RTOS tasks` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `RTOS tasks` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `RTOS tasks` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `RTOS tasks`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state

или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `RTOS tasks` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `RTOS tasks` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 141. bootloader

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте embedded/bare metal эту потребность закрывает именно это понятие. `bootloader` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `bootloader` таковы: Ищи `bootloader` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `bootloader` обычно такая: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `bootloader` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `bootloader` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `bootloader` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `bootloader` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `bootloader` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `bootloader` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `bootloader` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `bootloader` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и

permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `bootloader`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `bootloader` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 142. embedded Linux rootfs

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое general reverse engineering это выглядит так: `embedded Linux rootfs` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `лабораторный отчет` проявления находятся здесь: Ищи `embedded Linux rootfs` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне `embedded Linux rootfs` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи для `embedded Linux rootfs`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: запомнить слово, но не знать, где оно живет и как доказывается. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `embedded Linux rootfs` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `embedded Linux rootfs` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `embedded Linux rootfs`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `embedded Linux rootfs` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `embedded Linux rootfs` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `embedded Linux rootfs` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `embedded Linux rootfs` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `embedded Linux rootfs` через свои модели. Ghidra может назвать функцию, IDA может построить graph,

debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `embedded Linux rootfs` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `embedded Linux rootfs` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `embedded Linux rootfs` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `embedded Linux rootfs` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `embedded Linux rootfs` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `embedded Linux rootfs` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `embedded Linux rootfs` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `embedded Linux rootfs` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 143. evidence table

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В general reverse engineering без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `evidence table` - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `evidence table` в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне `evidence table` должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `evidence table`: Главные соседи для `evidence table`: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `evidence table` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `evidence table` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `evidence table` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `evidence table`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `evidence table` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `evidence table` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `evidence table` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `evidence table` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `evidence table` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот

механизм вообще понадобился в устройстве программы. В долгой учебной траектории `evidence table` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 144. study plan

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя general reverse engineering. study plan - это не пароль в закрытый клуб, а рабочее имя для наблюдаемой роли. Чтобы термин стал понятным, нужно связать его с object, address, rule и verification. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте план развития это понятие проявляется через места, которые можно указать пальцем: Ищи study plan в текущем артефакте: file bytes, memory map, registers, function graph, data table, runtime event или documentation. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под study plan устроен так: На низком уровне study plan должен свестись к изменению состояния: какие bytes читаются, какой register меняется, какой memory address используется, какая branch выбирается, какой внешний объект затрагивается или какая table entry интерпретируется. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у study plan такие: Главные соседи для study plan: bytes, address, offset, type, function, control flow, data flow, tool output и source document. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: запомнить слово, но не знать, где оно живет и как доказывается. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить study plan в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на study plan со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для study plan особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают study plan через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же study plan часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера study plan должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда study plan попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское

раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `study plan` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `study plan` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `study plan` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `study plan` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `study plan` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `study plan` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `study plan` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `study plan` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `study plan` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `study plan` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 12

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: PE parser, unwind analyzer, ELF loader view, shell command table, vector recovery, STM32 GPIO, UART protocol, RTOS tasks, bootloader, embedded Linux rootfs, evidence table, study plan. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 13. Том XIII. Mach-O, Apple и мобильная среда

Этот том закрывает Apple-ветку: Mach-O, dyld, подписи, Objective-C, Swift и ограничения мобильной платформы.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 145. Mach-O header

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Apple/Mach-O. `Mach-O header` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `Mach-O header` можно не вообще где-то, а в конкретных следах: Ищи `Mach-O header` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `Mach-O header` не живет один. Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `Mach-O header` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `Mach-O header` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `Mach-O header` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `Mach-O header` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `Mach-O header` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `Mach-O header` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `Mach-O header` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно

наблюдение поддерживает другое. Если `Mach-O header` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `Mach-O header` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `Mach-O header` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 146. load command

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Apple/Mach-O это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `load command` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `iOS application bundle` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `load command` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `load command` выглядит так: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `load command` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `load command` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `load command` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `load command` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `load command` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `load command` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `load command` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `load command`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы

формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `load command` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `load command` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 147. segment/section

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Apple/Mach-O эту потребность закрывает именно это понятие. `segment/section` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `segment/section` таковы: Ищи `segment/section` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `segment/section` обычно такая: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `segment/section` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `segment/section` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `segment/section` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `segment/section` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `segment/section` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `segment/section` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `segment/section` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `segment/section` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack;

loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `segment/section`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `segment/section` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 148. dyld

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое Apple/Mach-O это выглядит так: `dyld` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `Apple binary или application bundle` проявления находятся здесь: Ищи `dyld` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `dyld` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `dyld` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `dyld`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `dyld` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `dyld` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `dyld` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `dyld` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `dyld` через свои модели. Ghidra может назвать функцию, IDA может построить graph,

debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `dyld` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `dyld` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `dyld` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `dyld` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `dyld` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `dyld` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `dyld` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `dyld` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 149. codesign

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Apple/Mach-O без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `codesign` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `codesign` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `codesign`: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `codesign` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `codesign` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `codesign` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `codesign`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `codesign` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `codesign` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `codesign` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `codesign` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `codesign` встречается в книге, рядом должен появиться

не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `codesign` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 150. entitlements

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя Apple/Mach-O. `entitlements` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `iOS application bundle` это понятие проявляется через места, которые можно указать пальцем: Ищи `entitlements` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `entitlements` устроен так: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `entitlements` такие: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `entitlements` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `entitlements` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `entitlements` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `entitlements` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `entitlements` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `entitlements` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между

bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `entitlements` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `entitlements` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `entitlements` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `entitlements` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `entitlements` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `entitlements` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `entitlements` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `entitlements` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `entitlements` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `entitlements` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `entitlements` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 151. Objective-C runtime

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Apple/Mach-O. `Objective-C runtime` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `Objective-C runtime` можно не вообще где-то, а в конкретных следах: Ищи `Objective-C runtime` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `Objective-C runtime` не живет один. Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, a bytecode без runtime. Главная ловушка: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `Objective-C runtime` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `Objective-C runtime` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, a hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `Objective-C runtime` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-a и начинается ответственность loader-a? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `Objective-C runtime` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `Objective-C runtime` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `Objective-C runtime` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `Objective-C runtime` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому

реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `Objective-C runtime` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `Objective-C runtime` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `Objective-C runtime` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 152. Swift metadata

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Apple/Mach-O это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `Swift metadata` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `Apple binary или application bundle` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `Swift metadata` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `Swift metadata` выглядит так: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `Swift metadata` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `Swift metadata` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `Swift metadata` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `Swift metadata` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `Swift metadata` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `Swift metadata` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `Swift metadata` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `Swift metadata`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы

формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `Swift metadata` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `Swift metadata` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 153. iOS sandbox

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Apple/Mach-O эту потребность закрывает именно это понятие. `iOS sandbox` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `iOS sandbox` таковы: Ищи `iOS sandbox` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `iOS sandbox` обычно такая: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `iOS sandbox` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `iOS sandbox` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `iOS sandbox` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `iOS sandbox` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `iOS sandbox` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `iOS sandbox` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `iOS sandbox` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `iOS sandbox` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode -

virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `iOS sandbox`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `iOS sandbox` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 154. IPA

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое Apple/Mach-O это выглядит так: `IPA` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `Apple binary или application bundle` проявления находятся здесь: Ищи `IPA` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `IPA` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `IPA` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `IPA`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `IPA` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `IPA` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `IPA` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `IPA` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `IPA` через свои модели. Ghidra может назвать функцию, IDA может построить graph,

debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же IPA часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера IPA должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда IPA попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У IPA почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина IPA - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности IPA часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять IPA в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, IPA помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 155. arm64e/PAC

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Apple/Mach-O без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `arm64e/PAC` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `arm64e/PAC` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `arm64e/PAC`: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `arm64e/PAC` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `arm64e/PAC` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `arm64e/PAC` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `arm64e/PAC`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `arm64e/PAC` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `arm64e/PAC` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `arm64e/PAC` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `arm64e/PAC` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `arm64e/PAC` встречается в

книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории arm64e/PAC нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 156. crash log

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя Apple/Mach-O. `crash log` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `Apple binary или application bundle` это понятие проявляется через места, которые можно указать пальцем: Ищи `crash log` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `crash log` устроен так: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `crash log` такие: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `crash log` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `crash log` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `crash log` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `crash log` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `crash log` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `crash log` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают

термин с остальной теорией. Когда `crash log` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `crash log` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `crash log` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `crash log` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `crash log` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `crash log` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `crash log` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `crash log` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `crash log` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `crash log` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `crash log` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 13

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: Mach-O header, load command, segment/section, dyld, codesign, entitlements, Objective-C runtime, Swift metadata, iOS sandbox, IPA, arm64e/PAC, crash log. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 14. Том XIV. Android, ART, JNI и мобильный Linux

Этот том связывает Android как Linux-систему, managed-runtime, native-библиотеки и мобильную лабораторию.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 157. APK

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Android. APK в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть APK можно не вообще где-то, а в конкретных следах: Ищи APK в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. APK не живет один. Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У APK есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда APK попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для APK полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин APK встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории APK нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя APK должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary APK редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если APK связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что

позволяют правила и наблюдения. В embedded и hardware-мире APK напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда APK объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 158. DEX

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Android это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. DEX в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `Android application` сначала ищут не красивую интерпретацию, а признаки существования: Ищи DEX в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг DEX выглядит так: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. DEX нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для DEX важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке DEX часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить DEX как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР DEX ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У DEX есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении DEX важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает DEX, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу DEX может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis -

защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `DEX` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 159. ART

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Android эту потребность закрывает именно это понятие. ART в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы ART таковы: Ищи ART в APK structure, AndroidManifest, DEX methods, native .so, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг ART обычно такая: анализировать APK как обычный ZIP или .so как весь Android behavior. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя ART должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary ART редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если ART связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире ART напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда ART объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для ART полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если ART кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста ART вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность ART, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия.

Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому ART здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 160. OAT/VDEX

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое Android это выглядит так: `OAT/VDEX` в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `Android application` проявления находятся здесь: Ищи `OAT/VDEX` в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `OAT/VDEX` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `OAT/VDEX` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `OAT/VDEX`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `OAT/VDEX` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `OAT/VDEX` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `OAT/VDEX` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `OAT/VDEX` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `OAT/VDEX` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через

допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же OAT/VDEX часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера OAT/VDEX должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда OAT/VDEX попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У OAT/VDEX почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина OAT/VDEX - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности OAT/VDEX часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять OAT/VDEX в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, OAT/VDEX помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 161. JNI

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Android без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. JNI в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи JNI в APK structure, AndroidManifest, DEX methods, native .so, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи JNI: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: анализировать APK как обычный ZIP или .so как весь Android behavior. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для JNI полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если JNI кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста JNI вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность JNI, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому JNI здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У JNI есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда JNI попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для JNI полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин JNI встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной

траектории `JNI` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 162. Binder IPC

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя Android. `Binder IPC` в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `Android application` это понятие проявляется через места, которые можно указать пальцем: Ищи `Binder IPC` в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `Binder IPC` устроен так: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `Binder IPC` такие: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `Binder IPC` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `Binder IPC` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `Binder IPC` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `Binder IPC` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `Binder IPC` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `Binder IPC` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `Binder IPC` попадает в отчет или объяснение, его нельзя оставлять

голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `Binder IPC` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `Binder IPC` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `Binder IPC` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `Binder IPC` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `Binder IPC` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `Binder IPC` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `Binder IPC` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `Binder IPC` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `Binder IPC` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `Binder IPC` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 163. SELinux

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - Android. SELinux в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть SELinux можно не вообще где-то, а в конкретных следах: Ищи SELinux в APK structure, AndroidManifest, DEX methods, native .so, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. SELinux не живет один. Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: анализировать APK как обычный ZIP или .so как весь Android behavior. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У SELinux есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда SELinux попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для SELinux полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин SELinux встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории SELinux нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя SELinux должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary SELinux редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если SELinux связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод

может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `SELinux` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `SELinux` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 164. A/B update

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Android это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `A/B update` в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `instrumented test device` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `A/B update` в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `A/B update` выглядит так: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `A/B update` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `A/B update` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `A/B update` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `A/B update` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `A/B update` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `A/B update` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `A/B update` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `A/B update`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `A/B update` может появляться под разным

углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `A/B update` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 165. boot image

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Apple/Mach-O эту потребность закрывает именно это понятие. `boot image` в Apple-среде живет между Mach-O format, dynamic loader `dyld`, code signing, Objective-C/Swift metadata и sandbox rules. Это слой, где структура файла тесно связана с политикой платформы. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `boot image` таковы: Ищи `boot image` в Mach-O header, load commands, segments, sections, symbol tables, Objective-C metadata, Swift descriptors, entitlement plist, crash log и контексте dyld shared cache. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Mach-O говорит loader-у, какие segments отобразить, какие libraries нужны, где находятся symbols и rebasing/binding information. На iOS добавляются подписи, entitlements и sandbox: бинарник анализируется не только как код, а как подписанный application bundle с политикой платформы. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: magic, CPU type, load command, segment, dylib dependency, code signature, sandbox, Objective-C selector, Swift type metadata и arm64e pointer authentication. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `boot image` обычно такая: переносить PE/ELF-модель без учета load commands, dyld shared cache и code signing. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `boot image` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `boot image` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `boot image` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `boot image` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `boot image` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `boot image` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `boot image` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `boot image` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual

machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `boot image`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `boot image` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 166. native library

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое Android это выглядит так: `native library` в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `Android native library` проявления находятся здесь: Ищи `native library` в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `native library` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `native library` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `native library`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `native library` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `native library` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `native library` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `native library` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `native library` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator

может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `native library` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `native library` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `native library` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `native library` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `native library` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `native library` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `native library` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `native library` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 167. Frida on Android

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Android без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `Frida on Android` в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `Frida on Android` в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `Frida on Android`: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `Frida on Android` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `Frida on Android` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `Frida on Android` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `Frida on Android`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `Frida on Android` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `Frida on Android` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `Frida on Android` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `Frida on Android` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `Frida on Android` встречается в книге, рядом должен появиться не только перевод, но и

маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `Frida on Android` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 168. logcat

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя Android. `logcat` в Android надо читать как часть системы, где APK содержит resources и DEX, ART исполняет managed code, JNI соединяет его с native libraries, а Binder связывает процессы. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `Android application` это понятие проявляется через места, которые можно указать пальцем: Ищи `logcat` в APK structure, AndroidManifest, DEX methods, native `.so`, ART/OAT/VDEX artifacts, Binder interfaces, SELinux denials, logcat output и device partitions. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `logcat` устроен так: На низком уровне Java/Kotlin method может закончиться native call через JNI, native library исполняется уже как ELF, а внешний эффект часто проходит через Binder service. Поэтому Android-reverse требует держать сразу несколько карт: managed code, native code, IPC, filesystem и platform policy. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `logcat` такие: Главные соседи: package, manifest, permission, DEX, ART, JNI boundary, Binder IPC, SELinux, native ABI и system image. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: анализировать APK как обычный ZIP или `.so` как весь Android behavior. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `logcat` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `logcat` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `logcat` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `logcat` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `logcat` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `logcat` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `logcat` попадает в отчет или объяснение, его нельзя оставлять голым английским

словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `logcat` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `logcat` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `logcat` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `logcat` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `logcat` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `logcat` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `logcat` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `logcat` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `logcat` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `logcat` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 14

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: APK, DEX, ART, OAT/VDEX, JNI, Binder IPC, SELinux, A/B update, boot image, native library, Frida on Android, logcat. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 15. Том XV. Managed runtimes: .NET, JVM, bytecode

Этот том объясняет, почему не весь реверс начинается с native assembly: иногда главный слой - байткод, метаданные и JIT.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 169. managed code

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - managed runtime. `managed code` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `managed code` можно не вообще где-то, а в конкретных следах: Ищи `managed code` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `managed code` не живет один. Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `managed code` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `managed code` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `managed code` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `managed code` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `managed code` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `managed code` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `managed code` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события.

Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `managed code` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `managed code` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `managed code` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 170. IL

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое managed runtime это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `IL` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `Java class file` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `IL` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `IL` выглядит так: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `IL` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `IL` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `IL` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `IL` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `IL` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `IL` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `IL` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `IL`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится

помощником. В разных книгах по реверсу `IL` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `IL` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 171. metadata token

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте managed runtime эту потребность закрывает именно это понятие. `metadata token` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `metadata token` таковы: Ищи `metadata token` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `metadata token` обычно такая: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `metadata token` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `metadata token` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `metadata token` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `metadata token` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `metadata token` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `metadata token` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `metadata token` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `metadata token` вызывает не одну ассоциацию, а

цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `metadata token`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `metadata token` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 172. assembly manifest

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое managed runtime это выглядит так: `assembly manifest` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `decompiler output` проявления находятся здесь: Ищи `assembly manifest` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `assembly manifest` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `assembly manifest` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `assembly manifest`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `assembly manifest` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `assembly manifest` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `assembly manifest` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `assembly manifest` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл

поведения часто появляется только после понимания runtime. Инструменты показывают `assembly manifest` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `assembly manifest` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `assembly manifest` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `assembly manifest` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `assembly manifest` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `assembly manifest` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `assembly manifest` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `assembly manifest` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `assembly manifest` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 173. .NET JIT

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В managed runtime без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `.NET JIT` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `.NET JIT` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `.NET JIT`: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `.NET JIT` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `.NET JIT` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `.NET JIT` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `.NET JIT`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `.NET JIT` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `.NET JIT` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `.NET JIT` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `.NET JIT` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная

инструкция. Если термин `.NET JIT` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `.NET JIT` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 174. GC

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя managed runtime. GC относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `Java class file` это понятие проявляется через места, которые можно указать пальцем: Ищи GC в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под GC устроен так: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у GC такие: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить GC в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на GC со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для GC особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают GC через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же GC часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера GC должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие

вопросы связывают термин с остальной теорией. Когда GC попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У GC почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина GC - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности GC часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять GC в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, GC помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. GC нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для GC важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке GC часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить GC как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР GC ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 175. Java class file

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - managed runtime. `Java class file` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `Java class file` можно не вообще где-то, а в конкретных следах: Ищи `Java class file` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `Java class file` не живет один. Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `Java class file` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `Java class file` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `Java class file` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `Java class file` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `Java class file` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `Java class file` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `Java class file` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries,

register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `Java class file` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `Java class file` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `Java class file` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 176. JVM operand stack

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое Windows/PE это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `JVM operand stack` в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `decompiler output` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `JVM operand stack` в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `JVM operand stack` выглядит так: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `JVM operand stack` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `JVM operand stack` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `JVM operand stack` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `JVM operand stack` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `JVM operand stack` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `JVM operand stack` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `JVM operand stack` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения.

Trace - наблюдение, но не вся программа. Если инструмент показывает `JVM operand stack`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `JVM operand stack` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `JVM operand stack` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 177. constant pool

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте managed runtime эту потребность закрывает именно это понятие. `constant pool` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `constant pool` таковы: Ищи `constant pool` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `constant pool` обычно такая: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `constant pool` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `constant pool` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `constant pool` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `constant pool` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `constant pool` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `constant pool` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `constant pool` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `constant pool` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function

- ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `constant pool`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `constant pool` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 178. verification

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое managed runtime это выглядит так: `verification` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `Java class file` проявления находятся здесь: Ищи `verification` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `verification` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `verification` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `verification`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `verification` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `verification` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `verification` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `verification` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто

появляется только после понимания runtime. Инструменты показывают `verification` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `verification` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `verification` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `verification` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `verification` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `verification` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `verification` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `verification` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `verification` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 179. reflection

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В managed runtime без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `reflection` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `reflection` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `reflection`: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `reflection` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `reflection` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `reflection` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `reflection`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `reflection` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `reflection` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `reflection` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `reflection` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница

часто объясняет больше, чем отдельная инструкция. Если термин `reflection` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `reflection` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 180. obfuscator

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя managed runtime. `obfuscator` относится к миру, где программа сначала описана не native-инструкциями CPU, а байткодом, metadata и runtime rules. CPU все равно исполняет машинный код, но важный смысл может жить уровнем выше - в metadata, type system, verifier или JIT. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `decompiler output` это понятие проявляется через места, которые можно указать пальцем: Ищи `obfuscator` в IL или JVM bytecode, metadata tables, constant pool, assembly manifest, class descriptors, method signatures, reflection calls, JIT output и runtime exceptions. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `obfuscator` устроен так: На низком уровне runtime проверяет или загружает metadata, строит типы, подготавливает method body, иногда компилирует hot code в native, перемещает objects через GC и скрывает многие детали платформы за правилами virtual machine. Поэтому decompiler может показать почти исходный код, но obfuscator и reflection способны перенести смысл в metadata и runtime decisions. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `obfuscator` такие: Главные соседи: bytecode, verifier, metadata token, type system, JIT, GC, reflection, exception model и native interop. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: думать, что managed binary проще native только потому, что decompiler output выглядит читаемым. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `obfuscator` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `obfuscator` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `obfuscator` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `obfuscator` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `obfuscator` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `obfuscator` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и

kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `obfuscator` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `obfuscator` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `obfuscator` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `obfuscator` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `obfuscator` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `obfuscator` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `obfuscator` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `obfuscator` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `obfuscator` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `obfuscator` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `obfuscator` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 15

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: managed code, IL, metadata token, assembly manifest, .NET JIT, GC, Java class file, JVM operand stack, constant pool, verification, reflection, obfuscator. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 16. Том XVI. Kernel, drivers, UEFI и boot chain

Этот том аккуратно вводит уровни ниже обычного процесса: firmware, загрузку ОС, kernel-mode и драйверные границы.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 181. UEFI

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - kernel/boot. `UEFI` относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `UEFI` можно не вообще где-то, а в конкретных следах: Ищи `UEFI` в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `UEFI` не живет один. Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: читать драйвер как обычную программу или забывать privilege boundary. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `UEFI` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `UEFI` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `UEFI` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `UEFI` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `UEFI` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `UEFI` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `UEFI` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое.

Если UEFI связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире UEFI напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда UEFI объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 182. DXE driver

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое kernel/boot это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `DXE driver` относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `UEFI image` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `DXE driver` в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `DXE driver` выглядит так: Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: читать драйвер как обычную программу или забывать privilege boundary. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `DXE driver` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `DXE driver` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `DXE driver` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `DXE driver` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `DXE driver` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `DXE driver` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `DXE driver` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `DXE driver`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры,

анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `DXE driver` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `DXE driver` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 183. PEI

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте Windows/PE эту потребность закрывает именно это понятие. PEI в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы PEI таковы: Ищи PEI в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг PEI обычно такая: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя PEI должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary PEI редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если PEI связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире PEI напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда PEI объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для PEI полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если PEI кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста PEI вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack;

loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность PEI, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому PEI здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 184. ACPI table

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое kernel/boot это выглядит так: `ACPI table` относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `UEFI image` проявления находятся здесь: Ищи `ACPI table` в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: читать драйвер как обычную программу или забывать privilege boundary. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `ACPI table` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `ACPI table` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `ACPI table`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `ACPI table` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `ACPI table` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `ACPI table` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `ACPI table` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime.

Инструменты показывают `ACPI table` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `ACPI table` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `ACPI table` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `ACPI table` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `ACPI table` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `ACPI table` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `ACPI table` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `ACPI table` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `ACPI table` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 185. kernel mode

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В kernel/boot без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `kernel mode` относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `kernel mode` в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `kernel mode`: Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: читать драйвер как обычную программу или забывать privilege boundary. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `kernel mode` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `kernel mode` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `kernel mode` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `kernel mode`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `kernel mode` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `kernel mode` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `kernel mode` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `kernel mode` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница

часто объясняет больше, чем отдельная инструкция. Если термин `kernel mode` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `kernel mode` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 186. driver object

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя kernel/boot. `driver object` относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `Windows driver` это понятие проявляется через места, которые можно указать пальцем: Ищи `driver object` в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `driver object` устроен так: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `driver object` такие: Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: читать драйвер как обычную программу или забывать privilege boundary. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `driver object` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `driver object` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `driver object` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `driver object` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `driver object` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `driver object` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где

проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `driver object` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `driver object` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `driver object` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `driver object` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `driver object` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `driver object` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `driver object` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `driver object` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `driver object` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `driver object` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `driver object` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 187. IRP

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - kernel/boot. IRP относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть IRP можно не вообще где-то, а в конкретных следах: Ищи IRP в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. IRP не живет один. Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: читать драйвер как обычную программу или забывать privilege boundary. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У IRP есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда IRP попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для IRP полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин IRP встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории IRP нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя IRP должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary IRP редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если IRP связан с

безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `IRP` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `IRP` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 188. IOCTL

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое kernel/boot это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `IOCTL` относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `Windows driver` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `IOCTL` в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `IOCTL` выглядит так: Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: читать драйвер как обычную программу или забывать privilege boundary. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `IOCTL` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `IOCTL` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `IOCTL` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `IOCTL` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `IOCTL` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `IOCTL` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `IOCTL` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `IOCTL`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает

быть авторитетом и становится помощником. В разных книгах по реверсу `IOCTL` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `IOCTL` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 189. Linux kernel module

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте kernel/boot эту потребность закрывает именно это понятие. `Linux kernel module` относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `Linux kernel module` таковы: Ищи `Linux kernel module` в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `Linux kernel module` обычно такая: читать драйвер как обычную программу или забывать privilege boundary. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `Linux kernel module` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `Linux kernel module` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `Linux kernel module` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `Linux kernel module` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `Linux kernel module` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `Linux kernel module` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `Linux kernel module` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста

`Linux kernel module` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `Linux kernel module`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `Linux kernel module` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 190. sysfs/debugfs

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое kernel/boot это выглядит так: `sysfs/debugfs` относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `Linux kernel module` проявления находятся здесь: Ищи `sysfs/debugfs` в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: читать драйвер как обычную программу или забывать privilege boundary. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `sysfs/debugfs` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `sysfs/debugfs` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `sysfs/debugfs`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `sysfs/debugfs` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `sysfs/debugfs` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `sysfs/debugfs` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `sysfs/debugfs` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто

появляется только после понимания runtime. Инструменты показывают `sysfs/debugfs` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `sysfs/debugfs` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `sysfs/debugfs` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `sysfs/debugfs` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `sysfs/debugfs` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `sysfs/debugfs` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `sysfs/debugfs` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `sysfs/debugfs` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `sysfs/debugfs` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 191. eBPF

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В kernel/boot без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. eBPF относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи eBPF в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи eBPF: Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: читать драйвер как обычную программу или забывать privilege boundary. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для eBPF полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если eBPF кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста eBPF вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность eBPF, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому eBPF здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У eBPF есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда eBPF попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для eBPF полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин eBPF встречается в книге,

рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории eBPF нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 192. Secure Boot

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя kernel/boot. `Secure Boot` относится к слоям, где код работает до обычного пользовательского процесса или ниже него. Здесь цена ошибки выше: другой privilege level, другой memory model, другой способ входа и другой набор доказательств. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `UEFI image` это понятие проявляется через места, которые можно указать пальцем: Ищи `Secure Boot` в UEFI images, boot logs, ACPI tables, driver entry points, dispatch tables, IOCTL handlers, kernel modules, sysfs/debugfs nodes, eBPF programs и secure boot policy. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `Secure Boot` устроен так: На низком уровне цепочка загрузки постепенно передает управление от firmware к bootloader и затем к kernel. Код драйвера не вызывается как обычная функция из `main`: его дергает kernel через зарегистрированные точки входа, структуры запросов и обратные вызовы. UEFI-образы и kernel-артефакты могут использовать знакомые форматы вроде PE/ELF, но контекст их исполнения не пользовательский режим. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `Secure Boot` такие: Главные соседи: privilege, firmware phase, boot service, runtime service, kernel object, device object, request packet, ioctl code, module init и trust chain. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: читать драйвер как обычную программу или забывать privilege boundary. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `Secure Boot` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `Secure Boot` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `Secure Boot` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `Secure Boot` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `Secure Boot` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `Secure Boot` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode,

между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `Secure Boot` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `Secure Boot` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `Secure Boot` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `Secure Boot` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `Secure Boot` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `Secure Boot` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `Secure Boot` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `Secure Boot` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `Secure Boot` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `Secure Boot` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `Secure Boot` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 16

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: UEFI, DXE driver, PEI, ACPI table, kernel mode, driver object, IRP, IOCTL, Linux kernel module, sysfs/debugfs, eBPF, Secure Boot. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 17. Том XVII. Защита, obfuscation и автоматизированный анализ

Этот том показывает защитные техники и инструменты анализа без превращения учебника в инструкцию по вредоносной эксплуатации.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Повторная связка 193. packer

`packer` уже разобран раньше как самостоятельное понятие. В этом томе он нужен не как новая глава, а как мост к контексту `Том XVII. Защита, obfuscation и автоматизированный анализ`: тот же термин переносится в другую среду, где меняются соседние понятия, источники правил и способы проверки.

Повторная связка 194. anti-debug

`anti-debug` уже разобран раньше как самостоятельное понятие. В этом томе он нужен не как новая глава, а как мост к контексту `Том XVII. Защита, obfuscation и автоматизированный анализ`: тот же термин переносится в другую среду, где меняются соседние понятия, источники правил и способы проверки.

Понятие 195. control-flow flattening

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте defensive analysis эту потребность закрывает именно это понятие. `control-flow flattening` - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `control-flow flattening` таковы: Ищи `control-flow flattening` в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `control-flow flattening` обычно такая: выдавать tool result за окончательный вывод или путать detection с explanation. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `control-flow flattening` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `control-flow flattening` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `control-flow flattening` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `control-flow flattening` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `control-flow flattening` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `control-flow flattening` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `control-flow flattening` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `control-flow flattening` вызывает не одну ассоциацию, а цепочку. Например,

address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `control-flow flattening`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `control-flow flattening` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 196. opaque predicate

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое defensive analysis это выглядит так: `opaque predicate` - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `coverage report` проявления находятся здесь: Ищи `opaque predicate` в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: выдавать tool result за окончательный вывод или путать detection с explanation. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `opaque predicate` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `opaque predicate` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `opaque predicate`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `opaque predicate` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `opaque predicate` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `opaque predicate` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `opaque predicate` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `opaque`

`predicate` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `opaque predicate` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `opaque predicate` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `opaque predicate` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `opaque predicate` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `opaque predicate` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `opaque predicate` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `opaque predicate` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `opaque predicate` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 197. string encryption

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В defensive analysis без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `string encryption` - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `string encryption` в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `string encryption`: Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: выдавать tool result за окончательный вывод или путать detection с explanation. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `string encryption` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `string encryption` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `string encryption` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `string encryption`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `string encryption` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `string encryption` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `string encryption` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `string encryption` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к

устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `string encryption` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `string encryption` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 198. VM-based obfuscation

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя defensive analysis. `VM-based obfuscation` - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `analysis rule` это понятие проявляется через места, которые можно указать пальцем: Ищи `VM-based obfuscation` в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `VM-based obfuscation` устроен так: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `VM-based obfuscation` такие: Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: выдавать tool result за окончательный вывод или путать detection с explanation. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `VM-based obfuscation` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `VM-based obfuscation` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `VM-based obfuscation` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `VM-based obfuscation` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `VM-based obfuscation` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `VM-based obfuscation` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между

файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда VM-based obfuscation попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У VM-based obfuscation почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина VM-based obfuscation - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности VM-based obfuscation часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять VM-based obfuscation в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, VM-based obfuscation помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. VM-based obfuscation нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для VM-based obfuscation важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке VM-based obfuscation часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить VM-based obfuscation как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР VM-based obfuscation ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 199. YARA

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - defensive analysis. YARA - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть YARA можно не вообще где-то, а в конкретных следах: Ищи YARA в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. YARA не живет один. Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: выдавать tool result за окончательный вывод или путать detection с explanation. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У YARA есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда YARA попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для YARA полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин YARA встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории YARA нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя YARA должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary YARA редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если YARA связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод

делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире YARA напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда YARA объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 200. capa

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое defensive analysis это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `capa` - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `defensive analysis lab sample` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `capa` в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `capa` выглядит так: Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: выдавать tool result за окончательный вывод или путать detection с explanation. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `capa` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `capa` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `capa` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `capa` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `capa` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `capa` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `capa` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `capa`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и

становится помощником. В разных книгах по реверсу `capa` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `capa` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 201. Frida

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте defensive analysis эту потребность закрывает именно это понятие. `Frida` - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `Frida` таковы: Ищи `Frida` в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `Frida` обычно такая: выдавать tool result за окончательный вывод или путать detection с explanation. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `Frida` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `Frida` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `Frida` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `Frida` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `Frida` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `Frida` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `Frida` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `Frida` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `Frida`, а вводить ее слоями. Сначала

смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `Frida` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 202. fuzzing

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое defensive analysis это выглядит так: `fuzzing` - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `defensive analysis lab sample` проявления находятся здесь: Ищи `fuzzing` в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: выдавать tool result за окончательный вывод или путать detection с explanation. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `fuzzing` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `fuzzing` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `fuzzing`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `fuzzing` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `fuzzing` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `fuzzing` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `fuzzing` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `fuzzing` через свои модели. Ghidra может назвать функцию, IDA может

построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `fuzzing` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `fuzzing` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `fuzzing` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `fuzzing` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `fuzzing` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `fuzzing` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `fuzzing` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `fuzzing` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 203. symbolic execution

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В defensive analysis без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. `symbolic execution` - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи `symbolic execution` в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи `symbolic execution`: Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: выдавать tool result за окончательный вывод или путать detection с explanation. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для `symbolic execution` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `symbolic execution` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `symbolic execution` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `symbolic execution`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `symbolic execution` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У `symbolic execution` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `symbolic execution` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `symbolic execution` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к

устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `symbolic execution` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `symbolic execution` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 204. taint analysis

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя defensive analysis. `taint analysis` - это инструмент или техника анализа защищенного/запутанного поведения. В учебнике это рассматривается defensive: цель - понять и описать behavior, а не научить злоупотреблению. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `defensive analysis lab sample` это понятие проявляется через места, которые можно указать пальцем: Ищи `taint analysis` в entropy changes, unusual control flow, API traces, instrumentation hooks, YARA/capa matches, fuzzing crashes, symbolic states, taint propagation и anti-analysis checks. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `taint analysis` устроен так: На низком уровне защитные техники ломают привычную прямую дорогу от bytes к псевдокоду, похожему на исходный текст. Packer может создавать code at runtime, obfuscation скрывает control/data flow, anti-debug меняет поведение под наблюдением, fuzzing перебирает входы, symbolic execution рассуждает о путях, а taint analysis отслеживает влияние данных. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `taint analysis` такие: Главные соседи: triage, hypothesis, instrumentation, coverage, rule match, behavior evidence, limitation и report discipline. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: выдавать tool result за окончательный вывод или путать detection с explanation. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `taint analysis` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `taint analysis` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `taint analysis` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `taint analysis` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `taint analysis` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `taint analysis` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между

user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `taint analysis` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `taint analysis` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `taint analysis` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `taint analysis` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `taint analysis` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `taint analysis` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `taint analysis` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `taint analysis` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `taint analysis` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `taint analysis` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `taint analysis` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 17

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: packer, anti-debug, control-flow flattening, opaque predicate, string encryption, VM-based obfuscation, YARA, capa, Frida, fuzzing, symbolic execution, taint analysis. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Том 18. Том XVIII. Hardware security и полевые системы

Этот том расширяет embedded-реверс до физического мира: сигналы, память, шины, питание, отладочные порты и доверенная загрузка.

Главная мысль тома: фундамент появляется не из списка слов, а из связей. Ниже каждое понятие раскрывается через слой существования, низкий механизм и соседние понятия.

Понятие 205. side channel

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - hardware security. `side channel` переносит реверс из файла в физический мир. Здесь доказательство может быть не только инструкцией, но и сигналом на линии, формой импульса, дампом памяти, реакцией платы или записью логического анализатора. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `side channel` можно не вообще где-то, а в конкретных следах: Ищи `side channel` в board photos, pinout, datasheets, bus traces, SPI flash dumps, JTAG/SWD configuration, oscilloscope captures, boot ROM behavior, TrustZone transitions и secure element communication. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне устройство - не абстрактная программа. Питание, reset, clock, timing на шине и protection fuses определяют, что вообще можно наблюдать. Side-channel видит косвенную утечку, fault injection пытается нарушить вычисление, а hardware trace требует калибровки до интерпретации. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `side channel` не живет один. Главные соседи: voltage, clock, reset, bus protocol, timing, flash layout, debug port, threat model, safety и repeatability. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: считать единичный trace доказательством или игнорировать риск повредить устройство. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `side channel` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `side channel` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `side channel` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `side channel` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `side channel` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `side channel` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `side channel` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `side channel` связан с

безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `side channel` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `side channel` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 206. fault injection

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое hardware security это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `fault injection` переносит реверс из файла в физический мир. Здесь доказательство может быть не только инструкцией, но и сигналом на линии, формой импульса, дампом памяти, реакцией платы или записью логического анализатора. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `hardware target` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `fault injection` в board photos, pinout, datasheets, bus traces, SPI flash dumps, JTAG/SWD configuration, oscilloscope captures, boot ROM behavior, TrustZone transitions и secure element communication. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне устройство - не абстрактная программа. Питание, reset, clock, timing на шине и protection fuses определяют, что вообще можно наблюдать. Side-channel видит косвенную утечку, fault injection пытается нарушить вычисление, а hardware trace требует калибровки до интерпретации. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `fault injection` выглядит так: Главные соседи: voltage, clock, reset, bus protocol, timing, flash layout, debug port, threat model, safety и repeatability. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: считать единичный trace доказательством или игнорировать риск повредить устройство. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `fault injection` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `fault injection` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `fault injection` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `fault injection` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `fault injection` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `fault injection` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `fault injection` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `fault injection`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа

control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `fault injection` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `fault injection` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 207. glitching

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте hardware security эту потребность закрывает именно это понятие. `glitching` переносит реверс из файла в физический мир. Здесь доказательство может быть не только инструкцией, но и сигналом на линии, формой импульса, дампом памяти, реакцией платы или записью логического анализатора. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `glitching` таковы: Ищи `glitching` в board photos, pinout, datasheets, bus traces, SPI flash dumps, JTAG/SWD configuration, oscilloscope captures, boot ROM behavior, TrustZone transitions и secure element communication. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне устройство - не абстрактная программа. Питание, reset, clock, timing на шине и protection fuses определяют, что вообще можно наблюдать. Side-channel видит косвенную утечку, fault injection пытается нарушить вычисление, а hardware trace требует калибровки до интерпретации. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: voltage, clock, reset, bus protocol, timing, flash layout, debug port, threat model, safety и repeatability. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `glitching` обычно такая: считать единичный trace доказательством или игнорировать риск повредить устройство. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `glitching` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `glitching` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `glitching` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `glitching` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `glitching` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `glitching` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `glitching` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `glitching` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть

фундамент. В книге для новичка важно не прятать сложность `glitching`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `glitching` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Повторная связка 208. logic analyzer

`logic analyzer` уже разобран раньше как самостоятельное понятие. В этом томе он нужен не как новая глава, а как мост к контексту `Том XVIII. Hardware security и полевые системы`: тот же термин переносится в другую среду, где меняются соседние понятия, источники правил и способы проверки.

Понятие 209. oscilloscope

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В Windows/PE без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. oscilloscope в Windows-мире надо читать как часть договора между файлом PE, загрузчиком Windows, компоновщиком и отладчиком. Это не просто поле в hex editor: поле получает смысл только после того, как loader построил image, применил relocations, подготовил imports и права доступа к страницам памяти. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи oscilloscope в заголовках PE, section table, data directories, import/export tables, exception directory, символах PDB или в памяти процесса. Один и тот же байт может иметь file offset на диске и другой RVA в загруженном образе. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне Windows не исполняет EXE как плоский файл. Loader читает headers, создает memory mappings, задает page protections, исправляет адреса через relocation records, заполняет таблицы imports и передает управление entry point. Если речь об unwind, debugger и exception dispatcher используют metadata, а не угадывают стек по красивой форме кода. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи oscilloscope: Главные соседи: file offset, RVA, VA, ImageBase, SectionAlignment, loader, x64 ABI, exception handling и debugger symbols. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: прочитать число из файла как виртуальный адрес или назвать import полным поведением программы. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для oscilloscope полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если oscilloscope кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста oscilloscope вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность oscilloscope, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому oscilloscope здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У oscilloscope есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда oscilloscope попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для oscilloscope полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode

переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `oscilloscope` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `oscilloscope` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 210. SPI Flash

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя embedded/bare metal. `SPI Flash` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `hardware target` это понятие проявляется через места, которые можно указать пальцем: Ищи `SPI Flash` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `SPI Flash` устроен так: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `SPI Flash` такие: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `SPI Flash` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `SPI Flash` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `SPI Flash` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `SPI Flash` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `SPI Flash` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `SPI Flash` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode,

между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда SPI Flash попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У SPI Flash почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина SPI Flash - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности SPI Flash часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять SPI Flash в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, SPI Flash помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. SPI Flash нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для SPI Flash важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке SPI Flash часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить SPI Flash как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР SPI Flash ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Понятие 211. JTAG boundary scan

Представь, что перед нами не слово из документации, а маленький мост между человеком и машиной. На одной стороне моста находятся bytes, registers, memory pages, tables или hardware signals. На другой стороне мы хотим получить человеческую фразу: что это значит и зачем оно нужно. Контекст здесь - embedded/bare metal. `JTAG boundary scan` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Это объяснение важно проговаривать полностью, потому что короткий перевод часто обманывает. Новичок запоминает слово, но не понимает, какую работу оно выполняет в системе. Увидеть `JTAG boundary scan` можно не вообще где-то, а в конкретных следах: Ищи `JTAG boundary scan` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Эти следы связывают термин с материальной основой. Без материальной основы термин превращается в впечатление. Ниже интерфейса действует такой механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Здесь нет магии. Есть правила формата, процессора, runtime, операционной системы или устройства. Реверсер читает эти правила и смотрит, как они оставили след в объекте. `JTAG boundary scan` не живет один. Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Если одно из этих соседних понятий понято поверхностно, то и само объяснение останется тонким. Например, нельзя понимать stack без ABI, RVA без image base, MMIO без reference manual, а bytecode без runtime. Главная ловушка: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Чтобы не попасть в нее, полезно каждый раз разделять четыре вещи: наблюдение, правило, вывод и границу вывода. Это теория доказательности, а не практическое упражнение. У `JTAG boundary scan` есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда `JTAG boundary scan` попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для `JTAG boundary scan` полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин `JTAG boundary scan` встречается в книге, рядом должен появиться не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `JTAG boundary scan` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления. С точки зрения читателя `JTAG boundary scan` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `JTAG boundary scan` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles,

metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `JTAG boundary scan` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `JTAG boundary scan` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `JTAG boundary scan` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим.

Понятие 212. CAN bus

Такое понятие появляется потому, что сложная система должна как-то согласовать разные части: compiler, linker, loader, runtime, CPU, OS, device или debugger. В слое hardware security это особенно заметно: одно и то же число может быть бессмысленным без договора, который говорит, как его читать. `CAN bus` переносит реверс из файла в физический мир. Здесь доказательство может быть не только инструкцией, но и сигналом на линии, формой импульса, дампом памяти, реакцией платы или записью логического анализатора. Слово становится понятным, когда мы знаем его назначение. Оно либо описывает место, либо переход, либо таблицу, либо правило вызова, либо механизм восстановления, либо границу между слоями. Для `hardware target` сначала ищут не красивую интерпретацию, а признаки существования: Ищи `CAN bus` в board photos, pinout, datasheets, bus traces, SPI flash dumps, JTAG/SWD configuration, oscilloscope captures, boot ROM behavior, TrustZone transitions и secure element communication. Такие признаки не обязаны сразу доказывать смысл. Они дают точку, от которой можно строить объяснение. Дальше вступает внутреннее устройство: На низком уровне устройство - не абстрактная программа. Питание, reset, clock, timing на шине и protection fuses определяют, что вообще можно наблюдать. Side-channel видит косвенную утечку, fault injection пытается нарушить вычисление, а hardware trace требует калибровки до интерпретации. Если объяснить это совсем просто, система берет одно состояние и превращает его в другое. Реверсер пытается понять, какое правило стояло между ними. Сеть вокруг `CAN bus` выглядит так: Главные соседи: voltage, clock, reset, bus protocol, timing, flash layout, debug port, threat model, safety и repeatability. Эта сеть защищает от наивного чтения. Нельзя вытащить один термин из системы и ожидать, что он сохранит полный смысл. Типичный сбой понимания: считать единичный trace доказательством или игнорировать риск повредить устройство. Он возникает, когда человек берет верхнее имя, но не проверяет нижний механизм. В хорошей книге термин должен быть не ярлыком, а дверью к механизму. `CAN bus` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `CAN bus` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `CAN bus` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `CAN bus` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `CAN bus` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите. У `CAN bus` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `CAN bus` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `CAN bus`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и

становится помощником. В разных книгах по реверсу `CAN bus` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `CAN bus` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу.

Понятие 213. radio firmware

Если объяснять это понятие человеку без IT-базы, я бы начал не с английского слова, а с потребности. В системе есть задача, которую надо решить: найти место, передать управление, связать модули, восстановить состояние, описать устройство или разделить уровни ответственности. В контексте embedded/bare metal эту потребность закрывает именно это понятие. `radio firmware` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Поэтому термин нельзя оставлять без окружающей истории. Он должен отвечать на вопрос 'для чего это существует', а не только 'как это переводится'. Видимые следы `radio firmware` таковы: Ищи `radio firmware` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Когда след найден, нельзя сразу объявлять его окончательным смыслом. След похож на улику: он важен, но требует правила чтения. Правило чтения опирается на механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. В этом месте соединяются документация и объект. Документация говорит, как должно работать правило, а объект показывает, что именно лежит перед нами. Связанные узлы: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Их нужно проговаривать рядом, потому что новичок часто учит термины отдельно. Отдельные термины быстро забываются, а связи превращаются в фундамент. Ошибка вокруг `radio firmware` обычно такая: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Исправляет ее не уверенный тон, а аккуратная причинная цепочка: что наблюдаем, по какому правилу читаем, какой вывод делаем, что пока не знаем. С точки зрения читателя `radio firmware` должен отвечать на вопрос 'что я теперь вижу, чего раньше не видел?'. Хороший термин не увеличивает туман, а делит сложность на понятные части: место, правило, действие, граница, следствие. Внутри большого binary `radio firmware` редко встречается в одиночку. Он цепляется за constants, strings, xrefs, table entries, register roles, metadata и события. Поэтому реверсер строит не список терминов, а сеть, где одно наблюдение поддерживает другое. Если `radio firmware` связан с безопасностью, нужно быть особенно аккуратным. Слишком широкий вывод делает анализ красивым, но слабым. Слишком узкий вывод может потерять смысл. Фундаментальная теория учит держать масштаб: говорить ровно то, что позволяют правила и наблюдения. В embedded и hardware-мире `radio firmware` напоминает, что программа не живет в пустоте. Адрес может оказаться register устройства, задержка может быть частью протокола, interrupt может изменить buffer между двумя инструкциями, а питание и clock могут влиять на наблюдаемость. Когда `radio firmware` объяснен живым языком, читатель должен уметь пересказать его без заучивания: 'это нужно для того-то, живет там-то, связано с этим, ломается при такой ошибке'. Если пересказ невозможен, значит объяснение было слишком сухим. Для `radio firmware` полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если `radio firmware` кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста `radio firmware` вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и

permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность `radio firmware`, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому `radio firmware` здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента.

Понятие 214. boot ROM

Это понятие можно рассматривать как часть биографии бинарника. Сначала был исходный замысел автора или схема устройства. Потом compiler, assembler, linker, loader, runtime или hardware превратили этот замысел в машинные следы. Реверсер читает следы в обратную сторону. В слое embedded/bare metal это выглядит так: `boot ROM` в embedded - это часть связи между прошивкой, memory map и физическим устройством. Здесь нет привычного процесса с loader-ом ОС: есть Flash, RAM, периферия, interrupt vectors и startup code. Даже если термин кажется знакомым, его смысл меняется от среды. Слово может быть тем же, а правила вокруг него другими. Для объекта `logic trace` проявления находятся здесь: Ищи `boot ROM` в raw firmware, vector table, linker script, startup code, device reference manual, SVD-файле, MMIO addresses, interrupt handlers, logic analyzer trace или OpenOCD session. Это место, где теория касается материала. Пока нет такого места, термин остается общим разговором. Внутри работает следующий механизм: На низком уровне CPU берет начальные значения из vector table, переходит в reset handler, runtime копирует `.data` и очищает `.bss`, затем firmware настраивает clocks и peripheral registers. Запись по адресу MMIO может не просто менять число в памяти, а включать тактирование, запускать DMA, очищать interrupt flag или менять состояние ножки микроконтроллера. Не обязательно знать все детали сразу. Но нужно понимать направление: кто читает данные, кто меняет состояние, кто потом использует результат. Вокруг стоят связанные понятия: Главные соседи: base address, Flash/SRAM ranges, reset handler, interrupt, peripheral register, bit field, volatile, DMA и clock tree. Они образуют контекст. Например, loader без file format неполон, ABI без stack и registers непонятен, interrupt без vector table и handler теряет смысл. Слабое место обычно такое: читать MMIO как RAM, забывать clock enable или считать нечетный Thumb-address ошибкой. Хороший фундамент учит не угадывать, а восстанавливать происхождение следа. У `boot ROM` есть человеческая причина существования: большие программы и устройства слишком сложны, чтобы все держалось на одном уровне. Нужны договоры, таблицы, адреса, режимы, обработчики, metadata и форматы. Термин называет один из таких элементов порядка. При чтении `boot ROM` важно не спутать причину и симптом. Высокая entropy - симптом, но не причина. Import - след зависимости, но не полное поведение. Metadata - описание, но не всегда истина исполнения. Trace - наблюдение, но не вся программа. Если инструмент показывает `boot ROM`, это начало разговора. Нужно спросить, откуда инструмент взял уверенность: из сигнатуры, анализа control flow, таблицы формата, debug info, emulator state или ручной разметки. Так tool перестает быть авторитетом и становится помощником. В разных книгах по реверсу `boot ROM` может появляться под разным углом. Системные книги объясняют слой ОС, binary analysis - поток данных и форматы, malware analysis - защитную лабораторию, hardware hacking - физическую среду. Здесь эти углы сведены в один фундамент. Когда фундамент построен, `boot ROM` перестает быть отдельным фактом. Он становится частью внутренней карты: от bytes к structures, от structures к state, от state к behavior, от behavior к доказанному выводу. Если посмотреть на `boot ROM` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `boot ROM` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `boot ROM` через свои

модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `boot ROM` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `boot ROM` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `boot ROM` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `boot ROM` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `boot ROM` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `boot ROM` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `boot ROM` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `boot ROM` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев.

Понятие 215. TrustZone

Иногда термин сначала кажется сухим, а потом оказывается несущей балкой. В hardware security без него трудно объяснить, почему байты, адреса или события ведут себя именно так, а не иначе. TrustZone переносит реверс из файла в физический мир. Здесь доказательство может быть не только инструкцией, но и сигналом на линии, формой импульса, дампом памяти, реакцией платы или записью логического анализатора. Важно, что это не школьное определение. Это объяснение роли. Роль всегда связана с другими ролями, поэтому термин нельзя вырывать из окружающего механизма. Материальная сторона понятия видна так: Ищи TrustZone в board photos, pinout, datasheets, bus traces, SPI flash dumps, JTAG/SWD configuration, oscilloscope captures, boot ROM behavior, TrustZone transitions и secure element communication. С этого начинается инженерная честность: не 'мне кажется', а 'я вижу такой след в таком слое'. Причинный механизм: На низком уровне устройство - не абстрактная программа. Питание, reset, clock, timing на шине и protection fuses определяют, что вообще можно наблюдать. Side-channel видит косвенную утечку, fault injection пытается нарушить вычисление, а hardware trace требует калибровки до интерпретации. Если выразить это коротко, низкий уровень отвечает на вопрос 'что физически или логически произошло с состоянием машины'. Соседи TrustZone: Главные соседи: voltage, clock, reset, bus protocol, timing, flash layout, debug port, threat model, safety и repeatability. Они работают как страховочная сетка. Чем лучше видны связи, тем меньше шанс спутать похожие вещи: offset и address, section и segment, bytecode и native code, обычную RAM и MMIO. Обычная ошибка: считать единичный trace доказательством или игнорировать риск повредить устройство. Поэтому объяснение должно быть длиннее одного предложения. Новичку нужно не определение, а маршрут от слова к механизму. Для TrustZone полезно держать в голове две перспективы. Первая - перспектива машины: какие данные читаются и какое состояние меняется. Вторая - перспектива исследователя: какое правило позволяет назвать это изменение осмысленным. Реверс начинается там, где эти перспективы соединяются. Если TrustZone кажется слишком абстрактным, его надо приземлить. Найти слой: file, memory, CPU, OS, runtime, firmware, device. Найти носитель: bytes, register, table, metadata, signal. Найти правило: ABI, ISA, format, protocol, manual. После этого абстракция становится рабочей. У сильного специалиста TrustZone вызывает не одну ассоциацию, а цепочку. Например, address тянет за собой memory map; function - ABI и stack; loader - relocations и permissions; interrupt - vector table и context; bytecode - virtual machine и verifier. Такая цепочка и есть фундамент. В книге для новичка важно не прятать сложность TrustZone, а вводить ее слоями. Сначала смысл, потом механизм, потом соседние понятия, потом исключения и платформенные различия. Если сразу бросить все детали, читатель утонет; если оставить только определение, он ничему не научится. Поэтому TrustZone здесь объясняется как часть общей системы reverse engineering, а не как словарная строка. Цель - чтобы после чтения человек мог открыть новый объект и понимать, какие вопросы задавать до запуска инструмента. У TrustZone есть еще одна сторона: его можно рассматривать как след решения, принятого до запуска программы. Где-то это решение принял compiler, где-то linker, где-то автор формата, где-то разработчик микроконтроллера. Поэтому реверсер спрашивает не только 'что это', но и 'на каком этапе это появилось'. Когда TrustZone попадает в memory image, его смысл может измениться по сравнению с file view. Loader мог применить relocations, dynamic linker мог заполнить таблицу, runtime мог подготовить metadata, а hardware мог изменить register asynchronously. Поэтому теория всегда разводит файл, образ памяти и живое исполнение. Для TrustZone полезна привычка смотреть на границы. Где заканчивается ответственность compiler-а и начинается ответственность loader-а? Где user-mode переходит в kernel-mode? Где bytecode превращается в native code? Где CPU обращается уже не к RAM, а к устройству? Граница часто объясняет больше, чем отдельная инструкция. Если термин TrustZone встречается в книге, рядом должен появиться

не только перевод, но и маленькая причинная история. Без истории читатель запоминает звук. С историей он понимает, почему этот механизм вообще понадобился в устройстве программы. В долгой учебной траектории `TrustZone` нужно возвращать несколько раз. Сначала на уровне интуиции, затем на уровне документации, потом на уровне disassembly, затем на уровне runtime и, наконец, на уровне отчета. Так понятие перестает быть чужим словом и становится частью мышления.

Понятие 216. secure element

Здесь лучше всего начать не с отдельного перевода, а с рабочей роли понятия внутри слоя hardware security. `secure element` переносит реверс из файла в физический мир. Здесь доказательство может быть не только инструкцией, но и сигналом на линии, формой импульса, дампом памяти, реакцией платы или записью логического анализатора. Если убрать это понятие из картины, рядом сразу расползутся другие слова: адрес перестанет иметь карту, функция потеряет договор вызова, а байты снова станут просто непонятной массой. В реальном объекте `hardware target` это понятие проявляется через места, которые можно указать пальцем: Ищи `secure element` в board photos, pinout, datasheets, bus traces, SPI flash dumps, JTAG/SWD configuration, oscilloscope captures, boot ROM behavior, TrustZone transitions и secure element communication. Здесь важно не то, что инструмент показал красивое имя, а то, что у наблюдения есть координата и правило чтения. Механизм под `secure element` устроен так: На низком уровне устройство - не абстрактная программа. Питание, reset, clock, timing на шине и protection fuses определяют, что вообще можно наблюдать. Side-channel видит косвенную утечку, fault injection пытается нарушить вычисление, а hardware trace требует калибровки до интерпретации. На этом уровне полезно думать причинно. Есть исходный материал, есть правило, есть изменение состояния, и только затем появляется человеческий смысл. Соседние понятия у `secure element` такие: Главные соседи: voltage, clock, reset, bus protocol, timing, flash layout, debug port, threat model, safety и repeatability. Они нужны не для словаря, а для защиты от ложных выводов. Стоит перепутать один соседний слой, и весь вывод начинает выглядеть правдоподобно, но ехать по неверной карте. Чаще всего ошибаются так: считать единичный trace доказательством или игнорировать риск повредить устройство. Поэтому сильное объяснение всегда отвечает на три вопроса: кто создает это явление, кто им пользуется и по какому правилу мы имеем право его читать. Если переносить `secure element` в другой мир, нужно переносить не буквы, а роль. В Windows рядом могут оказаться PE и x64 ABI, в Linux - ELF и dynamic linker, в embedded - memory map и reference manual, в managed runtime - metadata и virtual machine. Так термин становится частью единой системы. Если посмотреть на `secure element` со стороны toolchain, то есть цепочки compiler - assembler - linker - loader, становится видно, что многие низкоуровневые явления не написаны автором программы напрямую. Они появляются как следствие сборки. Compiler выбирает инструкции и registers, linker раскладывает sections и symbols, loader строит image, runtime готовит язык к выполнению. Поэтому термин нужно связывать не только с исходным кодом, но и с тем этапом, который мог его породить. Для `secure element` особенно важно различать статическую и динамическую картину. Static view показывает файл, таблицы, disassembly, metadata и предполагаемые связи. Dynamic view показывает живой процесс, registers, memory mappings, events, syscalls, exceptions или hardware behavior. Иногда static view достаточен для структурного вывода, но смысл поведения часто появляется только после понимания runtime. Инструменты показывают `secure element` через свои модели. Ghidra может назвать функцию, IDA может построить graph, debugger может показать register, emulator может дать trace, parser может вывести field. Но каждый tool видит объект через допущения. Поэтому фундаментальный подход требует отделять правило платформы от способа, которым инструмент его отобразил. В Windows, Linux и embedded один и тот же `secure element` часто меняет форму, но сохраняет глубинную роль. В Windows многое идет через PE, x64 ABI, SEH и loader. В Linux - через ELF, dynamic linker, syscalls и signals. В embedded - через startup, vector table, MMIO, interrupts, DMA и datasheet. Понимание становится прочным, когда ты видишь эту общую роль за разными оболочками. Для начинающего реверсера `secure element` должен стать не словом для узнавания, а точкой входа в вопрос. Что здесь является данными? Кто задает правило? Кто исполняет механизм? Где проходит граница между файлом и памятью, между user-mode и kernel-mode, между CPU и

устройством, между bytecode и native code? Такие вопросы связывают термин с остальной теорией. Когда `secure element` попадает в отчет или объяснение, его нельзя оставлять голым английским словом. Рядом должно быть русское раскрытие: что это, зачем существует, в каком слое живет, какие соседние понятия нужны для понимания, что произойдет при неверном чтении. Иначе текст выглядит профессионально только внешне, но не строит фундамент у читателя. У `secure element` почти всегда есть источник нормы: спецификация формата, ABI, ISA manual, документация runtime, reference manual устройства или описание протокола. Это принципиально: реверсер не придумывает правила заново, он восстанавливает, какое правило применимо к конкретному объекту. Если правило неизвестно, вывод должен оставаться гипотезой. Еще одна глубина `secure element` - границы применимости. Одно и то же слово в разных архитектурах может означать похожую роль, но разные детали. Stack на Windows x64, System V AMD64 и Cortex-M exception entry устроен не одинаково. Loader в PE, ELF, Mach-O и bare-metal startup решает похожую задачу оживления кода, но делает это разными средствами. Фундамент строится на понимании роли и различий одновременно. В безопасности `secure element` часто становится местом ошибки. Неверный размер, неверный signedness, ложный pointer, испорченная relocation, неправильный unwind, неверная граница buffer, ошибочный MMIO bit, неправильная модель syscall или доверие к metadata могут привести к неправильному анализу или к реальной уязвимости. Поэтому теория реверса тесно связана с secure coding, vulnerability research и forensic thinking. Если объяснять `secure element` в большой книге, полезно постоянно возвращать читателя к одной линии: байты становятся структурами, структуры становятся состоянием, состояние становится поведением, поведение становится выводом. На каждом переходе можно ошибиться. На каждом переходе нужен свой источник правила. Это и есть сквозной фундамент reverse engineering. Наконец, `secure element` помогает связать твою текущую Windows-ВКР с будущим embedded-направлением. В ВКР ты учишься видеть PE, RVA, x64 ABI, unwind и доказательные признаки. В embedded те же привычки превращаются в чтение base address, vector table, MMIO, interrupts, DMA и bootloader layout. Меняются объекты, но остается дисциплина слоев. `secure element` нельзя честно объяснить без вопроса о доверии к источнику. Если правило взято из официальной спецификации, оно сильнее догадки. Если правило восстановлено из наблюдений, оно требует осторожности. Если его подсказал tool, нужно понимать, на каких эвристиках tool построил вывод. Для `secure element` важно различать нормальное, необычное и подозрительное. Нормальное соответствует ABI, format, runtime или datasheet. Необычное может быть следствием optimization, packing, custom toolchain или особенности платформы. Подозрительное начинается там, где необычность связана с поведением, границей доверия или сокрытием смысла. В Windows-ветке `secure element` часто упирается в PE, loader, x64 ABI, SEH, handles или Native API. В Linux-ветке - в ELF, dynamic linker, syscalls, signals и process mappings. В embedded-ветке - в startup, vector table, MMIO и interrupts. Одна роль, три разные грамматики. Типичная ошибка обучения - выучить `secure element` как карточку. Карточка быстро забывается, потому что не зацеплена за соседние механизмы. Глубокое понимание появляется, когда слово связывается с путем данных и путем управления. Для твоей ВКР `secure element` ценен еще и как часть доказательной культуры. Проект про unwind не просто парсит bytes; он показывает, как структура формата, ABI и metadata превращаются в признаки, которые можно обсуждать на защите.

Системная связка тома 18

Понятия этого тома нельзя читать отдельно. Они образуют одну модель: side channel, fault injection, glitching, logic analyzer, oscilloscope, SPI Flash, JTAG boundary scan, CAN bus, radio firmware, boot ROM, TrustZone, secure element. Хорошее понимание появляется, когда читатель может объяснить, какой слой задает правило, какой механизм исполняет это правило и какой соседний термин меняет интерпретацию. Так теория становится широкой, но не распадается на словарь.

Карта терминов: как английские слова связываются в одну систему

Этот раздел нужен, чтобы читатель не утонул в английских словах. В реверсе почти все термины пришли из документации, ABI, ISA, toolchain и operating systems. Их нельзя просто перевести один к одному. Нужно понять роль слова в системе.

Ниже - не сухой глоссарий. Это сеть. Каждый термин объясняется через соседей: с чем его чаще всего путают, где он появляется и какая проверка превращает слово в знание.

Узел 1. offset

`offset` - это смещение от начала выбранного объекта. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: file offset, RVA, stack offset, field offset, MMIO offset. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: сначала назови систему координат, потом число. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Путают offset с адресом и ищут байты не там. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 2. address

`address` - это номер места в адресном пространстве. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: VA, physical address, MMIO, base address, ASLR. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: проверь, кто задает карту: loader, MMU, linker script или reference manual. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Считают любое похожее число указателем. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 3. register

`register` - это быстрая ячейка внутри CPU. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: ABI, instruction, flags, debugger, interrupt context. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: смотри значение до инструкции и после нее. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Читают register без учета calling convention. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 4. stack

`stack` - это область памяти для возвратов, локальных данных и временного состояния. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: call, return, frame, unwind, interrupt stack. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: рисуй изменение SP/RSP по инструкциям. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Думают, что stack всегда имеет красивую цепочку rbp. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 5. function

`function` - это участок кода с договором входа и выхода. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: ABI, prologue, epilogue, return value, side effects. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: проверяй входы, выходы и измененную память. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Называют функцию по одной строке. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 6. ABI

`ABI` - это договор между кодом, compiler, linker и OS. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: registers, stack alignment, calling convention, object format. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: выбери ABI до чтения вызова. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Переносят правила Windows на Linux или ARM. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 7. loader

`loader` - это механизм, который превращает файл в образ памяти. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: sections, segments, relocations, imports, permissions. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: сравни файл, memory map и specification. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Считают, что файл копируется в память один к одному. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 8. runtime

`runtime` - это код и правила, которые обслуживают программу вокруг ее логики. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: startup, libc, CRT, GC, exceptions, scheduler. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: отделяй стандартную обвязку от предметной логики. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Анализируют startup как будто это алгоритм приложения. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 9. MMIO

`MMIO` - это память-похожий доступ к регистрам устройства. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: base address, register offset, bit field, volatile. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: проверяй manual и side effect. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Читают peripheral register как обычную RAM. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 10. unwind

`unwind` - это правила восстановления стека после call/exception. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: .pdata, .xdata, CFI, CFA, saved registers. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: сопоставляй metadata с prologue/epilogue. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Ждут frame pointer там, где compiler его убрал. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 11. bytecode

`bytecode` - это инструкции виртуальной машины, а не native CPU. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: IL, DEX, JVM, verifier, JIT. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: сначала читай virtual machine model, потом native output. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Сразу ищут x64 assembly и пропускают metadata. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Узел 12. hook

`hook` - это контролируемая точка наблюдения или перехвата. Если сказать еще проще, это способ не потерять координаты в низкоуровневом материале.

Связан с: Frida, debugger breakpoint, API trace, inline patch. Эти соседи важны потому, что реверсер почти никогда не видит термин в одиночестве. Например, address без memory map опасен, stack без ABI неполон, MMIO без reference manual почти бессмыслен, а bytecode без runtime model легко обманет красивым decompiler output.

Как проверять: записывай место, аргументы, момент и ограничения. Проверка должна быть написана так, чтобы другой человек смог повторить ее на том же объекте.

Типовая ошибка: Думают, что hook показывает все поведение программы. Исправление всегда одно: назвать систему координат, источник правила и границу вывода.

Как пользоваться картой

Когда встречаешь новый термин, не спрашивай только «как это переводится?». Спроси: где это живет, кто это создает, кто это читает, какой инструмент это показывает, какая документация задает правило, и что сломается, если я ошибусь. Такой вопрос превращает словарь в инженерное мышление.

Как проверять теорию по документации

Официальная документация в этой книге нужна не как украшение. Она задает правила: формат PE, ABI Windows x64, ELF gABI, DWARF, Arm/RISC-V ISA, UEFI, Android runtime, JVM, .NET, OpenOCD, QEMU, YARA, Frida и другие. Но документация сама по себе не делает вывод за исследователя. Она отвечает на вопрос "как устроено правило". Бинарный объект отвечает на вопрос "что мы наблюдаем". Теория связывает первое со вторым.

Если в тексте написано `RVA`, проверять нужно не перевод слова, а отношение RVA к ImageBase, section table и file offset. Если написано `ABI`, проверять нужно регистры аргументов, сохранение регистров, стек и возвращаемое значение. Если написано `MMIO`, проверять нужно memory map, base address периферии, register offset, bit field и side effects.

Такая проверка не превращает книгу в практикум. Это часть теории: научная фраза должна быть связана с правилом, иначе она остается красивым словом.

Приложение A. Большой словарь терминов

Этот словарь не заменяет главы. Он нужен как быстрый якорь: встретил английское слово - вернулся сюда, вспомнил смысл и связь с практикой.

Термин	Разжеванное объяснение
ABI	Application Binary Interface - правила уже скомпилированного кода: регистры аргументов, результат, стек, сохраняемые регистры, выравнивание и бинарная совместимость.
API	Application Programming Interface - правила для программиста на уровне исходного кода: какие функции вызвать, какие параметры передать, что получить.
address	Адрес - число, обозначающее место в некотором адресном пространстве. Нужно всегда уточнять: file offset, virtual address, physical address, MMIO address.
ASLR	Address Space Layout Randomization - рандомизация расположения модулей и областей памяти, чтобы адреса были менее предсказуемыми.
basic block	Базовый блок - последовательность инструкций с одним входом и одним выходом без внутренних ветвлений.
big-endian	Порядок байтов, где старший байт числа лежит раньше младшего.
binary	Бинарник - файл или образ, состоящий из машинных байтов: executable, library, firmware, object file.
bootloader	Загрузчик устройства или системы, который подготавливает среду и передает управление основной программе или ОС.
breakpoint	Точка останова, где debugger прерывает выполнение, чтобы исследователь мог посмотреть состояние.
caller	Код, который вызывает функцию и обычно готовит аргументы.
callee	Функция, которую вызвали; она выполняет работу и возвращает результат по правилам ABI.
calling convention	Соглашение о вызове функций: где аргументы, где результат, кто сохраняет регистры, как устроен стек.
CFG	Control Flow Graph - граф потока управления: базовые блоки и переходы между ними.
checksum	Контрольная сумма для обнаружения случайных ошибок данных; не равна криптографической подписи.
compiler	Компилятор - программа, которая переводит исходный код в более низкое представление и машинный код.
context switch	Переключение контекста: сохранение состояния одной задачи и восстановление другой.
data directory	Каталог важных таблиц PE: imports, exports, relocations, resources, exception data и другие.
decompiler	Декомпилятор - инструмент, который пытается поднять assembly в псевдокод высокого уровня.
disassembler	Дизассемблер - инструмент, который переводит машинные байты в assembly.
DMA	Direct Memory Access - механизм, позволяющий периферии переносить данные в память без постоянного участия CPU.
DWARF	Формат отладочной и unwind-информации, широко используемый в ELF/Linux/embedded.
ELF	Executable и Linkable Format - основной формат исполняемых файлов, библиотек и object files в Linux и многих embedded-сценариях.
entry point	Точка входа, адрес, куда loader передает управление после подготовки образа.
exception	Исключение: особое событие выполнения, требующее передачи управления обработчику или runtime-механизму.
file offset	Смещение от начала файла на диске; не то же самое, что virtual address в процессе.
firmware	Прошивка - программный образ для устройства, часто включающий код, данные, конфигурацию и иногда файловую систему.
flags	Флаги CPU - биты состояния после операций: ноль, перенос, знак, переполнение и другие.
frame pointer	Регистр-ориентир текущего stack frame, например rbp в классическом x86/x64 коде.
GOT	Global Offset Table - таблица адресов в ELF, используемая для динамической линковки.

Термин	Разжеванное объяснение
GPIO	General Purpose Input/Output - универсальные ножки микроконтроллера для входа и выхода.
handler	Обработчик события: exception handler, interrupt handler, signal handler, callback.
heap	Область динамической памяти, откуда выделяются блоки через malloc/new или похожие механизмы.
image base	Базовый виртуальный адрес, от которого считаются RVA в загруженном PE-образе.
immediate	Непосредственное значение, зашитое прямо в инструкцию, а не прочитанное из памяти.
interrupt	Прерывание - событие, которое временно передает управление обработчику, например от таймера или UART.
ISA	Instruction Set Architecture - архитектура набора инструкций: регистры, команды, режимы адресации, исключения.
JTAG	Аппаратный отладочный интерфейс для доступа к CPU/памяти/регистрам и boundary scan.
linker	Линковщик - соединяет object files и libraries, раскладывает секции, применяет символы и relocation rules.
linker script	Сценарий линковщика, особенно важный в embedded: задает, какие секции в какие адреса памяти попадут.
little-endian	Порядок байтов, где младший байт многобайтового числа лежит первым.
loader	Загрузчик, который отображает executable в память, применяет relocations, разрешает imports и запускает entry point.
MMIO	Memory-Mapped I/O - управление периферией через обычные чтения и записи по специальным адресам памяти.
object file	Объектный файл - промежуточный результат компиляции с кодом, секциями, символами и relocations.
offset	Смещение от начала чего-либо: файла, секции, структуры, стека. Без контекста слово почти бесполезно.
PE	Portable Executable - формат Windows EXE/DLL/драйверов с headers, sections, imports, relocations и data directories.
PLT	Procedure Linkage Table - переходники вызовов динамических функций в ELF.
prologue	Начало функции, где она готовит stack frame, сохраняет регистры и выделяет место.
raw firmware	Сырой образ прошивки без универсального заголовка; требует угадывания или знания base address и архитектуры.
register	Быстрое маленькое хранилище внутри CPU, например rax, rsp, rip, r0, sp, lr, a0.
relocation	Запись или процесс исправления адресов, когда окончательное расположение кода/данных стало известно.
return address	Адрес, куда функция должна вернуться после выполнения.
RIP-relative	Адресация относительно текущей инструкции на x86-64; часто используется для доступа к глобальным данным.
RTOS	Real-Time Operating System - ОС реального времени для embedded, где важны задачи, задержки и предсказуемость.
RVA	Relative Virtual Address - виртуальный адрес относительно image base, часто в PE.
section	Именованный участок файла или образа: код, данные, таблицы, ресурсы, отладочная информация.
segment	Участок, который loader отображает в память; в ELF program headers описывают сегменты.
shadow space	32 байта, которые caller резервирует на стеке в Windows x64 для регистровых аргументов callee.
stack	Стек - область временной памяти вызовов функций, обычно содержит return address, сохраненные регистры и локальные данные.
stack frame	Кадр стека одного вызова функции.
symbol	Имя, связанное с адресом функции, переменной, секции или другого объекта.
syscall	Системный вызов - переход из пользовательского кода к сервису ядра ОС.

Термин	Разжеванное объяснение
`TLS`	Thread Local Storage - данные, отдельные для каждого потока; в PE также важны TLS callbacks.
`unwind`	Разматывание стека: восстановление предыдущих кадров, регистров и пути вызовов, особенно при exceptions.
`VA`	Virtual Address - виртуальный адрес внутри адресного пространства процесса.
`vector table`	Таблица адресов обработчиков exceptions/interrupts, особенно важная в Cortex-M.
`volatile`	Ключевое слово C, запрещающее компилятору считать чтение/запись обычной оптимизируемой памятью; нужно для MMIO.
`watchpoint`	Точка наблюдения, которая останавливает выполнение при чтении или записи конкретного адреса.
`xref`	Cross-reference - ссылка на место, где адрес, функция, строка или данные используются.

Карточки ключевых понятий

Ниже - не сухой перевод, а рабочее объяснение: где термин проявляется, с чем связан и какая ошибка чаще всего ломает анализ.

ABI

Буквально это «бинарный интерфейс приложения». Слово binary здесь ключевое: речь уже не о том, как программист написал функцию, а о том, как готовый машинный код договорился передавать данные.

Где увидеть: Видно в регистрах перед `call`, в форме stack frame, в том, какие регистры функция сохраняет, и в том, где появляется результат после возврата.

Связь с другими понятиями: Связывает компилятор, linker, loader, ОС, debugger и реверсера. Если ABI понят неверно, аргументы функций читаются неправильно.

Типовая ошибка и проверка: Типовая ошибка - переносить Windows x64 правила на Linux x86-64 или ARM. Проверка: выпиши регистры аргументов до вызова и сравни с официальным ABI.

offset

Offset - это не адрес, а расстояние от выбранного начала. Само слово без уточнения почти ничего не значит.

Где увидеть: В PE это может быть file offset или RVA, в структуре - смещение поля, в stack frame - расстояние от `rbp/rsp`, в MMIO - смещение регистра от base.

Связь с другими понятиями: Связан с address, section, structure, stack, data directory и linker script.

Типовая ошибка и проверка: Ошибка - искать RVA как file offset. Проверка: всегда пиши «offset от чего именно» и делай преобразование через таблицу секций или карту памяти.

stack

Stack - временная память вызовов, где программа хранит дорогу назад и рабочие данные текущей функции.

Где увидеть: Видно через `rsp/sp`, `push/pop`, `sub rsp`, сохраненные регистры, return address, локальные переменные.

Связь с другими понятиями: Связан с function call, return address, ABI, unwind, exceptions, RTOS tasks.

Типовая ошибка и проверка: Ошибка - считать, что стек всегда выглядит как `push rbp; mov rbp,rsp`. В оптимизированном коде frame pointer может отсутствовать.

register

Register - маленькое быстрое хранилище внутри CPU. Это не переменная исходного кода, а часть состояния процессора.

Где увидеть: Видно в каждой инструкции: `rax`, `rcx`, `rip`, `rsp`, `r0`, `lr`, `pc`, `a0`, `ra`.

Связь с другими понятиями: Связан с ISA, ABI, calling convention, flags, instruction execution.

Типовая ошибка и проверка: Ошибка - думать, что один регистр всегда означает одну переменную. Компилятор переиспользует регистры.

RVA

Relative Virtual Address - адрес относительно image base. В PE это один из главных способов ссылаться на данные внутри образа.

Где увидеть: Видно в Optional Header, data directories, import/export tables, exception directory.

Связь с другими понятиями: Связан с VA, image base, section table, loader, relocations.

Типовая ошибка и проверка: Ошибка - прибавлять RVA к началу файла. Проверка: найди секцию, вычисли section offset, прибавь PointerToRawData.

VA

Virtual Address - адрес внутри виртуального адресного пространства процесса. Программа работает именно с ним, а не с file offset.

Где увидеть: Видно в debugger, memory map, disassembler listing после загрузки image base.

Связь с другими понятиями: Связан с OS loader, MMU, pages, ASLR, modules, heap, stack.

Типовая ошибка и проверка: Ошибка - путать VA из debugger с положением байта на диске. Проверка: VA минус image base дает RVA, а RVA уже переводится в file offset.

MMIO

Memory-Mapped I/O - периферия, притворяющаяся памятью. CPU читает и пишет по адресу, но на другом конце аппаратный блок.

Где увидеть: Видно по адресам вроде `0x40000000`, read-modify-write, битовым маскам, обращениям к register base + offset.

Связь с другими понятиями: Связан с volatile, peripheral, reference manual, clocks, interrupts, DMA.

Типовая ошибка и проверка: Ошибка - считать MMIO обычной RAM-переменной. Проверка: найди адрес в memory map/reference manual и подпиши register/bit.

loader

Loader - тот, кто превращает файл в работающий образ памяти. Он не просто копирует байты, а применяет правила формата.

Где увидеть: В PE виден через mapping sections, imports, relocations, TLS callbacks. В ELF - через program headers, interpreter, dynamic linker.

Связь с другими понятиями: Связан с PE/ELF, relocations, imports, entry point, ASLR.

Типовая ошибка и проверка: Ошибка - думать, что файл на диске и образ в памяти совпадают байт-в-байт. Проверка: сравни raw section layout и virtual mapping.

linker

Linker соединяет object files в единый binary и решает, где окажутся секции и символы.

Где увидеть: Видно по section layout, symbol table, relocation records, map files, linker script.

Связь с другими понятиями: Связан с compiler, object file, sections, symbols, embedded memory map.

Типовая ошибка и проверка: Ошибка - приписывать linker-обвязку автору приложения. Проверка: отличай стандартные startup/runtime секции от пользовательской логики.

relocation

Relocation - запись о месте, где адрес нужно исправить после выбора реального расположения кода или данных.

Где увидеть: В PE видна в base relocation table, в ELF - в `.rel/.rela` sections и dynamic relocations.

Связь с другими понятиями: Связана с ASLR, linker, loader, GOT, image base, symbols.

Типовая ошибка и проверка: Ошибка - считать адреса в файле окончательными. Проверка: посмотри, применялись ли relocations при загрузке.

unwind

Unwind - разматывание стека назад: восстановить предыдущий frame и регистры, чтобы понять цепочку вызовов или обработать exception.

Где увидеть: В Windows x64 видно через `.pdata/.xdata`, Runtime Function, UNWIND_INFO. В ELF - через `.eh_frame` и DWARF CFI.

Связь с другими понятиями: Связан с stack, ABI, exception handling, prologue, debugger stack trace.

Типовая ошибка и проверка: Ошибка - думать, что stack trace всегда истина. Проверка: сопоставь unwind metadata с реальным прологом функции.

prologue

Prologue - начало функции, где она готовит себя к работе: сохраняет регистры, выделяет стек, настраивает frame.

Где увидеть: Видно как `push`, `sub rsp`, сохранение nonvolatile registers, иногда установка `rbp`.

Связь с другими понятиями: Связан с ABI, unwind, stack frame, compiler optimization.

Типовая ошибка и проверка: Ошибка - ждать одинаковый prologue у всех функций. Проверка: учитывай optimization level и leaf functions.

epilogue

Epilogue - конец функции, где она возвращает стек и регистры в состояние caller.

Где увидеть: Видно как `add rsp`, `pop`, `ret`, иногда tail jump.

Связь с другими понятиями: Связан с prologue, return address, ABI, unwind restrictions.

Типовая ошибка и проверка: Ошибка - принимать любой `jmp` в конце за обычную ветку. Иногда это tail call.

interrupt

Interrupt - событие, которое временно отрывает CPU от текущего кода и ведет в handler.

Где увидеть: В embedded видно через vector table, NVIC, IRQ handlers, peripheral status flags.

Связь с другими понятиями: Связан с handler, vector table, RTOS, UART, timers, DMA.

Типовая ошибка и проверка: Ошибка - искать вызов interrupt handler как обычный `call`. Его вызывает аппаратный механизм.

handler

Handler - обработчик события. Смысл зависит от слоя: interrupt handler, exception handler, signal handler, callback.

Где увидеть: Видно как адрес в vector table, запись в exception metadata, callback pointer, function pointer table.

Связь с другими понятиями: Связан с events, interrupts, exceptions, callbacks, runtime.

Типовая ошибка и проверка: Ошибка - не уточнять, кто именно вызывает handler. Проверка: определи механизм вызова.

firmware

Firmware - программный образ устройства. Он может быть raw binary, контейнером обновления, ELF, HEX, rootfs или смесью частей.

Где увидеть: Видно в update-файлах, Flash dump, bootloader partitions, embedded Linux images.

Связь с другими понятиями: Связан с bootloader, memory map, MMIO, vector table, filesystem, signatures.

Типовая ошибка и проверка: Ошибка - загружать raw firmware в Ghidra как обычный executable без base address и архитектуры.

vector table

Vector table - таблица адресов обработчиков exceptions/interrupts. На Cortex-M первые два слова особенно важны: stack pointer и reset handler.

Где увидеть: Видно в начале firmware: значение SRAM-диапазона, затем Flash-адрес с Thumb bit.

Связь с другими понятиями: Связана с reset, interrupts, Cortex-M, startup code, base address.

Типовая ошибка и проверка: Ошибка - считать reset vector нечетным «битым адресом». Младший бит часто означает Thumb-state.

volatile

Volatile в C говорит компилятору: это значение нельзя оптимизировать как обычную память, оно может измениться извне.

Где увидеть: В embedded встречается вокруг MMIO-регистров и shared данных с interrupt handlers.

Связь с другими понятиями: Связан с compiler optimization, MMIO, interrupts, hardware registers.

Типовая ошибка и проверка: Ошибка - думать, что volatile делает операцию атомарной. Он не заменяет locks, barriers и critical sections.

DMA

DMA позволяет периферии переносить данные без каждой инструкции CPU.

Где увидеть: Видно по настройке source, destination, length, enable bits и interrupt on complete.

Связь с другими понятиями: Связан с UART/SPI/ADC, buffers, cache coherency, interrupts.

Типовая ошибка и проверка: Ошибка - искать CPU-store в буфер, который заполняет DMA. Проверка: найди настройку DMA channel.

GOT

GOT - таблица адресов, через которую ELF-код обращается к внешним функциям и глобальным данным.

Где увидеть: Видно в disassembly как обращения к memory slots, которые dynamic linker заполняет реальными адресами.

Связь с другими понятиями: Связана с PLT, dynamic linker, relocations, shared libraries.

Типовая ошибка и проверка: Ошибка - принимать GOT-entry за обычную переменную без учета dynamic linking.

PLT

PLT - набор переходников для вызова внешних функций в ELF.

Где увидеть: Видно как `puts@plt`, `printf@plt`, маленькие stub-функции перед переходом через GOT.

Связь с другими понятиями: Связана с GOT, lazy binding, dynamic relocations.

Типовая ошибка и проверка: Ошибка - думать, что `call puts@plt` сразу ведет в libc-функцию без промежуточного механизма.

symbol

Symbol - имя, связанное с адресом. Это подсказка, но не абсолютная истина.

Где увидеть: Видно в symbol tables, PDB, DWARF, map files, exports.

Связь с другими понятиями: Связан с linker, debug info, names, xrefs, reports.

Типовая ошибка и проверка: Ошибка - верить имени без проверки поведения. Символ может быть отсутствующим, старым или misleading.

xref

Xref - перекрестная ссылка: кто использует эту строку, функцию, адрес или данные.

Где увидеть: Видно в Ghidra/IDA как references to/from.

Связь с другими понятиями: Связан с strings, call graph, data flow, function discovery.

Типовая ошибка и проверка: Ошибка - смотреть только на саму строку и не идти по xrefs к коду, который ее использует.

entry point

Entry point - адрес, куда loader передает управление после подготовки образа.

Где увидеть: В PE лежит как AddressOfEntryPoint RVA, в ELF - e_entry, в Cortex-M reset handler играет похожую стартовую роль.

Связь с другими понятиями: Связан с loader, runtime, startup code, main.

Типовая ошибка и проверка: Ошибка - думать, что entry point всегда равен `main`. Обычно перед `main` работает runtime.

TLS

Thread Local Storage - данные потока; в PE важны еще TLS callbacks, которые могут выполняться до обычного entry point.

Где увидеть: Видно в PE TLS directory и callbacks array.

Связь с другими понятиями: Связано с loader, initialization, anti-debug tricks, runtime.

Типовая ошибка и проверка: Ошибка - начинать анализ только с AddressOfEntryPoint и пропускать TLS callbacks.

heap

Heap - область динамической памяти для объектов, размер и время жизни которых неизвестны заранее.

Где увидеть: Видно через `malloc/free`, `new/delete`, HeapAlloc, custom allocators.

Связь с другими понятиями: Связан с pointers, structures, use-after-free, buffers.

Типовая ошибка и проверка: Ошибка - думать, что все локальные данные лежат на stack. Большие и долгоживущие объекты часто в heap.

syscall

Syscall - системный вызов, переход к ядру ОС за услугой: файл, сеть, память, процесс.

Где увидеть: В Linux видно как `syscall` instruction или wrappers libc, в Windows часто через ntdll и Native API.

Связь с другими понятиями: Связан с OS boundary, sandboxing, tracing, dynamic analysis.

Типовая ошибка и проверка: Ошибка - смотреть только на high-level imports и не замечать прямые syscalls или thin wrappers.

watchpoint

Watchpoint останавливает выполнение при доступе к адресу памяти.

Где увидеть: Видно в debugger/hardware debug как data breakpoint на read/write.

Связь с другими понятиями: Связан с debugger, memory corruption, MMIO, data flow.

Типовая ошибка и проверка: Ошибка - ставить breakpoint на функцию, когда нужно поймать запись в конкретный buffer.

raw firmware

Raw firmware - просто байты без универсальной самодокументации.

Где увидеть: Видно как `.bin`, где нет PE/ELF headers, но могут быть vector table, strings, tables, machine code.

Связь с другими понятиями: Связан с base address, architecture, endian, memory map.

Типовая ошибка и проверка: Ошибка - ждать от raw-файла, что он сам скажет, куда его загрузить.

RTOS

RTOS - маленькая ОС реального времени, где важны tasks, scheduler и предсказуемые задержки.

Где увидеть: Видно по task creation, queues, semaphores, scheduler start, context switch code.

Связь с другими понятиями: Связана с interrupts, stack per task, embedded behavior.

Типовая ошибка и проверка: Ошибка - искать всю логику в `main`, когда реальное поведение живет в задачах.

Матрица проверки фактов и источников

Эта матрица нужна, чтобы книга не выглядела как набор уверенных утверждений без опоры. Она не заменяет документацию, но показывает, куда возвращаться, когда в исследовании появляется конкретный вопрос.

Источник	Ссылка	Что проверять	Как превращать в доказательство
Microsoft PE/COFF specification	https://learn.microsoft.com/en-us/windows/win32/debug/pe-format	PE, COFF, RVA, VA, sections, imports, exports, relocations	Любой вывод о PE-поле сверять с именем поля, размером, границей секции и правилом RVA-to-file-offset.
Microsoft x64 calling convention	https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention?view=msvc-170	Windows x64 ABI, регистры аргументов, shadow space, volatile/nonvolatile	Аргументы и возвращаемые значения подтверждать состоянием регистров до call и после return.
Microsoft x64 exception handling	https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64?view=msvc-170	.pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO	Unwind-вывод проверять сопоставлением prologue, unwind codes и диапазона функции.
System V ABI / ELF gABI	https://refspecs.linuxfoundation.org/elf/gabi4+/contents.html	ELF object files, program loading, dynamic linking	В Linux начинать загрузочную модель с program headers, а не с красивых section names.
x86-64 psABI	https://gitlab.com/x86-psABIs/x86-64-ABI	System V AMD64 ABI, Linux/macOS register calling convention	Не переносить Windows-регистры аргументов на Linux: проверять ABI перед чтением вызова.
DWARF Standards Committee	https://dwarfstd.org/	debug info, CFI, CFA, line info, DIE	Debug/unwind-данные читать как правила восстановления состояния, а не как гарантированный исходный код.
Intel Software Developer Manuals	https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html	x86/x64 ISA, paging, exceptions, instruction semantics	Смысл инструкции проверять по ISA, особенно флаги, порядок операций с памятью и привилегированное поведение.
Arm Architecture Reference Manual	https://developer.arm.com/documentation/ddi0487/latest/	AArch64 architecture, exceptions, registers	Для ARM не угадывать по x86-привычкам: проверять режим, набор инструкций и уровень исключения.
Arm Cortex-M documentation	https://developer.arm.com/documentation/	Cortex-M vector table, exceptions, NVIC, memory map	Reset vector, Thumb bit и interrupt handlers проверять по архитектурному руководству и reference manual конкретного устройства.
RISC-V ratified specifications	https://riscv.org/specifications/ratified/	RISC-V ISA, privileged architecture, extensions	Всегда указывать ISA profile/extensions, иначе смысл инструкции может быть неполным.
Apple Mach-O documentation	https://developer.apple.com/documentation/kernel/mach-o	Mach-O, load commands, Apple binary format	Для Apple-бинарника начинать с magic, CPU type и load commands, затем смотреть контекст dyld и code signing.
Android runtime и Dalvik / ART	https://source.android.com/docs/core/runtime	Android runtime, DEX, ART, managed/native boundary	В Android разделять Java/Kotlin bytecode, результат работы ART, JNI native code и поведение Binder.
Microsoft .NET managed code	https://learn.microsoft.com/en-us/dotnet/standard/managed-code	.NET managed code, runtime services, CLR	Managed-бинарник проверять через metadata, IL и поведение runtime/JIT, а не только через native dump.
Oracle Java SE specifications	https://docs.oracle.com/javase/specs/	Java Language Specification и JVM Specification	JVM-вывод проверять по class file, constant pool, operand stack и модели verifier.

Источник	Ссылка	Что проверять	Как превращать в доказательство
UEFI Forum specifications	https://uefi.org/specifications	UEFI, ACPI, Platform Initialization, boot services	Boot-chain выводы привязывать к фазе PEI/DXE/BDS и текущей версии UEFI/PI.
OpenOCD User Guide	https://openocd.org/doc/html/index.html	JTAG, SWD, flash programming, debug adapters	Hardware debug действия описывать через adapter, target, reset strategy и flash command.
QEMU System Emulation docs	https://www.qemu.org/docs/master/system/	system emulation, machine model, devices	Эмуляцию не считать реальным железом: явно писать, какая machine/device model проверялась.
Frida documentation	https://frida.re/docs/home/	dynamic instrumentation, hooks, process inspection	Instrumentation-вывод подтверждать тем, где именно стоит hook и какие arguments/events он зафиксировал.
YARA documentation	https://yara.readthedocs.io/en/stable/	pattern rules, strings, modules, conditions	YARA-правило доказывает совпадение признаков, а не полное поведение; это triage, а не verdict.
Mandiant capa documentation	https://mandiant.github.io/capa/	capability extraction from executable files	capa помогает найти возможности, но rule match нужно проверять в дизассемблере и явно описывать ограничения вывода.
angr documentation	https://docs.angr.io/	symbolic execution, CFG, program state	Symbolic execution требует модели входов, памяти и external calls; без модели результат ограничен.
LLVM libFuzzer documentation	https://llvm.org/docs/LibFuzzer.html	coverage-guided fuzz testing	Fuzzing доказывает найденные падения и достигнутое coverage, но не доказывает отсутствие ошибок.

Как читать матрицу

Если источник официальный, это еще не означает, что любой вывод автоматически доказан. Нужно показать связь: конкретное поле, конкретный register, конкретный instruction, конкретный loader step, конкретный trace. Документация дает правило; артефакт дает наблюдение; исследователь связывает их проверкой.

Если источник не официальный, например книга или tool documentation, его роль другая. Книга дает педагогическую модель и практический маршрут. Tool documentation explains tool behavior. Но нормативное правило binary format, ABI, ISA или platform policy лучше сверять с primary documentation.

Приложение B. Официальные источники и книги для углубления

Официальная документация нужна не для того, чтобы читать ее как роман. Ее читают как карту правил. Когда бинарник задает конкретный вопрос - где аргумент, как устроен header, что значит bit, как loader применяет relocation - нужно возвращаться к первоисточнику.

Официальная документация

Источник	Ссылка	Для чего читать
Microsoft PE format	https://learn.microsoft.com/en-us/windows/win32/debug/pe-format	Официальная структура PE/COFF, headers, sections, data directories.
Microsoft x64 calling convention	https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention	Правила аргументов, регистров, shadow space и результата в Windows x64.
Microsoft x64 stack usage	https://learn.microsoft.com/en-us/cpp/build/stack-usage	Стек, alignment, frame functions и правила для Windows x64.
Microsoft x64 prolog и epilog	https://learn.microsoft.com/en-us/cpp/build/prolog-and-epilog	Ограничения пролога/эпилога, связанные с unwind.
Microsoft x64 exception handling	https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64	Runtime function entries и unwind data Windows x64.
Intel 64 и IA-32 SDM	https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html	Официальные руководства по x86/x64 ISA, registers, exceptions, memory.
AMD64 Architecture Programmer's Manual	https://www.amd.com/en/support/tech-docs/amd64-architecture-programmers-manual-volumes-1-5	Архитектура AMD64 и системные детали.
System V AMD64 ABI	https://gitlab.com/x86-psABIs/x86-64-ABI	ABI Linux/x86-64: аргументы, stack, ELF-соглашения.
ELF gABI	https://refspecs.linuxfoundation.org/elf/gabi4+/contents.html	Общее описание ELF: headers, segments, sections, relocations.
Arm ABI	https://github.com/ARM-software/abi-aa	AAPCS и другие ABI-документы Arm.
Arm CMSIS-Core	https://arm-software.github.io/CMSIS_5/Core/html/index.html	Cortex-M core support, registers, exceptions, NVIC.
Arm Developer Documentation	https://developer.arm.com/documentation	Официальные руководства по Cortex-M/Cortex-A и архитектуре Arm.
RISC-V specifications	https://riscv.org/technical/specifications/	Официальные спецификации RISC-V ISA и privileged architecture.
STMicroelectronics STM32 MCU docs	https://www.st.com/en/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus.html	Datasheets/reference manuals для STM32.
Espressif ESP-IDF	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/	Официальная документация ESP32/ESP-IDF.
OpenOCD User's Guide	https://openocd.org/doc/html/index.html	JTAG/SWD debugging, GDB server, flash programming.
QEMU system emulation	https://qemu.readthedocs.io/en/master/system/index.html	Эмуляция систем и архитектур.
Ghidra official site	https://ghidra-sre.org/	Официальная страница Ghidra SRE.
Rizin book	https://book.rizin.re/	Открытая документация по Rizin reverse engineering framework.
DWARF Debugging Standard	https://dwarfstd.org/	Стандарт отладочной и unwind-информации DWARF.
Zephyr Project Documentation	https://docs.zephyrproject.org/latest/	RTOS Zephyr, drivers, device model, embedded development.
FreeRTOS Documentation	https://www.freertos.org/Documentation/RTOS_book.html	RTOS concepts: tasks, queues, semaphores, scheduling.
U-Boot Documentation	https://docs.u-boot.org/en/latest/	Bootloader для embedded/Linux-устройств.

Книги и большие курсы, на которые стоит опираться

Книга	Чем полезна
Eldad Eilam, Reversing: Secrets of Reverse Engineering	Классическая книга по мышлению реверсера, чтению assembly и восстановлению логики программ.
Chris Eagle, The IDA Pro Book	Практическая школа интерактивного анализа в IDA: функции, xrefs, типы, скрипты.
Michael Sikorski, Andrew Honig, Practical Malware Analysis	Хорошая методика статического и динамического анализа Windows-образцов в лаборатории.
Randall Hyde, The Art of Assembly Language	Полезна для понимания assembly как языка работы с памятью, регистрами и инструкциями.
Bryant, O'Hallaron, Computer Systems: A Programmer's Perspective	Сильная база по тому, как программы выглядят на уровне машины, памяти, linking и процессов.
David A. Patterson, John L. Hennessy, Computer Organization и Design	Архитектурная база: инструкции, процессор, память, представление данных.
Jonathan Corbet, Alessandro Rubini, Greg Kroah-Hartman, Linux Device Drivers	Полезна для понимания драйверного мышления и связи кода с устройствами.
Jean J. Labrosse, MicroC/OS-II и embedded RTOS materials	Классическая линия для понимания RTOS, задач и контекстов в embedded.

Важное правило: не копируй стиль одной книги слепо. Сильный учебник по реверсу должен соединять несколько традиций: системное программирование, архитектуру компьютера, форматы файлов, отладку, embedded-железо и культуру доказательного отчета.